



PHD

Type Systems for Probabilistic lambda-Calculi

Majury, Georgi

Award date:
2025

Awarding institution:
University of Bath

[Link to publication](#)

Alternative formats

If you require this document in an alternative format, please contact:
openaccess@bath.ac.uk

Copyright of this thesis rests with the author. Access is subject to the above licence, if given. If no licence is specified above, original content in this thesis is licensed under the terms of the Creative Commons Attribution-NonCommercial-NoDerivs 4.0 International (CC BY-NC-ND 4.0) Licence (<https://creativecommons.org/licenses/by-nc-nd/4.0/>). Any third-party copyright material present remains the property of its respective owner(s) and is licensed under its existing terms.

Take down policy

If you consider content within Bath's Research Portal to be in breach of UK law, please contact: openaccess@bath.ac.uk with the details. Your claim will be investigated and, where appropriate, the item will be removed from public view as soon as possible.

Type Systems for Probabilistic λ -Calculus

submitted by

Georgina Majury

for the degree of Doctor of Philosophy

of the

University of Bath

Department of Computer Science

December 2024

COPYRIGHT

Attention is drawn to the fact that copyright of this thesis rests with the author. A copy of this thesis has been supplied on condition that anyone who consults it is understood to recognise that its copyright rests with the author and that they must not copy it or use material from it except as permitted by law or with the consent of the author.

This thesis may be made available for consultation
within the University Library and may be photocopied
or lent to other libraries for the purposes of
consultation with effect from.....(date)

Signed on behalf of the Faculty of Science

Declarations

The material presented here for examination for the award of a research degree by research has not been incorporated into a submission for another degree.

I am the author of this thesis, and the work described therein was carried out by myself personally, in collaboration with my supervisor.

Georgina Majury

Summary

This thesis is an exploration in two parts dealing with the nature of termination in probabilistically-branching variants of the lambda calculus.

The first part approaches the issue of probabilistic-termination of the Probabilistic Event lambda-calculus – a confluent probabilistic calculus with the effect decomposed into choice and event-generation – via a novel and simple type system. The thesis goes on to explore the semantics of this calculus through a machine operated by a sequential extension. A normalisation result on the terms of this extension, typed by probabilistically-parameterised stack transformations, reflects back onto the original system, giving the desired termination result.

The second part presents an unfinished investigation into almost-sure termination via sequential calculus, wherein we define an annotated two-sided sequent calculus, and outline its relationship to co-recursive loops and to continuations. Termination of the non-recursive fragment is shown via an embedding in a known natural deduction formulation, while the result for the full system – which corresponds to almost-sure termination – is left as a conjecture from the given probabilistic variant of the type system.

Acknowledgements

This thesis has been the culmination of years of perseverance and I have been grateful for the support of many, many people. I cannot possibly thank them all for their kindnesses, great and small. I would like to take this opportunity to mention those who have had a specific impact on my life and work.

My supervisor, Willem Heijltjes, has had a massive role in shaping this work. He took my ideas on probabilistic computation and reframed them into a suitable thesis topic which it has been a joy to work on. Throughout the write-up he provided detailed and helpful feedback and gave me confidence in my writing. Despite the challenges that these years have posed he has always been available to support me whether our meetings were a quick knock on the door, a zoom call or a socially distanced walk in the park. I am incredibly grateful for his unlimited reserves of patience and understanding.

The enthusiasm and irreverence of my second supervisor, the late Alessio Guglielmi, drew me to Bath in the first place. Whether over coffee and cake, walking with gelato, or shopping for vitamins, Alessio always had a willing ear to listen to my worries: fresh and ancient. Around him formed a community of fun, nurturing and inquisitiveness that I am honoured to be part of. It saddens me greatly that I will never hear his thoughts on this work.

I would like to thank my Examiners Alex Kavvos and Guy McCusker for their thoroughness and humour which combined to make the viva a pleasant experience. I was set in good stead for the viva by the rigorous and extended supervisions of my undergraduate director of studies, Andras Zsak.

While living in Bath my longtime flatmate and friend, Maryam, kept me sane, helped me prioritise, and has been my role model for persistent and structured thesis writing. My contemporaries: Chris, Torie, Dan, Hollie, Keji and Cameron; provided many interesting, amusing and illuminating discussions. I would also like to thank all the friends and family that have cajoled me through the write-up and helped hunt down typos and rogue punctuation and the staff and students of Nendrum College for their support and energy.

Contents

1	Introduction	5
1.1	The lambda-Calculus	6
1.2	Type Theory	9
1.3	The Probability Problem	12
1.4	Probabilistic lambda-Calculi	13
1.5	Sequential lambda-Calculus	15
1.6	Continuations	18
1.7	Outline	19
I	Probabilistic Termination	21
2	The Probabilistic Event lambda-Calculus	23
2.1	Simple Types	28
3	A Type System for Probabilistic Termination	31
3.1	Probabilistic Types	32
3.2	The Rules in Detail	36
3.2.1	Events in Context	39
3.3	Main Theorem	40
3.4	Comparing to Counting Quantifiers	41
4	The Machine	53
4.1	Sequencing	54
4.1.1	Call-by-Value	58
4.2	Sequential Probabilistic Types	60

4.3	Big-Step Semantics	64
4.4	An Abstract Machine	70
4.5	Probabilistic Machine Termination	73
5	Termination	75
5.1	Weighted Evaluation	78
5.2	Head normalisation	86
II	Towards Almost-Sure Termination	89
6	The Join Calculus	91
6.1	Multi-conclusion Sequent Calculi	93
6.2	The Term Calculus	95
6.3	Term Assignment System	96
6.3.1	Diagram Intuition	99
6.4	Natural Deduction	101
6.5	Lambda-mu	105
7	Reductions	111
7.1	Beta Reduction Rules	112
7.2	Derivation Reduction	113
7.3	Non-Determinism	117
7.4	Permutations	118
7.5	Omissions	121
8	Towards Almost-Sure Termination	127
8.1	Co-recursion	128
8.2	Type System	130
8.3	Conclusion	133
	Bibliography	135
A	Machine Semantics for the Join Calculus	143

Chapter 1

Introduction

Theoretical computer science has its roots long before the advent of anything we would recognise as a modern computer. From Babbage’s Difference Engine inspiring Ada Lovelace to give a farsighted prediction of modern information encoding, to Turing and Church reasoning about the abilities of a hypothetical indefatigable human computer, the theoretical aspects of computer science have always led the practical.

During that early era computation was defined as a process that is “determined, applicable, effective and finite” in principle the same as the definitions of computation used today. There is, however, a growing interest in programs with non-standard operators. My particular interest lies in programs where the *determined* and *finite* conditions are loosened, which I will explore in this thesis by looking at extensions of the standard computational model.

Computation has, historically, been modelled in many ways. These definitions often take computation to be, by its nature, deterministic. This is of course desirable: we want to know that when we assign a task the answer we get will be correct, which for the most part means unique and replicable. For this reason non-determinism was initially shunned, seen as a negative effect to be abstracted away when talking about ideal machines.

“...we choose to ignore this unsavoury phenomenon, arguing that the

deterministic theory is at once simpler and more interesting, and can in any case be made to include the probabilistic theory by the simple device of adding new input facilities accessible only to gremlins.”

Regular Algebra for Finite Machines

John H. Conway

Probabilistic machines can be encoded by a universal computer [66], however, they are not computationally equivalent to their deterministic cousins [30]. There have been many advances in computational efficiency due to the inclusion of (psuedo)-probabilistic elements into programs. For instance, modern factoring algorithms use random choice to significantly decrease expected run time, and many NP problems are best addressed by checking if a randomly generated solution satisfies the constraints. Probability is also used extensively in AI where data is grouped and new inputs are assessed on their probability of belonging to a particular group, or outputs are randomly generated according to the distribution of the group.

1.1 The lambda-Calculus

Church’s thesis states that *Every intuitively computable function is recursive* [11]; this forms a link between programs and recursive functions, but how are recursive functions defined? In order to give a formal definition to the most general recursive functions Church created the λ -calculus: a formal theory that describes all objects with inputs and determined outputs, focusing on their composition.

The calculus was shown [71] to be equivalent to Turing machines, completing the link between functions and computation. The untyped λ -calculus provides an alternative to Turing machines as a universal model of computation. Although it is a simple concept, with a fairly intuitive presentation, it has become a backbone to the study of computation.

Despite the algebraic theory being simple enough to define in a couple of lines, the expressive power of the lambda calculus is impressive. The terms of the calculus are, from the outset, higher order: functions which operate on functions. This makes it the perfect starting point for designing functional programming

languages such as Lisp. Unencumbered by the notion of a physical process the calculus is also easily extended, making it ideal for the introduction and study of new, non-standard connectives.

With this in mind, let us look at the components of the standard λ -calculus. The simplest building block of a term is the variable which can be thought of as standing in for some externally-defined term, or simply as algebraic formalism. The lambda calculus is designed to model functions, these are formed via abstraction. Abstraction allows us to take any term and change any variable into an input. So $\lambda x.N$ means the function which takes in an x and uses it in the evaluation of N . Finally there is application: this construct allows us to use our functions, feeding them arguments to use in the evaluation. The meaning of NM therefore is to apply the function N to the term M , or equivalently to use M as the first argument in the evaluation of N . The following grammar gives these rules for term generation, and is used to declare that no other term constructions are available:

$$M, N := x \mid \lambda x.N \mid NM$$

Understanding of these terms is gained through consideration of their reductions, which follow the same motivation of definition. Where we have an application with enough knowledge of the function $(\lambda x.N)M$ we can start to evaluate it by passing in the argument M , to be used in N in place of x , the first abstracted variable. Formally this is written $(\lambda x.N)M \rightarrow [M/x]N$ where $[M/x]N$ is the *capture-avoiding* substitution of M for x in N . Capture-avoidance prevents free variables from becoming bound during substitution, which would fundamentally alter the meaning of a term.

During beta reduction if a variable bound to the active binder occurs within the scope of another abstraction, the argument which assumes that place under substitution may contain a variable which is (nominally) the same as the inner binder. Morally, however, they are not truly the same variable; since in the overall term they have different binders (or perhaps none at all). From this arises the requirement for a capture avoiding condition on substitution. Ideally variables with different meaning would have different names, however this is not always the case. We may need to rename some variables during substitution. Free variables

cannot be changed, since they are controlled externally to the (sub)term, so we must work with the bound variables. It does not matter what abstracted variable a function is defined in terms of: as long as the name of the binder and all instances within its scope change concurrently there will be no problems. The process of renaming a bound variable is known as *alpha-conversion*. Terms which are equivalent up to alpha-conversion have the same meaning and are often treated as indistinguishable.

Equational reasoning refers to the process of evaluating a term by replacing sub-terms with equivalent terms. An equational theory can be established for the lambda calculus in which terms are considered equal if they have the same “black box” behaviour. The main disadvantage of equational reasoning as a means of evaluation is that there are many ways in which to replace a term, not all of them helpful. For this reason we more usually look at rewrite rules, in which a direction is placed on the substitutions, moving in principle from more to less complex terms. A disadvantage of an approach based on rewrite-rules is that you cannot undo steps because the reverse of a rewrite step is generally not a rewrite. What happens if we do reductions in the wrong order? Is there a *wrong order*?

Thankfully the answer to this question is *no*: it is true that there are differences in efficiency between different reduction paths¹ but due to the *Church-Rosser* property, we can never end up stuck as the result of a reduction decision. Sometimes known as the diamond property, this states that if two reductions are applied to a term then there are subsequent reduction steps for each outcome which result in the same final term. The λ -calculus has confluence of reduction.

Now that we know that it is not necessary to be clever about order of reduction it may be possible to fix a *strategy*: a set of rules which determine which of the redexes to evaluate next. Strategies remove the non-determinism from evaluation of terms and change the lambda calculus from a model of computation into a programming language. There are several common reduction strategies, each with their own advantages and pitfalls: call-by-name, call-by-value, leftmost-outermost, lazy reduction, call-by-push-value...

¹especially if they involve ineffectual reductions such as reducing a non terminating term, or reducing a sub-term that does not need evaluated

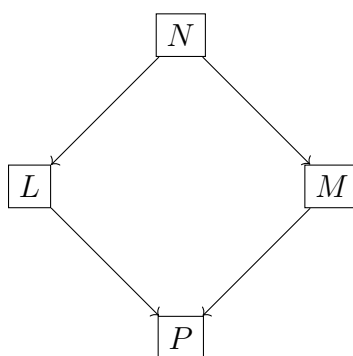


Figure 1-1: The Church-Rosser Property

1.2 Type Theory

Thinking of programs as functions is very useful, and largely possible because of the incredibly flexible definition of function used in the lambda calculus: something which, given an input produces a determined output. However, there are difficulties associated with studying such a general class of objects. For instance, not all terms of the lambda calculus can be interpreted as functions, some represent non-terminating computations and so have no defined output. The proposed solution is *types*.

Types are important both conceptually and in practice. The concept first appears in Russell's *Principia Mathematica* [73] as an approach to the inconsistency of naïve set theory demonstrated in his eponymous paradox. From our perspective type systems are a key means of ensuring program safety; providing guarantees on the behaviour of typed terms. This may encompass: accepted arguments, outputs, run time, termination and more.

Over many years mathematicians refined their notion of function to the set-theoretic definition: a function from set A to set B is a subset of $A \times B$ (in other words a set of pairs, with first element in A and second in B) such that for every element a of A there is exactly one pair (a, b) in the function. This definition has the advantage that without knowing the computational process of a function we can still see the valid inputs it is expecting, and what is a possible output, simply by checking which sets the function is between. Without resorting to the entire structure of set theory, a similar sort of information can be attached to a

fragment of the lambda calculus, through type theory.

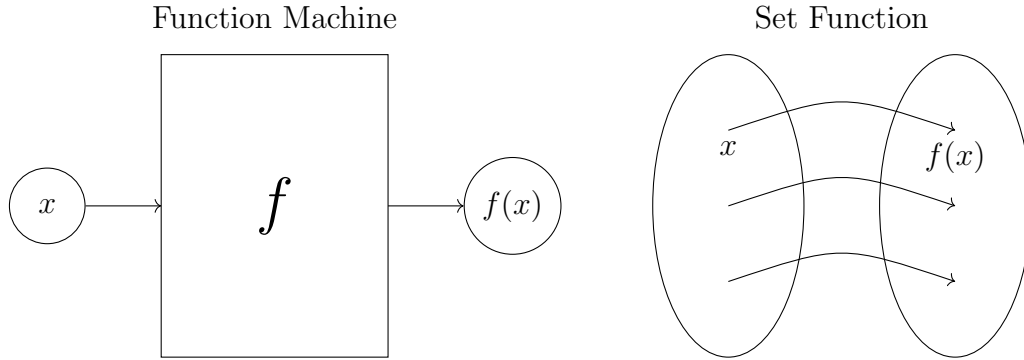


Figure 1-2: Machine versus Set Theoretic model of Functions

Simple type theory corresponds most closely to set-theoretic notions. Instead of having a set of possible inputs there is a type-theoretic descriptor of the sort of elements a function can take. In the most distilled version of this method there are no type variables: everything is built up from a single base type as shown in the following grammar:

$$A, B ::= o \mid A \rightarrow B$$

A simple type system is shown in Figure 1-3. On reflection this type system is quite restrictive. Consider the identity function $\lambda x.x$: when typing it one must determine what arguments it is supplied with before typing it as $o \rightarrow o$ or $(o \rightarrow o) \rightarrow (o \rightarrow o)$ or $((o \rightarrow o) \rightarrow (o \rightarrow o)) \rightarrow (o \rightarrow o) \rightarrow (o \rightarrow o) \dots$ any of these, but never more than one at once. In reality an empty program will act as an identity no matter what initial state it is given. It seems necessary to encode more general types.

Simple types, by their restrictive nature, give very strong results about a small fragment of a calculus. Defining type systems is a compromise, one wishes to find

$$\frac{}{\Gamma, x : A \vdash x : A} \quad \frac{\Gamma, x : A \vdash M : B}{\Gamma \vdash \lambda x. M : A \rightarrow B} \quad \frac{\Gamma \vdash M : A \rightarrow B \quad \Gamma \vdash N : A}{\Gamma \vdash M N : B}$$

Figure 1-3: Simple typing rules for the λ -calculus.

a type system which can be applied to a non-trivial fragment of the calculus and gives useful information on this fragment.

There are many ways in which simple types have been extended to encompass more details of the lambda-calculus. Seven such extensions are studied in Barendregt's λ -cube. The cube starts off from the simply-typed calculus and extends it along three independent axes: dependent typing allows types to depend on the terms as well as the types of their subterms; polymorphic extensions allow you to abstract over types, placing type variables in the place of specific types; finally, one may have types which are used to define other types. One particular type system for the lambda calculus - intersection types - provides a typed universal model of computation. Extending and tweaking the type system is an expected part of studying the lambda calculus.

From a modern view point the key property of most type systems² is the *guarantee* of strong normalisation: the typeable fragment excludes non-terminating terms. Simple types, idempotent and non-idempotent (quantitative) intersection types, Hoare logic: all provide different kinds of safety guarantees. In this exposition we aim to stay close to simple types to leave open the possibility of type inference. Intersection types can likely be derived in the usual way. Types also play an important role in the Curry-Howard correspondence which links proofs to programs via the type system. A term of the lambda calculus can be thought of as manipulating types, executing the proof of the logical statement corresponding to its type.

Several techniques are standard when proving normalisation/termination of reduction [72]. Despite the casualness with which they are invoked each represents a significant achievement in the field. The problem of normalisation is essentially one of consistency. The lambda calculus is *inconsistent* if all terms are inter-reducible or equal under an equational theory; showing that there are distinct unique normal forms precludes this. Godel's incompleteness theorem showed that a system cannot be sufficiently complex to prove its own consistency, so these proofs by their nature require the development of additional machinery. For instance, abstract reducibility proofs in the style of Tait [69] (weak normali-

²common to all type systems without a recursor, but not those with, for instance haskell

sation) and Girard (extension to polymorphic types, strong normalisation) create a family of sets of strongly-normalising terms, define a type interpretation to it, then show that all typed terms are within the interpretation of their type, and thus strongly-normalising. Gandy-style proofs of termination [28] instead introduce a well-founded, orderable representation of terms (monotone functionals) and show that reduction results in terms with smaller representations. While now standard, the ideas behind these methods were innovative; implementations of them are not always transparent and may in themselves pose difficulties.

1.3 The Probability Problem

As we have discussed, reduction of the untyped λ -calculus enjoys strategy independent confluence: due to the Church-Rosser property. In this way, a term can fully describe a computation without external specifications. However, when probabilistic operators are introduced this confluence disappears. This is because when the outcome of a term is not determined it may change on each iteration, hence evaluation does not commute with duplication. The problem arises when you wish to consider reduction within a context – since subterms may later be duplicated – or consider normal forms, which are unlikely to be uniquely defined.

In many cases the literature does not approach this as a problem, but simply a fact of life that comes with effects. Markov categories go so far as to make the ability to commute with duplication the *definition* of deterministic [27]. It is not possible for papers to completely ignore the phenomenon as it clearly interferes with the properties of reduction, rather they sidestep the issue in a number of ways. The most straightforward fix is to pick a reduction strategy (with or without justification) and stick to it. A variation on this is to consider two opposed reduction strategies separately, treating each as fixed. Authors who do not want to impose a strategy often place restrictions on the grammar or side conditions on reduction, which in the end have the same effect as fixing a strategy. The constructionists equivalent of considering two strategies then, is to define a composite calculus with a second form of either abstraction or application which conforms to one reduction strategy, restricting the unannotated version to the

other.

The most common evaluation strategies considered in the literature are call-by-value, which evaluates arguments before evaluation, and call-by-name [58] which only evaluates arguments when they are needed within the term. Each has advantages: where all the arguments of a function are required, using call-by-value minimises the number of evaluations needed, whereas call-by-name prevents the unnecessary evaluation of arguments and reduces the need for variable storage.

We use the Probabilistic event λ -calculus (PE λ) as our basic probabilistic calculus. This calculus takes an unusual approach; namely to maintain confluence and ensure that both strategies remain available. To overcome the impasse described above, PE λ removes the ambiguity from duplication by introducing an *event binder*. The placement of the binder controls when an event may be evaluated and allows PE λ to encode multiple evaluation strategies. Both common strategies are easy to encode in PE λ by enforcing a convention on the placement and scope of event generators. Maximising the scope of all generators essentially fixes event outcomes first so causes the term to evaluate like call-by-value. Minimising the scope of all generators protects subterms from evaluation thus behaves like call-by-name.

1.4 Probabilistic lambda-Calculi

Although earlier models were used to study probabilistic computation, the first resembling a probabilistic lambda calculus was Saheb-Djahromi's Probabilistic LCF [63] in 1978. This probabilistic language, based on PCF, was built with the intention of formalising the often-discussed, but abstract, notion of probabilistic semantics. The main differences between the lambda calculus and PCF are the explicit inclusion of natural numbers and conditionals, which are encodable in, but not native to, the lambda calculus as well as a syntactic fixed point operator (as opposed to the one expressible in the lambda calculus, which is untypeable). Like many of early calculi, it deals with non-confluence of reduction by having different versions of abstraction for call-by-value and call-by-name. The proba-

bilistic operator in this case is annotated pairing, and the operational semantics feature a probabilistic reduction.

While the study of higher order probabilistic computation continued [51, 38, 39], little further was done in the domain of probabilistic lambda calculi for a few years: as it became fashionable to use the lambda calculus to study general non-determinism, with no allusion to the probability of reduction branches. In these scenarios [35, 19] the possibility of encountering infinitely many *choices* is a form of divergence. In the early 2000s interest rearose in stochastic versions of the lambda-calculus [57, 62], as a means of modelling and reasoning about probabilistic systems. These consider distributions of terms, exploring their semantics via probability monads and Bayesian networks, and relegate confluence to a side condition of reduction.

By 2011 the theory of probabilistic PCF was revisited and given a semantics in the form of Probabilistic coherence spaces [17], proven to be a fully-abstract model in 2014 [25] alongside the creation of a weighted intersection-type system for probabilistic PCF. In these papers choice of strategy is replaced by restrictions on the form of reductands: beta-reduction is permitted solely with determined arguments. Also in 2011 Dal Lago et al [16] focused on reduction strategies, showing that each strategy can be embedded into the other via a study of big and small step semantics: inductively and co-inductively defined.

The area has now gained some mainstream recognition, with a 2020 publication about probabilistic programming including a chapter [43] on probabilistic lambda calculi. Due to flurry of activity in the study of variants of the probabilistic lambda-calculus [65, 31, 13] in recent years I include only those which give significant consideration to the problem of strategy.

The 2018 paper “On Probabilistic Term Rewriting” [4] introduces multi-distributions as a novel framework for dealing with the branching nature of reduction in a probabilistic setting. Leventis’ 2019 paper [47], meanwhile, attempts to return confluence to a probabilistic lambda calculus; to this end it defines a reduction relation that naturally restricts to call-by-name evaluation, by disallowing the reduction of terms in argument position. In the same year and with the aim

of creating a foundational calculus for probabilistic computation, Faggian and Ronchi [26] take two separate extensions to the lambda calculus, which exhibit CBN and CBV and prove them equally valid, before considering a root calculus.

In the following chapters I use the probabilistic event lambda calculus [15] which deals with the issue of confluence by decomposing probabilistic choice into two operators: generator and dependent choice. This makes explicit where a new sample from the probability distribution is required, also; the act of sampling itself can be duplicated, or merely the choice. The result of this design decision is that although the outcome of a term is not determined, the distribution of possibilities is.

1.5 Sequential lambda-Calculus

Stack machines have played an important role in the development of programming languages. In particular, the lambda-calculus was first modelled by an abstract machine; the SECD machine, in 1964 [45]. This acronym stands for Stack, Environment, Control and Dump, the four registers of the machine. The SECD machine forces a *call-by-value* interpretation since only values may be pushed onto the stack. Contrastingly, the Krivine machine [42], an abstract machine for the computation of recursive functions, defines normal form reduction of a lambda term by implementing *call-by-name* reduction. However, there are disadvantages to this as a model of computation namely that the Krivine machine has no output and no explicit notion of successful termination.

The ability to express both call-by-name and call-by-value evaluation gives freedom to the control flow, because it allows choice of path while retaining confluence, and it also returns local control. It is thus desirable to embed both into one language.

Computational effects are intrinsically *sequential* commands. When they are combined with a fundamentally *denotational* system – one, such as the λ calculus, which is governed by an equational theory – order of evaluation is important. This means there is a loss of confluence and a reduction strategy is necessary to control

evaluation. The need for an evaluation strategy weakens the idea that terms have their own intrinsic meaning, and hampers semantic reasoning.

Syntactic control of evaluation is necessary when dealing with effectful programs. In practice, most functional programming languages use a call-by-value strategy and where call-by-name behaviour is required it is forced locally via thunks – wrappers which prevent argument evaluation. Despite its apparent impracticality, call-by-name behaviour is important: when reasoning about programs generated from proofs by the Curry-Howard correspondence the call-by-value interpretation does not convey the meaning originating from the proof.

The bang calculus [23], takes the notion of protection from evaluation by dummy abstraction used to embed the call-by-name interpretation within a call-by-value calculus, and makes it a secondary explicit operation, removing the need for dummy variables.

Moggi’s interpretation of effects-as-monads [55] successfully deals with multiple reduction strategies by modelling sequentiality as composition in the Kleisli category of a monad. However, this solution does not extend to multiple effects, since monads themselves do not compose. Many approaches have been taken to using monads to deal with concurrent effects including monad transformers [49], sums of monads [50], weakening monads to applicative functors which do compose [53]. Each of these approaches has limitations with the latter two needing lifting functions in order to access different effects, which suggests that the problem is not merely with compositionality, but rather with explicit composition.

The κ -calculus extended with thunks to give a higher-order-model of computation is the basis of a paper by Power, Robinson, and Thielecke [60] which connected Hasegawa’s κ -calculus [33] with a semantics in premonoidal categories. This semantics imposes sequentiality on composition in a monoidal category by weakening its naturality laws. A similar semantics was later applied to call-by-push-value, which generalises Kleisli composition of monads in a similar way.

There are, in fact, several similarities between call-by-push-value and the κ -calculus:

1. Sequentiality achieved through an operational semantics in the form of a stack machine.
2. Decomposition of terms into computations and values.
3. Embedding of call-by-name and call-by-value.

Levy’s call-by-push-value [48] interprets abstraction and application as push and pop instructions as used in stack machines, and extends this notion, allowing the stack to also model sequentiality. Distinguishing between evaluated and unevaluated terms with the slogan “a value is, a computation does” the types for this calculus are split into value and computation types. This interpretation makes the control flow explicit and unifies effects via a single monad formed by an adjunction which separates values and computations.

Call-by-push-value has also been extended with effect handlers to deal with *algebraic effects*. Algebraic effects are treated as algebraic theories in a way which allows equational reasoning, but not term reduction. These effect handlers generalise exception handlers in programming languages, enforcing sequentiality where it would otherwise be relaxed.

The Functional Machine Calculus (FMC) combines sequential computation, with *locations*: extra stacks which enable the expression of effects. For instance, probabilistic choice can be encoded through the inclusion of an extra, pop-only, location for a Boolean stream. The restriction of the FMC to the interpretation of the λ -calculus along with this location is the probabilistic event calculus, which we will work with extensively.

The FMC encodes the lambda calculus through pop-as-abstraction and push-as-application: an abstraction $\lambda x. M$ pops the top off the stack, say N , and continues as $\{N/x\}M^3$, while an application $M N$ pushes its argument N and continues as M . In this correspondence a variable indicates execution. The sequential aspect of the machine is present in the imperative instructions, including skip: a term which indicates termination of the term evaluation. The sequential lambda

³The use of substitution here represents a simplification of the Krivine machine, where the binding of N to x would be added to the evaluation environment.

calculus occurs as the fragment of the FMC formed of the lambda-calculus, extended solely with *sequential composition* $M ; N$ and its unit, the imperative *skip* $\star$. With the term viewed as a language of instruction sequences for this machine these constructs have the expected semantics: *concatenation* of machine instructions and the *empty* instruction. Since the machine is primary in this calculus, notions of input and output are at the forefront. This makes the notion of successful termination more explicit than in many other formulations. As in imperative languages, *skip* indicates successful termination of the machine.

The aim of the next part of this thesis is to give a notion of probabilistic termination. To get there we use a sequential extension of $\text{PE}\Lambda$ which shares the FMC's notion of successful termination as modelled by a complete run of a stack machine. For a machine, the type system is directly linked to the meaning. Rather than considering individual evaluation steps, we consider a *big step semantics* [59] which looks at the long-term evolution of a reduction system.

The features of explicit termination and modelling of both **cbn** and **cbv** evaluation make sequential extension an ideal tool for our purposes. Considering the big step semantics will allow us to reason about universal properties of typed terms and ultimately prove normalisation.

1.6 Continuations

Continuations can be seen as a generalisation of the condition for termination used in the sequential lambda-calculus. Whereas *skip* indicates that a term has been completed, a continuation indicates further computation.

Continuations are a means of controlling sequential behaviour. Directing execution to a particular part of the program, they form the basis of imperative programming and have appeared in programming languages in the form of *goto* statements since ALGOL60. Landin's J-operator, a precursor to continuations, was the first extension of the lambda calculus to model the semantics of labels and jumps as used in ALGOL60. The J-operator applied to a subterm of a computation indicates that if that subterm is evaluated then its outcome is taken

to be the result of the entire term. Exception handling⁴ is an important aspect of code hygiene and exceptions in λ -calculus are frequently studied through continuations.

The notion of continuations and their impact on control flow is very useful for compilers. There has been recent work on using sequent calculus and sequentially-used continuations for compiler optimisation [21, 52] and it is from these works that we adopt the terminology of jumps and joins. Continuations have a Curry-Howard relation with classical logic, expressed in Parigot's $\lambda\mu$ -calculus [56]. The join calculus shares several features with the lambda mu calculus, in particular, named terms and duals to variables, the distinction being in the μ -abstraction; this relationship is discussed in more detail in Section 6.5.

1.7 Outline

The remainder of this thesis will draw on the concepts outlined above to explore the interaction of computation with probabilistic choice. The next part focuses on PEA, extending its type system in Chapter 3 to express probabilistic termination in a natural way. This system is further proven to follow its intuition by placing a *lower bound* on the probability that a term's reduction sequence terminates.

Chapter 4 introduces a sequential extension of PEA along with an operational semantics in the form of a modified Krivine machine [34], where sequential composition takes the place of call-by-value application. The type system for PEA is lifted to a semantic type system defined on the sequential calculus, in which types take the form of a stack transformation annotated with a probability p . These are used in a proof of termination of reduction (in other words normalisation) inspired by the restrictions imposed in previous publications.

In brief, a run of the machine corresponds to head reduction: a term of the calculus is guaranteed to head normalise when this machine runs to completion. Considering the big step semantics of the machine, a measure on the machine

⁴A key inspiration behind effect handling.

runs shows that terminating run branches account for at least a proportion p of the resulting distribution. This result easily projects back onto the original type system.

Part II introduces a sequent calculus annotated with the join calculus, which is a calculus with continuations, designed to model probabilistic termination. This system is capable of handling probabilities and almost-surely-terminating terms, as defined by co-recursion. Co-recursion is the main topic of Chapter 8, however, we will first consider the non-recursive fragment of the Join Calculus, for which we provide a type system. With the type system in place to provide admissibility checks we define a notion of beta reduction and demonstrate the permutations which cause reduction of terms. Furthermore, we show that permutation of independent rules behaves as expected. Normalisation of the system is shown via an interpretation in natural deduction with sums and products. The extension of this normalisation result to the full calculus is left as a conjecture.

Part I

Probabilistic Termination

Chapter 2

The Probabilistic Event lambda-Calculus

In this chapter I recall the Probabilistic Event λ -Calculus from [15] with which I will be working for the next few chapters. The main point of this calculus is confluence without restriction of reduction strategy.

Most probabilistic calculi extend the λ -calculus using a single operator $\oplus$ to express probabilistic choice. In the probabilistic event λ -calculus [15] probabilistic choice is instead decomposed ($M \oplus N = \boxed{a}. M \ a \ N$) into two operators: a binary choice, a , and an event generator, $\boxed{a}$.

When in argument position $\boxed{a}. M$ acts as a wrapper, similar to *thunking* in call-by-push value [48] and the bang-calculus [23, 24], protecting the operator from evaluation.

Definition 1 Terms of the *probabilistic event λ -calculus* (PE Λ) are given by the following grammar

$$M, N ::= x \mid \lambda x. M \mid M N \mid \boxed{a}. M \mid M \ a \ N$$

Where $\{\dots, x, y, z\}$ are *variables*, $\{a, b, c, \dots\}$ are *events* and both sets are assumed to be countable.

The first three constructors of this grammar: *variable*, *abstraction* and *application* are identical to their λ -calculus counterparts. The final two allow the expression of probabilities: a *generator* $\boxed{a}.M$ flips a coin and binds the result ($\top$ or $\perp$) to a ; the *choice* or *conditional* $M a N$ evaluates to M if a is *true*, and to N otherwise. Note that the events are technically variables, however, term variables are instantiated by a term of the calculus whereas events are Boolean variables: instantiated by $\top$ or $\perp$.

The definition of free and bound variables follows the usual convention for the λ -calculus; a variable x is *bound* if it is within the scope of an abstraction $\lambda x.$ and *free* otherwise. The set of free variables of a term M is denoted $\text{fv}(M)$. Events are set up analogously: $\boxed{a}$ acts as an abstraction for the conditional a . All instances of a within the scope of $\boxed{a}$ are bound to, and dependent on, the outcome of the event. Events which are not bound are free, $\text{fe}(M)$ is the set of free events appearing in the term M . As with term variables, bound events can be renamed, and event variables are considered up to alpha conversion. Renaming provides a natural means to distinguish duplicated events.

The unannotated generators $\boxed{a}$ are assumed to be *fair* throughout, with $\top$ and $\perp$ occurring with equal probability. Where required a *biased* generator is denoted $\boxed{a}_p$ where p is a *probability*: a real number between 0 and 1 inclusive. This evaluates to $\top$ with probability p , and $\perp$ with $1 - p$.

The following are standard notions of *context*, *head context*, and *applicative context*. It is worth noting that this is not the definition of head context originally used for $\text{PE}\lambda$ (which is more properly a spine context) but it does give an equivalent definition of head reduction: the contexts not included by this definition contain an outer beta redex, thus the reducible terms are unchanged, as are the normal forms.

Definition 2 *Contexts* C , *head contexts* H , and *applicative contexts* A are defined as follows.

$$\begin{aligned} C &::= \{ \} \mid \lambda x. C \mid C M \mid M C \mid \boxed{a}. C \mid C a M \mid M a C \\ H &::= \lambda x. H \mid A \quad A ::= \{ \} \mid A M \end{aligned}$$

A context C or H with the hole replaced by M is written $C\{M\}$ respectively $H\{M\}$. As in the λ -calculus, this is a *capturing* operation; abstractions and event generators in C may capture free variables and events in M .

Under this definition a head context can always be written in the form

$$\lambda x_1 \dots \lambda x_n. \{ \} M_1 \dots M_m$$

where m and n are potentially zero.

The basic reduction rule of the calculus, beta reduction, is identical to that of lambda calculus. This seems unremarkable until one considers that many probabilistic extensions of the lambda-calculus need to modify exactly this rule to remain consistent [47].

Definition 3 *Beta*-reduction ($\rightarrow_\beta$) and *head* β -reduction ($\rightarrow_{\beta h}$) are given by closing the beta-rule, below, under all contexts C respectively under head contexts H .

$$(\lambda x. M)N \rightarrow_\beta \{N/x\}M$$

Where $\{N/x\}M$ denotes the *capture-avoiding* substitution of N for x in M . Capture-avoidance is defined in such a way that neither free variables nor free events are captured.

Projection functions $\pi_\top^a$ and $\pi_\perp^a$, apply the effect of instantiating a (with *true* and *false* respectively) to a term M .

Definition 4 The *projection* function π_i^a for an event a and $i \in \{\perp, \top\}$ is given as follows, where $a \neq b$.

$$\begin{aligned} \pi_i^a(x) &= x & \pi_\top^a(M a N) &= \pi_\top^a(M) \\ \pi_i^a(\lambda x. M) &= \lambda x. \pi_i^a(M) & \pi_\perp^a(M a N) &= \pi_\perp^a(N) \\ \pi_i^a(M N) &= \pi_i^a(M) \pi_i^a(N) & \pi_i^a(\boxed{a}. M) &= \boxed{a}. M \\ \pi_i^a(M b N) &= \pi_i^a(M) b \pi_i^a(N) & \pi_i^a(\boxed{b}. M) &= \boxed{b}. \pi_i^a(M) \end{aligned}$$

In [15] Dal Lago et al. give two reduction relations for the probabilistic operators, both extending beta reduction. The system for which confluence is particularly important is *permutative reduction* $\rightarrow_p$ [15], which works solely on the syntax of a term. Through rewriting permutative-reduction internalises the reduction of $\boxed{a}.M$ to the sum of its two projections [15, Proposition 29].

$$\boxed{a}.M \rightarrow_p \boxed{a}.(\pi_+^a M) \ a \ (\pi_-^a M) \quad (\text{if } a \in \text{fe}(M)) .$$

Normal forms of this reduction have generators with minimal scope and are thus in the image of the call-by-name embedding shown below.

Here we focus on a more practical approach to reduction, namely projective reduction. Whereas permutative reduction orients the equations of an algebraic theory over terms, projective reduction looks directly at the meaning of event evaluation. This choice is made because of our interest in the termination of head reduction.

The result of projective reduction is not a straightforward term but rather a *multi-(sub-)distribution* [4] of terms: this is a finite multiset of weighted terms $\mathcal{M}$ written as $[p_1 \cdot M_1, \dots, p_n \cdot M_n]$ in which the *weight* of the distribution, defined as the sum of term weights $|\mathcal{T}| = \sum_{i \leq n} p_i$ is (at most) one. To simplify our notation, we use a number of shorthands.

- We convert implicitly between terms M and the singleton distribution $[1 \cdot M]$.
- Multiset union is written $\mathcal{S} + \mathcal{T}$.
- The empty multiset is $\emptyset$, this is distinct from multisets of weight 0.
- A distribution $\mathcal{M}$ is scaled to $p\mathcal{M}$ by multiplying each weight in $\mathcal{M}$ by p .
- The underlying probability (sub-)distribution of $\mathcal{M}$ is denoted $\lfloor \mathcal{M} \rfloor$ and is obtained by collecting like terms $p \cdot M$ and $q \cdot M$ as $(p+q) \cdot M$.

Note: where it is not confusing, both probability (sub-)distributions and multi-(sub-)distributions are referred to simply as *distributions*.

Definition 5 *Projective* reduction $(\rightarrow_\pi)$ is the following reduction relation

$$H\{\boxed{a}.M\} \rightarrow_\pi \frac{1}{2} \cdot H\{\pi_0^a(M)\} + \frac{1}{2} \cdot H\{\pi_1^a(M)\}$$

Head reduction $(\rightarrow_h) = (\rightarrow_{\beta_h}) \cup (\rightarrow_\pi)$ is defined as the union of head β -reduction and projective reduction.

Reduction is lifted easily to probability distributions of terms: if $M \rightarrow N$ then $[p \cdot M] + \mathcal{M} \rightarrow (pN) + \mathcal{M}$. When an individual term of the distribution is reducible the reduction step may be applied to the whole distribution and this results in the scaled outcome of reduction being placed in multiset union with the unchanged remainder of the distribution.

Projective reduction must have the above restriction to head contexts because distribution does not commute with application, in particular duplication; this results in the following inequality:

$$M(N + P) \neq MN + MP$$

Full reduction of the probabilistic event λ -calculus is confluent [15, Theorem 23]. The outcome of a computation is independent of the choice of reduction strategy so the divergent behaviour under CBN- and CBV-evaluation strategies observed with traditional probabilistic sums does not occur. However, the behaviour of each strategy may be encoded in the calculus. What follows is a brief discussion of the reasoning behind these embeddings and grammars describing their images.

A traditional probabilistic sum of terms, $M \oplus N$ can be formed by recombining the event and choice operators as $\boxed{a}.M a N$, where a is assumed to not occur freely in M or N . In practice this encoding corresponds to the call-by-name interpretation of probabilistic sums: the generator is placed such that only the choice it pertains to is within its scope, hence any duplication of a probabilistic argument must also duplicate the generator.

Fragment of PEA corresponding to a call-by-name probabilistic λ -calculus:

$$M, N ::= x \mid \lambda x. M \mid M N \mid \boxed{a}. M a N$$

A naïve implementation of call-by-value evaluation has all arguments evaluated before the first point of use, including any probabilistic sampling. In this formulation dummy wrappers do not protect probabilities from evaluation. Thus one may *maximise the scope* of all generators which prevents generators from ever being duplicated by evaluation, or equivalently, makes event valuation external to the term.

Fragment of PEA corresponding to a call-by-value probabilistic λ -calculus:

$$P, Q ::= x \mid \lambda x. P \mid P Q \mid P a Q \quad M = \boxed{}. P$$

Where $\boxed{}. P$ is the event closure $\boxed{a_1}. \boxed{a_2}. \dots \boxed{a_n}. P$ over free events $\{a_1, a_2, \dots, a_n\} = \text{fe}(P)$.

The CBV-embedding shown here gives only a trivial notion of probabilistic CBV behaviour: evaluating all probabilities immediately. It does not fully correspond to Plotkin's CBV [58] or the behaviour of ISWIM [46] (the original CBV version of Landin). To fully express that we will need the power of a sequential lambda-calculus which we will see in later chapters.

2.1 Simple Types

In this section we recall the simple type system defined in [15] which describes a strongly-terminating fragment of the calculus.

Definition 6 *Simple types* are given by the following grammar.

$$A, B ::= o \mid A \rightarrow B$$

A *typing context* Γ is a finite function from term variables to types, written

$x_1 : A_1, \dots, x_n : A_n$. A *typing judgment* $\Gamma \vdash M : A$ assigns the type A to the term M in the context Γ . The typing rules are given in Figure 2-1.

$$\begin{array}{c}
\overline{\Gamma, x : A \vdash x : A} \\
\\
\frac{\Gamma, x : A \vdash M : B}{\Gamma \vdash \lambda x. M : A \rightarrow B} \\
\\
\frac{\Gamma \vdash M : A \rightarrow B \quad \Gamma \vdash N : A}{\Gamma \vdash M N : B} \\
\\
\frac{\Gamma \vdash M : A \quad \Gamma \vdash N : A}{\Gamma \vdash M a N : A} \\
\\
\frac{\Gamma \vdash M : A}{\Gamma \vdash \boxed{a}. M : A}
\end{array}$$

Figure 2-1: Simple typing rules for the $\text{PE}\Lambda$.

These simple types ignore the probabilistic constructs of the calculus, *generator* and *choice*, requiring only that branches of a choice have equal types.

This treatment of probabilistic terms means that for a simply-typed $\text{PE}\Lambda$ -term every probabilistic branch of the computation corresponds to a simply-typed λ -term. These individual branches are thus strongly normalising. Since the type tree is finite and finitely branching, the typed terms are finite sums of strongly normalising terms. This gives the expected result: typed terms are strongly normalising. The simply-typed probabilistic event lambda calculus is proven to be strongly normalising with respect to full reduction: beta reduction extended with permutative reduction [15, Theorem 25].

The normal forms N_0 for this reduction system are characterised by the following grammars:

$$\begin{array}{lcl}
N_0 & ::= & N_1 \mid \boxed{a}. N_0 a N_0' \\
N_1 & ::= & N_2 \mid \lambda x. N_1 \\
N_2 & ::= & x \mid N_2 N_0
\end{array}$$

Chapter 3

A Type System for Probabilistic Termination

In the last chapter we recalled the simple type system introduced with PEA [15], which is analogous to that of the λ -calculus. This system does not address the probabilistic aspects of the calculus; the rules simply ignore the probabilistic constructs of choice and generator. It does, however, ensure strong normalisation of full reduction. Requiring the type system to be fully deterministic forces the typeable terms to be composed of probabilistic choices between typed deterministic terms, hence the strong results.

It is natural to consider what happens when we relax the guarantees, for instance, asking for probability of termination instead of termination or replacing run time with expected run time. This provides probabilistic information about a probabilistic system which is truly distinct from what can be defined on deterministic systems. In this part of the thesis we focus on the probability of termination. Because the idea of probabilistic termination for an arbitrary reduction relation is not well-defined one can only speak of probabilistic termination of a reduction strategy. The natural choice is thus to consider (weak) head normalisation which corresponds to the call-by-name strategy for the λ -calculus. For non-probabilistic λ -terms weak head-normalisation is a key property; for probabilistic λ -terms this property naturally generalises to a *probability* of weak head-normalisation.

This is because a term may make a probabilistic choice between a weakly head-normalising and a non-weakly head-normalising term.

This leads naturally to the notion of *probabilistic termination*: the probability that a given reduction mechanism reaches the normal form of a term [14, 24, 41, 6, 44]. This is the weakest form of termination defined on probabilistic terms, simply telling us a proportion of runs expected to terminate.

The next level up is *almost-sure termination* which I will consider later. For an almost-surely-terminating term the probability of termination is one. Although there are possible non-terminating runs for these terms their realisation requires specific infinite sequences of probabilistic outcomes which overall have probability zero. This is distinct from *certain termination* where there are no non-terminating branches. In fact, there is a notion of termination strictly between almost-sure and certain termination: *positive almost-sure termination* requires that the expected number of evaluation steps be finite.

To capture probabilistic termination I will now present a probabilistic type system on $\text{PE}\Lambda$. Constructed by modifying simple types to express the probability of an outcome, this system provides a lower bound for the probability that a reduction sequence terminates. In this case, probabilistic termination captures that a given fraction of all branches of a computation terminate where termination is given by *head normalisation*.

3.1 Probabilistic Types

The idea behind this probabilistic type system is simple and elegant; take the system of simple types in figure 2-1 and replace the base type $\mathbf{o}$ with a probability denoted by a real¹ $p, q, r \in [0, 1]$. Why should this work? The base type $\mathbf{o}$ can be thought of as indicating termination. After all, this would be the meaning of an inhabited base type. Consider the type $\mathbb{Z}$ inhabited by integers; a type $A_1 \rightarrow A_2 \dots \rightarrow \mathbb{Z}$ represents terms which, when given arguments of type $A_1, A_2 \dots$,

¹For the purposes of this work it would be sufficient to work with rational or 2-adic probabilities, however such a restriction is not necessary.

terminates successfully and returns an integer. The base type o , despite being uninhabited, behaves analogously in that it denotes a certainty of termination on receipt of inputs of the correct type. By replacing o with p we immediately obtain a means of expressing the probability of termination.

Our type system, then, consists solely of arrows and probabilities. How should these components interact? A clue may be gleaned from the permutation reduction system of the event lambda calculus, which contains the rule $(\boxed{a}.N)M \rightarrow \boxed{a}.NM$. In this rule the generator permutes past application. This can be interpreted as indicating that during application the probability of termination is independent of consumption of an input. Returning to the projective reduction system we are working with: consider the term $(\boxed{a}.I a \Omega)I$. This term undergoes projective reduction to create $\frac{1}{2}II + \frac{1}{2}\Omega I$ only one term of which is terminating hence the term terminates with probability $1/2$. The term $\boxed{a}.(I a \Omega)I$ reduces identically so should have the same type.

In [2] they guarantee that passing an argument to an arrow typed term with probability p , cannot affect the probability of termination by using *outer probabilities*. An outer probability annotates the overall type with a probability of termination. Rather than an outermost probability we chose to work with probability on return type. This formulation still ensures that probability can commute with the arrow, however, placing the probability internally has several advantages. It increases the simplicity and readability of the type system, reflects the intuition of the machine interpretation, and makes the typing rules conservative over simple types for the lambda-calculus. These features are apparent in the comparison with counting quantifiers at the end of this chapter.

Definition 7 *Probabilistic types* are given by the following grammar.

$$A, B ::= p \mid A \rightarrow B \qquad p \in \mathbb{P} := [0, 1]$$

For brevity of notation we adopt the convention that $\overline{A} \rightarrow p$ is shorthand for a sequence of antecedents $A_1 \rightarrow \dots \rightarrow A_n \rightarrow p$.

A key decision faced when designing the type system is where the typing rules

branch, is it at generator or at choice? The computation itself branches at the generator where the *die is cast* or the coin is tossed. Our type system matches this by branching at the generator construct. This can be done with projections, however, we want derivations to be inductive on terms as far as possible, therefore we opt for an event valuation instead.

Rather than making projections part of the language we include pre-evaluated events in the context along with the standard type judgements for variables. An event valuation has been chosen here to avoid projection, to highlight that the generator behaves similarly to abstraction and to place type branching at the generator rule. I will now define the type system and in the next section comment extensively on the reasoning behind it with a specific subsection which expands on event valuations in detail.

Definition 8 A *probabilistic typing judgement* $E \mid \Gamma \vdash M : A$ assigns a type A to the term M in the context of E and Γ , defined as follows:

- An *event valuation* E is a finite function from events to $\mathbb{B} = \{\perp, \top\}$, written as a sequence $a_1 \mapsto i_1, \dots, a_n \mapsto i_n$ with each $i \in \mathbb{B}$.
- A *typing context* Γ is a finite function from variables to types, written $x_1 : A_1, \dots, x_n : A_n$.

The manipulation of these probabilities is dealt with, along with other type interactions, by the typing rules.

Definition 9 The probabilistic type system for $\text{PE}\Lambda$ is given by the typing rules in Figure 3-1.

Unless otherwise specified, the generator $\boxed{a}$ is a fair coin toss. For this reason the typing rule for $\boxed{a}.M$ gives a probability of $\frac{1}{2}(p + q)$. A biased toss $\boxed{a}_r$ with probability of success r can be used to model any other *binary* distribution and would give a probability $rp + (1 - r)q$ in the same rule.

$$\begin{array}{c}
\overline{E \mid \Gamma, \textcolor{red}{x} : A \vdash \textcolor{blue}{x} : A} \\
\\
\frac{E \mid \Gamma, \textcolor{red}{x} : A \vdash \textcolor{red}{M} : B}{E \mid \Gamma \vdash \lambda \textcolor{red}{x}. \textcolor{red}{M} : A \rightarrow B} \\
\\
\frac{E \mid \Gamma \vdash \textcolor{red}{M} : A \rightarrow B \quad E \mid \Gamma \vdash \textcolor{red}{N} : A}{E \mid \Gamma \vdash \textcolor{red}{M} \textcolor{red}{N} : B} \\
\\
\frac{E, a \mapsto i \mid \Gamma \vdash \textcolor{red}{M}_i : A}{E, a \mapsto i \mid \Gamma \vdash \textcolor{red}{M}_\top a \textcolor{red}{M}_\perp : A} \\
\\
\overline{E \mid \Gamma \vdash \textcolor{red}{M} : \overline{A} \rightarrow 0} \\
\\
\frac{E, a \mapsto \perp \mid \Gamma \vdash \textcolor{red}{M} : \overline{A} \rightarrow \textcolor{blue}{p} \quad E, a \mapsto \top \mid \Gamma \vdash \textcolor{red}{M} : \overline{A} \rightarrow \textcolor{blue}{q}}{E \mid \Gamma \vdash \boxed{\textcolor{red}{a}}. \textcolor{red}{M} : \overline{A} \rightarrow \frac{1}{2}(p + q)} \\
\\
\frac{E \mid \Gamma \vdash \textcolor{red}{M} : \overline{A} \rightarrow \textcolor{blue}{q}}{E \mid \Gamma \vdash \textcolor{red}{M} : \overline{A} \rightarrow \textcolor{blue}{p}}_{(p < q)}
\end{array}$$

Figure 3-1: Typing rules for $\text{PE}\Lambda^{\mathbb{P} \rightarrow}$

3.2 The Rules in Detail

The similarities between the simple and probabilistic type systems can be seen in figure 3-2 which compares the rules directly.

$$\begin{array}{c}
\frac{}{\Gamma, x: A \vdash x: A} \\
\frac{\Gamma, x: A \vdash M: B}{\Gamma \vdash \lambda x. M: A \rightarrow B} \\
\frac{\Gamma \vdash M: A \rightarrow B \quad \Gamma \vdash N: A}{\Gamma \vdash M N: B} \\
\frac{\Gamma \vdash M: A \quad \Gamma \vdash N: A}{\Gamma \vdash M a N: A} \\
\frac{\Gamma \vdash M: A}{\Gamma \vdash \boxed{a}. M: A}
\end{array}
\qquad
\begin{array}{c}
\frac{}{E \mid \Gamma, x: A \vdash x: A} \\
\frac{E \mid \Gamma, x: A \vdash M: B}{E \mid \Gamma \vdash \lambda x. M: A \rightarrow B} \\
\frac{E \mid \Gamma \vdash M: A \rightarrow B \quad E \mid \Gamma \vdash N: A}{E \mid \Gamma \vdash M N: B} \\
\frac{E, a \mapsto i \mid \Gamma \vdash M_i: A}{E, a \mapsto i \mid \Gamma \vdash M_{\top} a M_{\perp}: A} \\
\frac{E, a \mapsto \perp \mid \Gamma \vdash M: \overline{A} \rightarrow p \quad E, a \mapsto \top \mid \Gamma \vdash M: \overline{A} \rightarrow q}{E \mid \Gamma \vdash \boxed{a}. M: \overline{A} \rightarrow \frac{1}{2}(p+q)} \\
\frac{}{E \mid \Gamma \vdash M: \overline{A} \rightarrow 0} \\
\frac{E \mid \Gamma \vdash M: \overline{A} \rightarrow q}{E \mid \Gamma \vdash M: \overline{A} \rightarrow p} (p < q)
\end{array}$$

Figure 3-2: Comparison with simple types

The system is conservative over simple types, in which case E is empty. If we momentarily ignore the event context we observe that there is little change to the three deterministic rules. This similarity is notable since it does not always occur when introducing probability². The probabilistic rules, however, look considerably different and require a little more consideration.

First let us consider the *null* rule which only occurs on the probabilistic side:

$$\frac{}{E \mid \Gamma \vdash M: \overline{A} \rightarrow 0}$$

² In the literature the difference between call-by-name and call-by-value evaluation may require multiple rules for application, or rules which deal more directly with the probabilistic components [10].

The deterministic rules all preserve the value of types while the convex combinations found in the event rule will always produce 1 if that is the only possible input. This rule is therefore necessary to allow the non-trivial typing of terms with non-certain probability of termination. Even a non-terminating term such as $\Omega = (\lambda x. xx)(\lambda x. xx)$, can be typed under this rule. In fact, for any term we can freely choose a type of the form $A_1 \rightarrow \dots \rightarrow A_n \rightarrow 0$ meaning that provided it is given correctly-typed inputs the evaluation terminates with probability at least zero. This does not contradict our intuition since divergent terms have probability zero of terminating, as do those given incorrect inputs. This rule both allows the typing of any term and expands the usefulness of this type system by offering a basis for discussing partial termination.

Proposition 10 *For a term M of the following fragment of $\text{PE}\Lambda$:*

$$M, N ::= x \mid \lambda x. M \mid M N \mid \boxed{a}. M a N$$

There is a type derivation $\Gamma \vdash M : A$ in the simple type system if and only if there is a type derivation $\Gamma \vdash M : A_1$ in the probabilistic system. Where A_1 is given by:

$$\begin{aligned} o_1 &::= 1 \\ (A \rightarrow B)_1 &::= A_1 \rightarrow B_1 \end{aligned}$$

Proof. by induction on typing derivations. □

Note that the fragment of the calculus on which the simply typed system coincides with the probabilistic system restricted to probability 1 is exactly the image of embedding a call-by-name probabilistic system into $\text{PE}\Lambda$. The restriction to this fragment is necessary because terms such as $M a \Omega a N$ can be typed in the probabilistic system but cannot be dealt with by the simple type system: each subterm must be typeable to give strong normalisation. The probabilistic system is focused on head normalisation which, by contrast, allows unevaluated subterms to be discarded without typing. The call-by-name encoding excludes multiple occurrences of a single event, thus removing terms such as those considered above, while the restriction to probability 1 enforces full type-matching beyond matching on arguments.

The interaction between terms with genuinely different probabilities of termination yields more possible outcomes than when dealing with certainties. The key rules dealing with these interactions are the choice and event rules.

Now let us consider typing terms with a free event using the *choice* rule.

$$\frac{E, a \mapsto i \mid \Gamma \vdash M_i : A}{E, a \mapsto i \mid \Gamma \vdash M_{\top} a M_{\perp} : A}$$

Rather than putting absolute restrictions on the types of terms which can be combined by the conditional, the probabilistic system allows the combination of any terms in the presence of an *event valuation* provided the relevant subterm is typeable. This has the advantage of allowing terms such as $\boxed{a}. M a \Omega a N$, which have non-terminating subterms which are inaccessible to the computation, to be typed without weakening the bounds.

Finally we consider the event binder. In the simple type system the binder had absolutely no effect on the type. In the probabilistic system, by contrast, this rule is powerful and implements the probabilistic aspect of the calculus, in its entirety.

$$\frac{E, a \mapsto 0 \mid \Gamma \vdash M : \overline{A} \rightarrow p \quad E, a \mapsto 1 \mid \Gamma \vdash M : \overline{A} \rightarrow q}{E \mid \Gamma \vdash \boxed{a}. M : \overline{A} \rightarrow \frac{1}{2}(p + q)}$$

The type derivation branching here, where the choice is made, rather than where it is implemented, embodies the idea that the event is the moment where the coin is flipped. Note that either side of the branch must have its arguments *type-matched* to allow the rule to be applied. This is required because $(\boxed{a}. M)N$ reduces to $\frac{1}{2}(\pi_a^{\perp} M)N + \frac{1}{2}(\pi_a^{\top} M)N$ meaning each $\pi_a^i M$ should be capable of accepting the argument N .

This rule features a convex combination of types and this guarantees that the type of the conclusion is also a probability.

To illustrate the typing consider the term $\boxed{a}. M a \Omega$, which gives a fair probabilistic choice between M and Ω . We want this computation to be well-typed regardless of the choice, therefore M and Ω should have the same input types which will allow them to be applied to the same arguments. Suppose M has type

$A \rightarrow B \rightarrow p$; we may use the null rule to choose $A \rightarrow B \rightarrow 0$ for Ω , even though this term is not classically typeable. This results in $\boxed{a}.M \ a \ \Omega$ having type $A \rightarrow B \rightarrow \frac{1}{2}p$. Let us check this type against our understanding of the term: given arguments of type A and B , the computation chooses M with probability $\frac{1}{2}$ and so terminates with probability at least $\frac{1}{2}p$, which matches the bound provided by the type system.

3.2.1 Events in Context

In this subsection I will look in greater detail at the event context, its semantic intuition, its role in the type system, and the motivation behind the choices made in this formalism.

Intrinsic to the PEA is a vision of probabilistic computation which proceeds by sampling and then implementing choice via conditionals or projection. It is useful to consider the similarities between the event generator and lambda abstraction in this context. Each binds a kind of variable which may later undergo substitution during evaluation. The key difference between these notions is where the binder draws from on evaluation of the term. Abstraction consumes a value provided within the term via application. In contrast, $\boxed{a}$ does not have an argument within the term but still must consume from some ambient probability space. This idea is explored a little in [15] while the FMC [34] was a result of exploring the analogy between the generator and application further. We do not want to include the probability stream within this type system, however, it is still necessary to think about the behaviour of terms when an event is evaluated.

There are two fairly standard approaches to including the outcome of an event within a type system; event valuations and projections [10]. I have opted for the former partly because it is notationally much clearer than nested projections, but also to retain some induction on terms.

Upon sampling its event, the term $\boxed{a}.M$ reduces to either $\pi_+^a M$ or $\pi_-^a M$ with equal probability. These may, in turn, terminate with distinct probabilities. Rather than including projections in our type derivations, for the reasons de-

scribed above, we record the assignment of a truth value to a in an additional context: the *event valuation*. The type derivation then projects a term $M a N$ to M or to N according to the value of a in the event valuation.

The following example compares the typing derivations of the CBN and CBV interpretations of probabilistic lambda-term $(\lambda x. x a x)(I + \Omega)$

Call-by-name interpretation: $(\lambda x. \boxed{a}. x a x)(\boxed{b}. I b \Omega)$

$$\begin{array}{c}
 \frac{a \mapsto i \mid x : 1 \rightarrow \frac{1}{2} \vdash x : 1 \rightarrow \frac{1}{2}}{a \mapsto i \mid x : 1 \rightarrow \frac{1}{2} \vdash x a x : 1 \rightarrow \frac{1}{2}} \\
 \frac{x : 1 \rightarrow \frac{1}{2} \vdash \boxed{a}. x a x : 1 \rightarrow \frac{1}{2}}{\vdash \lambda x. \boxed{a}. x a x : (1 \rightarrow \frac{1}{2}) \rightarrow 1 \rightarrow \frac{1}{2}} \\
 \frac{\vdash \lambda x. \boxed{a}. x a x : (1 \rightarrow \frac{1}{2}) \rightarrow 1 \rightarrow \frac{1}{2} \quad \frac{\frac{b \mapsto \top \mid \vdash I : 1 \rightarrow 1}{b \mapsto \top \mid \vdash I b \Omega : 1 \rightarrow 1} \quad \frac{b \mapsto \perp \mid \vdash \Omega : 1 \rightarrow 0}{b \mapsto \perp \mid \vdash I b \Omega : 1 \rightarrow 0}}{\vdash \boxed{b}. I b \Omega : 1 \rightarrow \frac{1}{2}}}{\vdash (\lambda x. \boxed{a}. x a x)(\boxed{b}. I b \Omega) : 1 \rightarrow \frac{1}{2}}
 \end{array}$$

Call-by-value interpretation: $\boxed{a}. \boxed{b}. (\lambda x. x a x)(I b \Omega)$

$$\begin{array}{c}
 \frac{a \mapsto i, b \mapsto \top \mid \vdash (\lambda x. x)(I) : 1 \rightarrow 1}{a \mapsto i, b \mapsto \top \mid \vdash (\lambda x. x)(I b \Omega) : 1 \rightarrow 1} \quad \frac{\frac{a \mapsto i, b \mapsto \perp \mid \vdash \lambda x. x : (1 \rightarrow 0) \rightarrow 1 \rightarrow 0 \quad a \mapsto i, b \mapsto \perp \mid \vdash \Omega : 1 \rightarrow 0}{a \mapsto i, b \mapsto \perp \mid \vdash (\lambda x. x)(\Omega) : 1 \rightarrow 0}}{a \mapsto i, b \mapsto \perp \mid \vdash (\lambda x. x)(I b \Omega) : 1 \rightarrow 0} \\
 \frac{a \mapsto i \mid \vdash \boxed{b}. (\lambda x. x)(I b \Omega) : 1 \rightarrow \frac{1}{2}}{a \mapsto i \mid \vdash \boxed{b}. (\lambda x. x a x)(I b \Omega) : 1 \rightarrow \frac{1}{2}} \\
 \frac{a \mapsto i \mid \vdash \boxed{b}. (\lambda x. x a x)(I b \Omega) : 1 \rightarrow \frac{1}{2}}{\vdash \boxed{a}. \boxed{b}. (\lambda x. x a x)(I b \Omega) : 1 \rightarrow \frac{1}{2}}
 \end{array}$$

Note that, although the final type is the same across both valuations, the derivations take different routes, particularly in relation to the subterm $\lambda x. x a x$.

3.3 Main Theorem

Having fully defined the type system we can now state the main result for the probabilistically-typed $\text{PE}\Lambda$ and arguably the main theorem of this half of the thesis, the proof of which is left to Chapter 5.

Theorem 11 *If $M : \bar{A} \rightarrow p$ then M head-normalises with probability $\geq p$.*

To be more precise, we prove head reduction to a distribution a proportion of at least p of which is in head-normal form. The following restatement is equivalent but makes this strategy explicit:

Theorem 12 *For closed M , if $M: \overline{A} \rightarrow p$ then $M \rightarrow_{\text{h}} \mathcal{N}_0 + \mathcal{N}_1$ where all terms in $\mathcal{N}_0$ are head normal and $|\mathcal{N}_0| \geq p$.*

This result will follow directly from the equivalent theorem, Theorem 38, for the expanded probabilistic calculus with sequential composition, to be introduced in the next chapter.

3.4 Comparing to Counting Quantifiers

Our work on the probabilistic type system for PEA is a response to the system $\text{C}\lambda_{\rightarrow}$ by Antonelli and Dal Lago [2] which uses *counting quantifiers* to achieve similar results. The purpose of $\text{C}\lambda_{\rightarrow}$ was to showcase the power of counting quantifiers through a Curry-Howard correspondence linking the Probabilistic event λ -calculus with an intuitionistic counting propositional logic. We do not attempt a Curry-Howard correspondence in this work: we solely tackle the problem of lower-bounding the probability of termination. Our hope was that approaching this problem from a different perspective would present a simpler solution. In this section I compare the system presented in [2] with our system, in particular the lower bounds on the probability of termination which can be obtained by each.

The results in both systems focus on probabilistic termination of head reduction. In particular, on the lower bound on the probability of *head normalisation* which arises from the type system. It is here that we can directly compare the two systems. We refer to the main example in [2, Example 6.2] which requires intersection types to produce an exact bound. In contrast, our approach produces this bound directly in Example 14, all the while morally remaining within the realm of simple types. The apparent simplicity of our approach belies the strength of the results achieved.

$$\begin{array}{c}
\frac{\text{fe}(\mathbf{b}) \subseteq X}{\Gamma, x : \mathfrak{s} \vdash^X x : \mathbf{b} \multimap \mathfrak{s}} \\[10pt]
\frac{\{\Gamma \vdash^X M : \mathbf{b}_i \multimap \mathfrak{s}\}_{1 \leq i \leq n} \quad \mathbf{b} \models \bigvee_i \mathbf{b}_i}{\Gamma \vdash^X M : \mathbf{b} \multimap \mathfrak{s}} \\[10pt]
\frac{\Gamma \vdash^{X \cup \{a\}} M : \mathbf{c} \multimap \mathfrak{s} \quad \Gamma \vdash^{X \cup \{a\}} N : \mathbf{d} \multimap \mathfrak{s} \quad \mathbf{b} \models (\mathbf{c} \wedge a) \vee (\mathbf{d} \wedge \neg a)}{\Gamma \vdash^{X \cup \{a\}} M a N : \mathbf{b} \multimap \mathfrak{s}} \\[10pt]
\frac{\Gamma, x : \mathfrak{s} \vdash^X M : \mathbf{b} \multimap \text{C}^q \tau}{\Gamma \vdash^X \lambda x. M : \mathbf{b} \multimap \text{C}^q(\mathfrak{s} \Rightarrow \tau)} \\[10pt]
\frac{\Gamma \vdash^X M : \mathbf{c} \multimap \text{C}^q(\mathfrak{s} \Rightarrow \tau) \quad \Gamma \vdash^X N : \mathbf{d} \multimap \mathfrak{s} \quad \mathbf{b} \models \mathbf{c} \wedge \mathbf{d}}{\Gamma \vdash^X MN : \mathbf{b} \multimap \text{C}^q \tau} \\[10pt]
\frac{\Gamma \vdash^{X \cup \{a\}} M : \mathbf{b} \wedge \mathbf{d} \multimap \text{C}^q \sigma \quad \mu(\mathbf{d}) \geq s}{\Gamma \vdash^X \boxed{a}. M : \mathbf{b} \multimap \text{C}^{qs} \sigma}
\end{array}$$

Figure 3-3: Typing rules for $\text{C}\lambda_{\rightarrow}$

Our approach avoids the need for counting quantifiers and the associated machinery used to collect the different probabilistic branches of a computation. In logic, quantifiers usually have two forms; universal quantifiers which say that a proposition holds for all satisfying assignments, and existential quantifiers which state that there exists a satisfying assignment which witnesses the property. Counting quantifiers generalise these concepts, expressing the existence of a multiplicity of unique witnesses to a statement. In the context of probabilistic types counting quantifiers are used as a means to express a proportional truth [3, 1], in other words to count the number of assignments satisfying the required proposition.

The typing rules for $\text{C}\lambda_{\rightarrow}$ are shown in figure 3-3. We have taken one liberty with the notation from the original paper: replacing both the choice $\oplus_a^i$ and the Boolean event variable x_a^i with a . This change is permissible because the index i in the original is unchanged by steps of the type derivation. The index acts as a convention for grouping together consecutive generators, which would remain

together throughout reduction. There is no intrinsic reason to have x_a^i and x_a^j with $i \neq j$ rather than distinct events a and b .

Our change to the indexing of events can be extended to the semantics – without changing any of the measure theoretic results – by considering the induced measure on the cylinder sets accessed by the term. In the original paper the semantics consisted of Boolean formulas ϕ over the variables x_a^i , with valuations taken from the space $(2^{\mathbb{N}})^X$, where X is a (finite) set of events. The finite nature of terms guarantees that the indices are bounded so the space of pairs (a, i) is finite. It follows that replacing distinct indices with distinct events does not reduce the expressivity of the calculus and results in a finite set of events. The cylinder set measure required to deal with infinite streams, collapses to a uniform probability measure on the finite sample space 2^X .

Cosmetic changes aside, let us expand on the semantics. Given an event valuation $E \in 2^X$ the semantic entailment of a Boolean formula ϕ over event variables from the domain X may be characterised syntactically as:

$$\begin{array}{c} \overline{E, a \mapsto \top \models a} \quad \overline{E, a \mapsto \perp \models \neg a} \quad \overline{E \models \top} \\[10pt] \frac{E \models \phi \quad E \models c}{E \models \phi \wedge c} \quad \frac{E \models \phi}{E \models \phi \vee c} \end{array}$$

We denote the set of event valuations which entail a Boolean formula ϕ by $\llbracket \phi \rrbracket_X = \{E \in 2^X \mid E \models \phi\}$. These correspond directly to the event contexts in our system.

Consider the uniform measure on 2^X : for a set $\mathcal{E} \subseteq 2^X$ the measure $\mu_X(\mathcal{E})$ is given by

$$\mu_X(\mathcal{E}) = \frac{|\mathcal{E}|}{2^{|X|}}$$

where $|S|$ denotes the size of a set S . The measure $\mu(\phi)$ from [2] coincides with $\mu_X(\llbracket \phi \rrbracket_X)$ where X is the domain of ϕ . Note that enlarging X to a superset of the domain does not change the value of this measure: since all possible valuations of extra events are permissible, the top and bottom of the fraction increase proportionally.

Let us think about how the two systems deal with probability. In $C\lambda_{\rightarrow}$ probability is introduced to the syntax of types through counting quantifiers C^p , which are essentially external to the type system. In contrast probabilities form the base types of $PE\Lambda^{\mathbb{P}\rightarrow}$. These systems of types can be shown to be isomorphic, through an interpretation of $C\lambda_{\rightarrow}$ in our system. Generally speaking, types of the form $C^p(\mathfrak{s}_1 \rightarrow \dots \rightarrow \mathfrak{s}_n \rightarrow o)$, with a fixed outer counting quantifier, map onto $PE\Lambda^{\mathbb{P}\rightarrow}$ types of the form $A_1 \rightarrow .. \rightarrow A_n \rightarrow p$ moving the probability from the outermost position, to the innermost.

Definition 13 The encoding $\llbracket - \rrbracket$ of the types $\mathfrak{s}$ and typing contexts Γ of $C\lambda_{\rightarrow}$ into our setting is as follows:

$$\begin{aligned} \llbracket C^q(o) \rrbracket &= q \\ \llbracket C^q(\mathfrak{s} \Rightarrow \tau) \rrbracket &= \llbracket \mathfrak{s} \rrbracket \rightarrow \llbracket C^q(\tau) \rrbracket \end{aligned} \quad \llbracket x_1 : \mathfrak{s}_1, \dots, x_n : \mathfrak{s}_n \rrbracket = \mathbf{x}_1 : \llbracket \mathfrak{s}_1 \rrbracket, \dots, \mathbf{x}_n : \llbracket \mathfrak{s}_n \rrbracket$$

Now the stage is set: Boolean formulas $\mathfrak{b}$ connect to event valuations and we have a translation between type systems. We can now formally state a relationship between the two systems. We leave the proof that our type system provides a lower bound for termination until the next chapter. For now, we work under the supposition that it does and show that the bounds provided are stronger than those attained by counting quantifiers alone.

First, let us look at an example which counting quantifiers could only weakly bound.

Example 14 We reprise the example $t[a, b]$ from [2, Section 6.2], here as T . In Figure 3-4 we derive the following type.

$$\boxed{a}. \boxed{b}. \boxed{c}. T : 1 \rightarrow \frac{3}{8} \quad \text{where} \quad T = ((I \, c \, \Omega) \, b \, \Omega) \, a \, (\Omega \, b \, I)$$

The bound of $\frac{3}{8}$ achieved here is strictly greater than the $\frac{1}{4}$ obtained by counting quantifiers, showing that the two systems are distinct. However, this is not sufficient to show that our bound is tighter, because some may be greater and some less than the bounds provided by $C\lambda_{\rightarrow}$. To show this it is necessary to prove that our bound is at least that of $C\lambda_{\rightarrow}$ everywhere; this is done by structural

induction on type derivations. Once that is proven this single example of a stricter bound will gain the significance of showing a stronger typing for termination.

Proposition 15 *If $E \in \llbracket \mathfrak{b} \rrbracket_X$ then*

$$\Gamma \vdash^X M : \mathfrak{b} \multimap \mathfrak{s} \quad \text{implies} \quad E \mid \llbracket \Gamma \rrbracket \vdash M : \llbracket \mathfrak{s} \rrbracket .$$

Proof. We proceed by induction on the typing derivation for $\Gamma \vdash^X M : \mathfrak{b} \multimap \mathfrak{s}$.

Identity rule

$$\frac{\text{fe}(\mathfrak{b}) \subseteq X}{\Gamma, x : \mathfrak{s} \vdash^X x : \mathfrak{b} \multimap \mathfrak{s}}$$

Regardless of the event context $E \in \llbracket \mathfrak{b} \rrbracket_X$ the required derivation is simply an instance of the corresponding identity rule:

$$\overline{E \mid \llbracket \Gamma \rrbracket, x : \llbracket \mathfrak{s} \rrbracket \vdash x : \llbracket \mathfrak{s} \rrbracket}$$

Abstraction rule

$$\frac{\Gamma, x : \mathfrak{s} \vdash^X M : \mathfrak{b} \multimap C^q \tau}{\Gamma \vdash^X \lambda x. M : \mathfrak{b} \multimap C^q(\mathfrak{s} \Rightarrow \tau)}$$

Returning to the definitions of the embedding: $\llbracket C^q(\mathfrak{s} \Rightarrow \tau) \rrbracket = \llbracket \mathfrak{s} \rrbracket \rightarrow \llbracket C^q(\tau) \rrbracket$.

Thus for $E \in \llbracket \mathfrak{b} \rrbracket_X$ we may construct the required derivation as:

$$\frac{E \mid \llbracket \Gamma \rrbracket, x : \llbracket \mathfrak{s} \rrbracket \vdash M : \llbracket C^q \tau \rrbracket}{E \mid \llbracket \Gamma \rrbracket \vdash \lambda x. M : \llbracket \mathfrak{s} \rrbracket \rightarrow \llbracket C^q \tau \rrbracket}$$

Application rule

$$\frac{\Gamma \vdash^X M : \mathfrak{c} \multimap C^q(\mathfrak{s} \Rightarrow \tau) \quad \Gamma \vdash^X N : \mathfrak{d} \multimap \mathfrak{s} \quad \mathfrak{b} \models \mathfrak{c} \wedge \mathfrak{d}}{\Gamma \vdash^X MN : \mathfrak{b} \multimap C^q \tau}$$

Let $E \in \llbracket \mathfrak{b} \rrbracket_X$. Since $\mathfrak{b} \models \mathfrak{c} \wedge \mathfrak{d}$ we have $E \in \llbracket \mathfrak{c} \rrbracket_X$ and $E \in \llbracket \mathfrak{d} \rrbracket_X$. Again using that $\llbracket C^q(\mathfrak{s} \Rightarrow \tau) \rrbracket = \llbracket \mathfrak{s} \rrbracket \rightarrow \llbracket C^q(\tau) \rrbracket$, we construct the required

$$\begin{aligned}
D_{\perp, \perp, \perp} &= \frac{\frac{\frac{\frac{\overline{a \mapsto \perp, b \mapsto \perp, c \mapsto \perp \mid x: 1 \vdash x: 1}}{a \mapsto \perp, b \mapsto \perp, c \mapsto \perp \mid \vdash \lambda x. x: 1 \rightarrow 1}}{a \mapsto \perp, b \mapsto \perp, c \mapsto \perp \mid \vdash I c \Omega: 1 \rightarrow 1}}{a \mapsto \perp, b \mapsto \perp, c \mapsto \perp \mid \vdash (I c \Omega) b \Omega: 1 \rightarrow 1}}{a \mapsto \perp, b \mapsto \perp, c \mapsto \perp \mid \vdash ((I c \Omega) b \Omega) a (\Omega b I): 1 \rightarrow 1} \\
D_{\perp, \perp} &= \frac{\frac{D_{\perp, \perp, \perp}}{a \mapsto \perp, b \mapsto \perp, c \mapsto \perp \mid \vdash T: 1 \rightarrow 1} \quad \frac{\overline{a \mapsto \perp, b \mapsto \perp, c \mapsto \top \mid \vdash T: 1 \rightarrow 0}}{a \mapsto \perp, b \mapsto \perp \mid \vdash \boxed{c}. T: 1 \rightarrow \frac{1}{2}}}{a \mapsto \perp, b \mapsto \perp \mid \vdash \boxed{c}. T: 1 \rightarrow \frac{1}{2}} \\
D_{\perp} &= \frac{\frac{D_{\perp, \perp}}{a \mapsto \perp, b \mapsto \perp \mid \vdash \boxed{c}. T: 1 \rightarrow \frac{1}{2}} \quad \frac{\overline{a \mapsto \perp, b \mapsto \top \mid \vdash \boxed{c}. T: 1 \rightarrow 0}}{a \mapsto \perp \mid \vdash \boxed{b}. \boxed{c}. T: 1 \rightarrow \frac{1}{4}}}{a \mapsto \perp \mid \vdash \boxed{b}. \boxed{c}. T: 1 \rightarrow \frac{1}{4}} \\
D_{\top, \top, i} &= \frac{\frac{\frac{\frac{\overline{a \mapsto \top, b \mapsto \top, c \mapsto i \mid x: 1 \vdash x: 1}}{a \mapsto \top, b \mapsto \top, c \mapsto i \mid \vdash \lambda x. x: 1 \rightarrow 1}}{a \mapsto \top, b \mapsto \top, c \mapsto i \mid \vdash \Omega b I: 1 \rightarrow 1}}{a \mapsto \top, b \mapsto \top, c \mapsto i \mid \vdash ((I c \Omega) b \Omega) a (\Omega b I): 1 \rightarrow 1}}{a \mapsto \top, b \mapsto \top, c \mapsto i \mid \vdash ((I c \Omega) b \Omega) a (\Omega b I): 1 \rightarrow 1} \\
D_{\top, \top} &= \frac{\frac{D_{\top, \top, \perp}}{a \mapsto \top, b \mapsto \top, c \mapsto \perp \mid \vdash T: 1 \rightarrow 1} \quad \frac{D_{\top, \top, \top}}{a \mapsto \top, b \mapsto \top, c \mapsto \top \mid \vdash T: 1 \rightarrow 1}}{a \mapsto \top, b \mapsto \top \mid \vdash \boxed{c}. T: 1 \rightarrow 1} \\
D_{\top} &= \frac{\frac{\overline{a \mapsto \top, b \mapsto \perp \mid \vdash \boxed{c}. T: 1 \rightarrow 0}}{a \mapsto \top, b \mapsto \perp \mid \vdash \boxed{c}. T: 1 \rightarrow 1} \quad \frac{D_{\top, \top}}{a \mapsto \top, b \mapsto \top \mid \vdash \boxed{c}. T: 1 \rightarrow 1}}{a \mapsto \top \mid \vdash \boxed{b}. \boxed{c}. T: 1 \rightarrow \frac{1}{2}} \\
D &= \frac{\frac{D_{\perp}}{a \mapsto \perp \mid \vdash \boxed{b}. \boxed{c}. T: 1 \rightarrow \frac{1}{4}} \quad \frac{D_{\top}}{a \mapsto \top \mid \vdash \boxed{b}. \boxed{c}. T: 1 \rightarrow \frac{1}{2}}}{\mid \vdash \boxed{a}. \boxed{b}. \boxed{c}. T: 1 \rightarrow \frac{3}{8}}
\end{aligned}$$

Figure 3-4: Typing derivations for Example 14.

derivation as follows.

$$\frac{E \mid \llbracket \Gamma \rrbracket \vdash \textcolor{red}{M} : \llbracket \mathfrak{s} \rrbracket \rightarrow \llbracket \text{C}^q \tau \rrbracket \quad E \mid \llbracket \Gamma \rrbracket \vdash \textcolor{red}{N} : \llbracket \mathfrak{s} \rrbracket}{E \mid \llbracket \Gamma \rrbracket \vdash \textcolor{red}{MN} : \llbracket \text{C}^q \tau \rrbracket}$$

Structural rule

$$\frac{\left\{ \Gamma \vdash^X M : \textcolor{blue}{b}_i \rightarrow \mathfrak{s} \right\}_{1 \leq i \leq n} \quad \textcolor{blue}{b} \models \bigvee_i \textcolor{blue}{b}_i}{\Gamma \vdash^X M : \textcolor{blue}{b} \rightarrow \mathfrak{s}}$$

There are two cases: $n = 0$ and $n \geq 1$.

When n is zero $\textcolor{blue}{b} \models \perp$, therefore $\llbracket \textcolor{blue}{b} \rrbracket_X = \emptyset$. Since there is no possible event valuation the statement follows vacuously.

For $n \geq 1$ we assume $\llbracket \textcolor{blue}{b} \rrbracket$ is nonempty and fix $E \in \llbracket \textcolor{blue}{b} \rrbracket_X$. Considering the semantic condition $\textcolor{blue}{b} \models \bigvee_i \textcolor{blue}{b}_i$ we see that since E entails a Boolean disjunction it must entail one of its components so $E \in \llbracket \textcolor{blue}{b}_i \rrbracket_X$ for some $i \leq n$. Applying the inductive hypothesis to the premiss $\Gamma \vdash^X M : \textcolor{blue}{b}_i \rightarrow \mathfrak{s}$ from the rule above gives the required derivation:

$$E \mid \llbracket \Gamma \rrbracket \vdash \textcolor{red}{M} : \llbracket \mathfrak{s} \rrbracket$$

Plus rule

$$\frac{\Gamma \vdash^{X \cup \{a\}} M : \textcolor{blue}{c} \rightarrow \mathfrak{s} \quad \Gamma \vdash^{X \cup \{a\}} N : \textcolor{blue}{d} \rightarrow \mathfrak{s} \quad \textcolor{blue}{b} \models (\textcolor{blue}{c} \wedge a) \vee (\textcolor{blue}{d} \wedge \neg a)}{\Gamma \vdash^{X \cup \{a\}} M \textcolor{blue}{a} N : \textcolor{blue}{b} \rightarrow \mathfrak{s}}$$

Since a is singled out in this rule let us consider an event valuation with a explicit; let $(E, a \mapsto i) \in \llbracket \textcolor{blue}{b} \rrbracket_{X \cup \{a\}}$ where i is one of $\top, \perp$. The semantic condition tells us that $\textcolor{blue}{b} \models (\textcolor{blue}{c} \wedge a) \vee (\textcolor{blue}{d} \wedge \neg a)$ giving two cases. If $(E, a \mapsto i) \in \llbracket \textcolor{blue}{c} \wedge a \rrbracket_{X \cup \{a\}}$ we have $i = \top$ and $(E, a \mapsto \top) \in \llbracket \textcolor{blue}{c} \rrbracket_{X \cup \{a\}}$. Otherwise $(E, a \mapsto i) \in \llbracket \textcolor{blue}{d} \wedge \neg a \rrbracket_{X \cup \{a\}}$ meaning that $i = \perp$ and $(E, a \mapsto \perp) \in \llbracket \textcolor{blue}{d} \rrbracket_{X \cup \{a\}}$. Since choice is not a branching rule in our type system each case matches with its corresponding premiss and via the induction hypothesis produces a derivation of the required form.

$$\frac{E, a \mapsto \top \mid \llbracket \Gamma \rrbracket \vdash M : \llbracket \mathfrak{s} \rrbracket}{E, a \mapsto \top \mid \llbracket \Gamma \rrbracket \vdash M \text{ a } N : \llbracket \mathfrak{s} \rrbracket} \quad \frac{E, a \mapsto \perp \mid \llbracket \Gamma \rrbracket \vdash N : \llbracket \mathfrak{s} \rrbracket}{E, a \mapsto \perp \mid \llbracket \Gamma \rrbracket \vdash M \text{ a } N : \llbracket \mathfrak{s} \rrbracket}$$

Counting rule

$$\frac{\Gamma \vdash^{X \cup \{a\}} M : \mathfrak{t} \wedge \mathfrak{d} \mapsto C^q \sigma \quad \mu(\mathfrak{d}) \geq s}{\Gamma \vdash^X \boxed{a}.M : \mathfrak{t} \mapsto C^{qs} \sigma}$$

As before we start by fixing an event valuation $E \in \llbracket \mathfrak{t} \rrbracket_X$, however, note that the domain of the premiss includes a , which is assumed not to be in X . The singleton $\{a\}$ is however the domain of $\mathfrak{d}$. This gives us cases on the possible values of $\mathfrak{d}$, considering evaluation up to Boolean equivalence there are only four: $\perp$, a , $\neg a$, or $\top$. The induction hypothesis will allow us to assume that $\llbracket C^q \sigma \rrbracket$ is of the form $\overline{A} \rightarrow q$ throughout.

- $\mathfrak{d}$ equivalent to $\perp$ ensures that $\mu(\mathfrak{d}) = 0$. So the required derivation is given by the null rule.

$$\overline{E \mid \llbracket \Gamma \rrbracket \vdash \boxed{a}.M : \overline{A} \rightarrow 0}$$

- $\mathfrak{d}$ equivalent to a implies that $\llbracket \mathfrak{d} \rrbracket_{\{a\}} = \{a \mapsto \top\}$ which in turn gives us that $\mu(\mathfrak{d}) = \frac{1}{2}$ and $(E, a \mapsto \top) \in \llbracket \mathfrak{t} \wedge \mathfrak{d} \rrbracket_{X \cup \{a\}}$. The required derivation can thus be constructed as follows, with the left premiss provided by the inductive hypothesis.

$$\frac{E, a \mapsto \top \mid \llbracket \Gamma \rrbracket \vdash M : \overline{A} \rightarrow q \quad \overline{E, a \mapsto \perp \mid \llbracket \Gamma \rrbracket \vdash M : \overline{A} \rightarrow 0}}{\frac{E \mid \llbracket \Gamma \rrbracket \vdash \boxed{a}.M : \overline{A} \rightarrow \frac{1}{2}q}{E \mid \llbracket \Gamma \rrbracket \vdash \boxed{a}.M : \overline{A} \rightarrow qs}^{\text{low } (s \leq \frac{1}{2})}$$

- $\mathfrak{d}$ equivalent to $\neg a$ works almost exactly as above: $\llbracket \mathfrak{d} \rrbracket_{\{a\}} = \{a \mapsto \perp\}$ so $\mu(\mathfrak{d}) = \frac{1}{2}$ and $(E, a \mapsto \perp) \in \llbracket \mathfrak{t} \wedge \mathfrak{d} \rrbracket_{X \cup \{a\}}$. This time the induction hypothesis is applied to the right premiss.

$$\frac{\frac{E, a \mapsto \top \mid \llbracket \Gamma \rrbracket \vdash M : \overline{A} \rightarrow 0 \quad E, a \mapsto \perp \mid \llbracket \Gamma \rrbracket \vdash M : \overline{A} \rightarrow q}{E \mid \llbracket \Gamma \rrbracket \vdash \boxed{a}. M : \overline{A} \rightarrow \frac{1}{2}q}}{E \mid \llbracket \Gamma \rrbracket \vdash \boxed{a}. M : \overline{A} \rightarrow qs}^{\text{low } (s \leq \frac{1}{2})}$$

- $\mathcal{d}$ equivalent to $\top$ implies that $\mu(\mathcal{d}) = 1$ and $(E, a \mapsto i) \in \llbracket \mathcal{b} \wedge \mathcal{d} \rrbracket_{X \cup \{a\}}$ for $i \in \{\top, \perp\}$. The inductive hypothesis provides two premisses for the required derivation.

$$\frac{E, a \mapsto \top \mid \llbracket \Gamma \rrbracket \vdash M : \overline{A} \rightarrow q \quad E, a \mapsto \perp \mid \llbracket \Gamma \rrbracket \vdash M : \overline{A} \rightarrow q}{E \mid \llbracket \Gamma \rrbracket \vdash \boxed{a}. M : \overline{A} \rightarrow q}}{E \mid \llbracket \Gamma \rrbracket \vdash \boxed{a}. M : \overline{A} \rightarrow qs}$$

□

As already discussed, Example 14 and its counterpart in [2, Section 6.2] show that the bounds given by these two type systems differ on some terms. More, that there are terms for which $\text{PE}\Lambda^{\mathbb{P} \rightarrow}$ gives a strictly higher bound. Meanwhile Proposition 15 tells us that for any lower bound given by $\text{C}\Lambda_{\rightarrow}$ it is possible to construct a derivation in $\text{PE}\Lambda^{\mathbb{P} \rightarrow}$ which also produces that bound. These combine to give the desired result comparing bounds.

Corollary 16 *For a given closed term M the type system $\text{PE}\Lambda^{\mathbb{P} \rightarrow}$ gives the same or higher termination bounds than $\text{C}\Lambda_{\rightarrow}$.*

Where does this improvement come from? The main difference between the two systems is where branching occurs. $\text{C}\Lambda_{\rightarrow}$ branches at *choice* terms $M a N$ or in a contraction rule. In $\text{PE}\Lambda^{\mathbb{P} \rightarrow}$ meanwhile, the rule for $\boxed{a}. M$ has branching between alternative outcomes, this rule in particular allows *branches with different termination bounds*.

$$\frac{E, a \mapsto \top \mid \llbracket \Gamma \rrbracket \vdash M : \overline{A} \rightarrow p \quad E, a \mapsto \perp \mid \llbracket \Gamma \rrbracket \vdash M : \overline{A} \rightarrow q}{E \mid \llbracket \Gamma \rrbracket \vdash \boxed{a}. M : \overline{A} \rightarrow \frac{1}{2}p + \frac{1}{2}q}$$

Let us try to simulate this rule in $\text{C}\lambda_{\rightarrow}$:

$$\frac{\Gamma \vdash^{X \cup \{a\}} M : \mathbf{b} \wedge a \multimap \text{C}^p \sigma \quad \Gamma \vdash^{X \cup \{a\}} M : \mathbf{b} \wedge \neg a \multimap \text{C}^p \sigma \quad \mathbf{b} \models (\mathbf{b} \wedge a) \vee (\mathbf{b} \wedge \neg a)}{\frac{\Gamma \vdash^{X \cup \{a\}} M : \mathbf{b} \multimap \text{C}^p \sigma}{\Gamma \vdash^X [\mathbf{a}]. M : \mathbf{b} \multimap \text{C}^p \sigma}}$$

The branching of the generator rule is captured here by a *contraction* on the two premisses; this requires that the probabilities be equal, $p = q$. The generator is subsequently added through the counting rule with $\mathbf{d} = \top$ and $\mu(\mathbf{d}) = 1$.

This matching of bounds is exactly the issue illustrated by Example 14. The two calculi give the same bounds up until the final branching, where, in order to combine distinct bounds, $\text{C}\lambda_{\rightarrow}$ requires that they be lowered to match.

We reprise the typing from [2, Section 6.2] to further investigate the discrepancy. For clarity we omit the initial derivations which simply combine the derivations for $\vdash^{\{a,b,c\}} I : \top \multimap \text{C}^1 \sigma$ and $\vdash^{\{a,b,c\}} \Omega : \perp \multimap \text{C}^1 \sigma$ using the plus rule.

$$\frac{\frac{\vdash^{\{a,b,c\}} T : a \wedge b \wedge c \multimap \text{C}^1 \sigma}{\vdash^{\{a,b\}} [\mathbf{c}]. T : a \wedge b \multimap \text{C}^{\frac{1}{2}} \sigma}}{\vdash^{\{a\}} [\mathbf{b}]. [\mathbf{c}]. T : a \multimap \text{C}^{\frac{1}{4}} \sigma}} \quad \frac{\frac{\vdash^{\{a,b,c\}} T : \neg a \wedge \neg b \multimap \text{C}^1 \sigma}{\vdash^{\{a,b\}} [\mathbf{c}]. T : \neg a \wedge \neg b \multimap \text{C}^1 \sigma}}{\vdash^{\{a\}} [\mathbf{b}]. [\mathbf{c}]. T : \neg a \multimap \text{C}^{\frac{1}{2}} \sigma}}$$

Recalling that $T = ((I \mathbf{c} \Omega) b \Omega) a (\Omega b I)$, the formulas $a \wedge b \wedge c$ and $\neg a \wedge \neg b$ can be thought of as picking out the two occurrences of I in T . These two subderivations for $[\mathbf{b}]. [\mathbf{c}]. T$, assign probabilities of $\frac{1}{4}$ and $\frac{1}{2}$ respectively. These may only be combined in a contraction as described above by *lowering* the second probability to match the $\frac{1}{4}$ of the first.

The ability to combine distinct probabilities is what allows $\text{PE}\Lambda^{\mathbb{P} \rightarrow}$ to produce the actual termination probability of $\frac{3}{8}$.

This analysis suggests that this issue with $\text{C}\lambda_{\rightarrow}$ may be fixed by adopting the generator typing rule of $\text{PE}\Lambda^{\mathbb{P} \rightarrow}$, adjusted appropriately as follows.

$$\frac{\Gamma \vdash^{X \cup \{a\}} M : \mathbf{c} \wedge a \multimap \text{C}^p \sigma \quad \Gamma \vdash^{X \cup \{a\}} M : \mathbf{d} \wedge \neg a \multimap \text{C}^q \sigma \quad \mathbf{c} \models \mathbf{c} \vee \mathbf{d}}{\Gamma \vdash^X [\mathbf{a}]. M : \mathbf{c} \multimap \text{C}^{\frac{1}{2}p + \frac{1}{2}q} \sigma}$$

In [2] the weakness of these bounds is dealt with by extending the type system to an intersection type system. Our probabilistic type system cannot match the results achieved on the intersection system. In part this is because our type system cannot assign multiple types to the same sub-term and thus cannot type the full range of terms. For instance $(\lambda f. f(fI))(\boxed{a}. I a \lambda y. \Omega)$ terminates with probability a quarter. The best bound our type system gives, however, is zero; because we cannot assign a fixed type to the sub-term f so are forced to type it using the zero rule. This limitation is a feature of remaining within a morally simple type system. In other kinds of type system this is dealt with by creating a list of potential types or expanding the grammar to include type variables which do not have a specific value in terms of a base type but allow one to indicate the type of a outcome dependent on the type of the the input.

Chapter 4

The Machine

In this chapter we extend the probabilistic event λ -calculus with *sequencing*. This involves the introduction of a *skip* term $\star$ and *sequential composition* of terms $N; M$. Introducing sequential composition will allow us to fully capture call-by-value probabilistic behaviour, while the skip term provides an intrinsic notion of termination.

As previously discussed, the explicit integration of call-by-name and call-by-value behaviour is needed in a probabilistic calculus. The introduction of new constructs – in the case of PEA decomposition of the probabilistic operator – is necessary to avoid imposing a global strategy. Other examples of such constructs include the separate consideration of a **cbv**- and a **cbn**-probabilistic sum by Faggian and Ronchi Della Rocca [26], and the addition of a secondary **cbv**-application by Antonelli, Dal Lago, and Pistone [2]. Such extensions are useful but may be in danger of lacking justification. It is difficult to point to a correct approach – since constructs can often express each other – however, we wish to avoid introducing ad hoc constructs: constructs which make things work but make no reference to an underlying intuition or semantics. We argue that the machine semantics supports the use of a sequential operator to model call-by-value reduction.

The use of sequentiality is in part motivated by our study of Antonelli et al’s construction with counting quantifiers [2]. The arguments presented therein – for

considering head-normalisation and for separating out call-by-value application – point to a sequential interpretation. We came to the conclusion that a perspective based in the Functional Machine Calculus [34], with its focus on the machine as operational semantics, would allow us to achieve at least the same probabilistic termination results without requiring counting quantifiers.

The *Functional Machine Calculus* (FMC) is a sequential calculus designed to handle effects, the terms of which operate a modified Krivine machine [42]. As well as implementing application and abstraction by pushing and popping terms from a stack, the machine is also augmented with locations: extra stacks used by effectful operators. The original aim of the Krivine machine was to formalise call-by-name semantics and in the process it provided a model for head reduction. It is for this reason that it is an obvious inspiration for our proof of head normalisation. Furthermore, the FMC has a natural means to express both **cbn** and **cbv** behaviour: the **cbn** λ -calculus as a fragment, and the **cbv** λ -calculus encoded via the syntax. This allows us to avoid introducing ad hoc constructs to interpret the call-by-value calculus, instead implementing a principled interpretation through sequential composition. Finally the FMC provides a new notion of *successful termination*, which is absent in the standard λ -calculus. This leads to a natural way to study probabilistic termination.

Following this motivation we adopt elements of the FMC into PEA to give the *sequential probabilistic event λ -calculus* (SPEA), defined in this chapter. Note that probabilistic computation is one of the effects which can be modelled in the FMC, in particular the PEA can be encoded in the FMC through a read only location: the generator $\boxed{a}.M$ of the PEA is interpreted in the FMC as an abstraction $\text{rnd}\langle a \rangle.M$ where the indicated location contains a stream of random values. The SPEA thus embeds into the FMC with two locations.

4.1 Sequencing

In this section we will define the sequential variant of the calculus. At this stage we also make a slight notational change, from application and abstraction to *push*

and *pop* operators. Application is written as $[N].M = M N$ while abstraction is $\langle x \rangle.M = \lambda x.M$. These match up to the operational intuition of *push* and *pop*: the term $[N].M$ can be read as “Push N onto the stack then run M .” while the term $\langle x \rangle.M$ means “Pop the top value from the stack and bind it to x .”

Definition 17 The *sequential probabilistic event λ -calculus* $\text{SPE}\Lambda$ is given by the following grammar.

$$M, N, P ::= x \mid \langle x \rangle.M \mid [N].M \mid \boxed{a}.M \mid M a N \mid \star \mid N ; M$$

With the above conversion in mind it can be seen that $\text{SPE}\Lambda$ is an extension of the $\text{PE}\Lambda$ with two operators: a binary sequencing operator $;$ and a constant skip term $\star$ which indicates completed evaluation.

Two notational conventions are worth mentioning:

- Where the standard lambda calculus syntax is used, it is taken to be shorthand for these stack operations.
- We may omit a trailing $\star$, for example writing $\langle x \rangle.[M].\star$ as $\langle x \rangle.[M]$ instead.

Projections translate and extend from $\text{PE}\Lambda$ to the $\text{SPE}\Lambda$ as follows, where $a \neq b$.

$$\begin{aligned} \pi_i^a(x) &= x & \pi_{\top}^a(M a N) &= \pi_{\top}^a(M) \\ \pi_i^a(\langle x \rangle.M) &= \langle x \rangle.\pi_i^a(M) & \pi_{\perp}^a(M a N) &= \pi_{\perp}^a(N) \\ \pi_i^a([N].M) &= [\pi_i^a(N)].\pi_i^a(M) & \pi_i^a(\boxed{a}.M) &= \boxed{a}.M \\ \pi_i^a(M b N) &= \pi_i^a(M) b \pi_i^a(N) & \pi_i^a(\boxed{b}.M) &= \boxed{b}.\pi_i^a(M) \\ \pi_i^a(\star) &= \star & \pi_i^a(N ; M) &= \pi_i^a(N) ; \pi_i^a(M) \end{aligned}$$

Contexts C are similarly extended to feature sequential composition while *head contexts* H and *applicative contexts* A are identical to those in the $\text{PE}\Lambda$. This consistency will allow all results on head reduction to project easily back to that

calculus.

$$\begin{aligned}
C &::= \{\} \mid \langle x \rangle . C \mid [M] . C \mid [C] . M \mid \boxed{a} . C \mid \\
&\quad C \mathbin{a} M \mid M \mathbin{a} C \mid C ; M \mid M ; C \\
H &::= \langle x \rangle . H \mid A \\
A &::= \{\} \mid [M] . A
\end{aligned}$$

In the stack notation head contexts are of the form:

$$\langle x_1 \rangle \dots \langle x_n \rangle . [M_1] \dots [M_m] . \{\}$$

The interaction of sequentiality with the other constructs is governed by the following *sequencing* reduction rules. These rules make sequential composition right-associative, and let the *prefixing* of *push*, *pop*, and the event generator propagate past it. These rules will serve to make the first action performed by the abstract machine be the outermost construct. This is necessary for head reduction since the other reduction rules do not apply within the scope of a sequencing operator.

$$\begin{aligned}
\star ; P &\rightarrow_{\sigma} P \\
\langle x \rangle . M ; P &\rightarrow_{\sigma} \langle x \rangle . (M ; P) \quad (x \notin \text{fv}(P)) \\
[N] . M ; P &\rightarrow_{\sigma} [N] . (M ; P) \\
\boxed{a} . M ; P &\rightarrow_{\sigma} \boxed{a} . (M ; P) \quad (a \notin \text{fe}(P)) \\
(N ; M) ; P &\rightarrow_{\sigma} N ; (M ; P)
\end{aligned}$$

The *sequencing* relation ($\rightarrow_{\sigma}$) is given by closing these under all contexts C , and *head sequencing* ($\rightarrow_{\sigma\text{h}}$) by closing under head contexts H only.

Lemma 18 *The head sequencing reduction ($\rightarrow_{\sigma\text{h}}$) is terminating.*

Proof. We impose a positive integer measure on sequential terms given by the sum of the symbolic length $|M|$ for every sub-term of the form $M ; N$ and show that it is reduced by sequencing reduction. With this proof strategy head contexts

do not contribute to the measure, thus the full claim follows from the base cases.

In the skip case the measure is reduced by one since the skip term of length one is removed from the sum. The pop, push and generator cases are similar in that one of the sub-terms contributing to the sum has its length decreased, and no new sub-terms of the correct form are created. The remaining case acts on the interaction of two sequencing operators. In this case the contribution to the sum of $|N; M| + |N|$ is replaced by a contribution of $|N| + |M|$. Thus the measure is decreased by each reduction step and the reduction is terminating.

□

The β -reduction rule in **SPEA** notation is as follows, with $\rightarrow_\beta$ given by closing under all contexts and $\rightarrow_{\beta h}$ by closing under head contexts.

$$[N].\langle x \rangle.M \rightarrow_\beta \{N/x\}M$$

Projective reduction, given below, is as defined in Chapter 2, and *head reduction* ($\rightarrow_h$) is the union of all three head relations, $(\rightarrow_{\beta h}) \cup (\rightarrow_{\sigma h}) \cup (\rightarrow_\pi)$.

$$H\{\boxed{a}.M\} \rightarrow_\pi H\{\pi_0^a(M)\}^{\frac{1}{2}} + H\{\pi_1^a(M)\}^{\frac{1}{2}}$$

Let us consider the shape of head-normal forms.

Proposition 19 *The closed head-normal forms of the **SPEA** have one of the following three forms.*

$$H\{\star\} \quad H\{x\} \quad H\{x; M\}$$

Proof. Observe that all terms are of the form $H\{M\}$, since H can be $\{\}$. Head contexts themselves contain no redexes, thus it suffices to consider the possible cases for M . In one direction, the cases shown here are indeed head-normal, this is immediate from examining the reduction rules.

In the opposite direction we must rule out other potential normal forms. An application term can be dealt with by simply choosing a better head context: one

which includes the push. Abstraction may be absorbed into a head context which is free of applications, otherwise it will combine with the innermost application to form a redex, in which case the term is not in normal form. An event generator would obviously be subject to projection reduction, and thus not normal. If M is a sequential composition we must examine the form of the first subterm; excepting the cases of variable and choice there will be a sequential reduction which applies. The variable is one of the permitted normal forms above. In the case of a choice construct, occurring either in the hole of the context or as a subterm of a sequent, the term would not be closed. $\square$

4.1.1 Call-by-Value

The point of adding sequential composition to the calculus is to properly model call-by-value semantics. It is time to see how that works. Central to the *cbv/cbn* dichotomy is control flow: the lambda calculus lacks any control over execution. Sequential composition, however, is all about order of evaluation, thus provides the missing component.

The call-by-value probabilistic λ -calculus $\Lambda_{\oplus}^{\text{cbv}}$ we encode here is due to Faggian and Ronchi Della Rocca [26].

Definition 20 *Terms and values* of the call-by-value probabilistic λ -calculus are given by the following grammars:

$$M, N ::= x \mid \lambda x.M \mid MN \mid M \oplus N \quad V, W ::= x \mid \lambda x.M$$

With the following reduction rules:

$$\begin{aligned} C\{(\lambda x.M)V\} &\rightarrow_{\beta_v} C\{[V/x]M\} \\ S\{M \oplus N\} &\rightarrow_{\oplus} [\tfrac{1}{2}S\{M\}, \tfrac{1}{2}S\{N\}] \end{aligned}$$

Where C is a context given by:

$$C ::= \{ \} \mid MC \mid CM \mid \lambda x.C \mid C \oplus M \mid M \oplus C$$

While S is a *surface context*, given by:

$$S ::= \{ \} \mid M S \mid S M$$

The call-by-value behaviour of this calculus is created in two ways. The separation of value terms restricts the beta reduction, while the surface context restricts the projective reduction.

To encode this calculus in the $\text{SPE}\Lambda$ we combine the encoding of a traditional probabilistic sum in $\text{PE}\Lambda$ with a standard encoding of cbv λ -terms [20, 60, 34] in a sequential calculus.

The encoding of call-by-value behaviour almost directly uses sequential composition $N ; M$ to encode the call-by-value evaluation of an application. To evaluate a term $M N$ the call-by-value semantics evaluates N to a value, evaluates M to a value then applies the result of M as a function to the result of N . The sequential composition $N ; M$ says to evaluate N then evaluate M ; what remains is to compute the final return of values and function application.

Definition 21 The interpretation of a call-by-value probabilistic λ -term in the $\text{SPE}\Lambda$ is defined as follows.

Values V, W and terms M, N encode by the translations $-_v$ and $-_t$ respectively:

Values V, W : $x_v = x$ $(\lambda x.M)_v = \langle x \rangle . M_t$	Terms M, N : $V_t = [V_v]$ $(M N)_t = N_t ; M_t ; \langle x \rangle . x$ $(M \oplus N)_t = \boxed{a} . M_t a N_t$
---	---

The operational intuition of this embedding is that a *push* represents a *return value*: a term M_t undergoes evaluation until it reaches the form $V_t = [V_v]$, this value interpretation V_v is then pushed onto the stack. Since push does not occur in the interpretation of any other term, we can see that only values are pushed on the stack, therefore anything subsequently called from the stack is a value.

The β -reduction of a term is *simulated* by the reduction relations on $\text{SPE}\Lambda$:

$$\begin{aligned}
 ((\lambda x.M) V)_t &= [V_v]; [\langle x \rangle.M_t]; \langle y \rangle.y \\
 &\rightarrow_{\sigma} [V_v]. [\langle x \rangle.M_t]. \langle y \rangle.y \\
 &\rightarrow_{\beta} [V_v]. \langle x \rangle.M_t \\
 &\rightarrow_{\beta} \{V_v/x\}M_t \\
 &= (\{V/x\}M)_t
 \end{aligned}$$

Under the given encoding, the probabilistic reduction rule of $\Lambda_{\oplus}^{\text{cbv}}$ corresponds to projective reduction applied only in surface contexts. The translation of $S\{M \oplus N\}$ can never place the probabilistic redex inside a *push*; this is exactly the requirement for correct behaviour.

$$S\{M \oplus N\}_t \rightarrow \frac{1}{2}S\{M\}_t + \frac{1}{2}S\{N\}_t$$

4.2 Sequential Probabilistic Types

Types for the sequential λ -calculus are reflective of stack transformations:

$$A_n \dots A_1 \Rightarrow B_1 \dots B_m$$

The meaning of the above type is that when this term is given an input stack of terms typed by A_1 through A_n , it will terminate successfully and return a stack with types B_1 through B_m . The notion of successful termination here is due to the inclusion of a skip construct and is reflective of the machine semantics. It is this version of termination we use when talking about probabilistic termination.

To create our type system for the $\text{SPE}\Lambda$, we qualify the above type system using the *probability* of successful termination.

Definition 22 *Sequential probabilistic types* are given by the following grammars,

where $p \in \mathbb{P}$ is a probability.

$$\begin{aligned} A, B, C &::= \overline{A} \xrightarrow{p} \overline{C} & (\text{types}) \\ \overline{A} &::= A_1 \dots A_n & (\text{type vectors}) \end{aligned}$$

A *typing judgement* $E \mid \Gamma \vdash M : A$ assigns a term M the type A in the context of an event context E which assigns Boolean outcomes to free events of the term and a variable context Γ which assigns types to the free variables. The judgement $E \mid \Gamma \vdash S : \overline{A}$ assigns a stack of terms S a type vector $\overline{A}$. The typed $\text{SPE}\Lambda$ is given by the typing rules in Figure 4-1. We may omit p when $p = 1$.

Observe that because stacks are first-in-last-out, the identity term on two elements is $\langle x \rangle. \langle y \rangle. [y]. [x]$, which has the order of x and y reversed between popping and pushing. We match this reversal in the type system, and assign this term the type $AB \Rightarrow BA$. Since we want identity types to be of the form $\overline{A} \Rightarrow \overline{A}$, in a type $\overline{A} \xrightarrow{p} \overline{C}$ we consider the antecedent type vector $\overline{A}$ to be reversed, as shown below:

$$\overline{A} \Rightarrow \overline{A} = A_n \dots A_1 \Rightarrow A_1 \dots A_n$$

The probabilistic types for the $\text{PE}\Lambda$ embed into sequential types as follows; where ε denotes an empty stack:

$$\overline{A} \rightarrow p = \overline{A} \xrightarrow{p} \varepsilon$$

With this identification, the two type systems coincide on the fragment of $\text{SPE}\Lambda$ which corresponds to $\text{PE}\Lambda$.

Proposition 23 *For M a term of the $\text{PE}\Lambda$, $E \mid \Gamma \vdash M : \overline{A} \rightarrow p$ if and only if $E \mid \Gamma \vdash M : \overline{A} \xrightarrow{p} \varepsilon$.*

Proof. Follows immediately from the typing rules of the two systems. $\square$

We show that every type is inhabited by a closed term. Define for every type A the *zero term* 0_A as follows: for $A = B_1 \dots B_m \xrightarrow{p} C_1 \dots C_n$,

$$0_A = \langle x_1 \rangle \dots \langle x_m \rangle. [0_{C_1}] \dots [0_{C_n}] .$$

$$\begin{array}{c}
\overline{E \mid \Gamma \vdash \star : \overline{A} \stackrel{1}{\Rightarrow} \overline{A}} \\
\\
\frac{E \mid \Gamma \vdash M : \overline{A} \stackrel{p}{\Rightarrow} \overline{B} \quad E \mid \Gamma \vdash N : \overline{B} \stackrel{q}{\Rightarrow} \overline{C}}{E \mid \Gamma \vdash M ; N : \overline{A} \stackrel{pq}{\Rightarrow} \overline{C}} \\
\\
\overline{E \mid \Gamma, x : \overline{A} \stackrel{p}{\Rightarrow} \overline{C} \vdash x : \overline{A} \overline{B} \stackrel{p}{\Rightarrow} \overline{B} \overline{C}} \\
\\
\frac{E \mid \Gamma, x : A \vdash M : \overline{B} \stackrel{p}{\Rightarrow} \overline{C}}{E \mid \Gamma \vdash \langle x \rangle . M : A \overline{B} \stackrel{p}{\Rightarrow} \overline{C}} \\
\\
\frac{E \mid \Gamma \vdash N : A \quad E \mid \Gamma \vdash M : A \overline{B} \stackrel{p}{\Rightarrow} \overline{C}}{E \mid \Gamma \vdash [N] . M : \overline{B} \stackrel{p}{\Rightarrow} \overline{C}} \\
\\
\frac{E, a \mapsto i \mid \Gamma \vdash M_i : A}{E, a \mapsto i \mid \Gamma \vdash M_{\top} a M_{\perp} : A} \\
\\
\frac{E, a \mapsto \top \mid \Gamma \vdash M : \overline{A} \stackrel{p}{\Rightarrow} \overline{C} \quad E, a \mapsto \perp \mid \Gamma \vdash M : \overline{A} \stackrel{q}{\Rightarrow} \overline{C}}{E \mid \Gamma \vdash \boxed{a} . M : \overline{A} \stackrel{\frac{1}{2}p + \frac{1}{2}q}{\Rightarrow} \overline{C}} \\
\\
\overline{E \mid \Gamma \vdash M : \overline{A} \stackrel{0}{\Rightarrow} \overline{C}} \\
\\
\frac{E \mid \Gamma \vdash M : \overline{A} \stackrel{q}{\Rightarrow} \overline{B}}{E \mid \Gamma \vdash M : \overline{A} \stackrel{p}{\Rightarrow} \overline{B}}^{p < q} \\
\\
\frac{(E \mid \Gamma \vdash M_i : A_i)_{i \leq n}}{E \mid \Gamma \vdash \varepsilon M_1 \dots M_n : A_1 \dots A_n}
\end{array}$$

Figure 4-1: The typed sequential probabilistic event lambda-calculus

Proposition 24 (Type inhabitation) *Every type A is inhabited by its zero term, $\vdash 0_A : A$.*

Proof. Given that $0_{\overline{A} \Rightarrow \overline{C}}$ inhabits type $\overline{A} \Rightarrow \overline{C}$, the typing of $0_{\overline{A} \Rightarrow \overline{C}}$ can be derived as follows:

$$\frac{\vdash 0_{\overline{A} \Rightarrow \overline{C}} : \overline{A} \Rightarrow \overline{C}}{\vdash 0_{\overline{A} \Rightarrow \overline{C}} : \overline{A} \Rightarrow \overline{C}}^{p < 1}$$

We can thus focus on types of the form $A = \overline{A} \Rightarrow \overline{C}$, proceeding by strong induction on the symbolic length of A

Base case: $A = \varepsilon \Rightarrow \varepsilon$

$$\frac{}{\vdash \star : \varepsilon \Rightarrow \varepsilon}$$

Induction step: $A = \overline{A} \Rightarrow \overline{C}$

Let $\overline{A} = A_1 \dots A_n$ and $\overline{C} = C_1 \dots C_m$, by the induction hypothesis each of $C_1, \dots, C_m$ is inhabited by its respective zero term.

The following shows the type derivation of $0_{\overline{A} \Rightarrow \overline{C}}$:

$$\frac{\begin{array}{c} \{x_i : A_i\}_{i \leq n} \vdash 0_{C_m} : C_m \quad \{x_i : A_i\}_{i \leq n} \vdash \star : \overline{C} \Rightarrow \overline{C} \\ \vdots \\ \{x_i : A_i\}_{i \leq n} \vdash 0_{C_1} : C_1 \quad \{x_i : A_i\}_{i \leq n} \vdash [0_{C_2}] \dots [0_{C_m}] \cdot \star : C_1 \Rightarrow \overline{C} \end{array}}{\begin{array}{c} \{x_i : A_i\}_{i \leq n} \vdash [0_{C_1}] \dots [0_{C_m}] \cdot \star : \varepsilon \Rightarrow \overline{C} \\ \hline x_1 : A_1, \dots, x_{n-1} : A_{n-1} \vdash \langle x_n \rangle \cdot [0_{C_1}] \dots [0_{C_m}] \cdot \star : A_n \Rightarrow \overline{C} \\ \vdots \\ \hline x_1 : A_1 \vdash \langle x_2 \rangle \dots \langle x_n \rangle \cdot [0_{C_1}] \dots [0_{C_m}] \cdot \star : \overline{A} \Rightarrow \overline{C} \end{array}}$$

□

4.3 Big-Step Semantics

In this section we introduce a *big-step semantics* for the SPEA in the form of an evaluation relation which connects input stacks and terms with distributions of return stacks. In the previous section the type system framed terms as probabilistically parameterised stack transformations, it is this intuition which we expand on here. Applying a given term to an input stack may give multiple successful outcomes (runs), each with an associated probability.

The output stacks which result from successful evaluation can be gathered together into a (sub-)probability multi-distribution over return stacks.

$$\mathcal{T} = [p_i \cdot T_i]_{i \leq n} = [p_1 \cdot T_1, \dots, p_n \cdot T_n]$$

which contains each T_i with probability p_i , the sum probability or the *weight* $|\mathcal{T}| = \sum_{i \leq n} p_i$ should be at most one. The empty distribution is written $\emptyset$.

Throughout this chapter the empty stack is written ε and stack concatenation denoted by juxtaposition, ST .

We may view $\mathcal{T}$ as a function where $\mathcal{T}(T_i) = p_i$ for $i \leq n$ and $\mathcal{T}(T) = 0$ otherwise. This convention is useful when defining operations on distributions.

- The sum of two distributions $\mathcal{S} + \mathcal{T}$ is point-wise: $(\mathcal{S} + \mathcal{T})(T) = \mathcal{S}(T) + \mathcal{T}(T)$.
- A distribution multiplied with a probability is $p\mathcal{T}$ where $(p\mathcal{T})(T) = p(\mathcal{T}(T))$.
- The *expansion* of a distribution $S\mathcal{T}$ consists of prepending the stack S on each component: $S\mathcal{T}(ST) = \mathcal{T}(T)$. Note that $S\mathcal{T}(R) = 0$ if R is not of the form ST .

Definition 25 The *evaluation* relation $S, \mathbf{M} \Downarrow \mathcal{T}$ is defined inductively by the rules in Figure 4-2.

The stack machine intuition is conserved in the following lemma, which states that the relation is not affected by inaccessible registers. Note that the converse

$$\begin{array}{c}
\overline{S, \star \Downarrow [1 \cdot S]} \qquad \overline{S, M \Downarrow \emptyset} \\
\\
\frac{S \ N, M \Downarrow \mathcal{T}}{S, [N]. M \Downarrow \mathcal{T}} \qquad \frac{S, \{N/x\} M \Downarrow \mathcal{T}}{S \ N, \langle x \rangle. M \Downarrow \mathcal{T}} \\
\\
\frac{R, M \Downarrow [p_i \cdot S_i]_{i \leq n} \qquad (S_i, N \Downarrow \mathcal{T}_i)_{i \leq n}}{R, M ; N \Downarrow \sum_{i \leq n} p_i \mathcal{T}_i} \\
\\
\frac{S, \pi_{\top}^a(M) \Downarrow \mathcal{T}_{\top} \qquad S, \pi_{\perp}^a(M) \Downarrow \mathcal{T}_{\perp}}{S, \boxed{a}. M \Downarrow \frac{1}{2}(\mathcal{T}_{\top} + \mathcal{T}_{\perp})}
\end{array}$$

Figure 4-2: The probabilistic evaluation relation

does not hold since a term may not evaluate with insufficient arguments.

Lemma 26 *If $S, M \Downarrow \mathcal{T}$ then $RS, M \Downarrow R\mathcal{T}$ for any stack R .*

Proof. By induction on the definition of $\Downarrow$. □

What follows is the main theorem of this section; it links the big step semantics to the type system. Proving this theorem will provide the main substance for the machine termination theorem given in the next section.

Theorem 27 *For a typed, closed term $M : \overline{A} \xrightarrow{p} \overline{C}$ and stack $S : \overline{A}$ there is an evaluation*

$$S, M \Downarrow \mathcal{T}$$

such that $|\mathcal{T}| \geq p$

The method we use to prove the above theorem comes from abstract reducibility proofs [69]: we first define an abstract set which contains terms with the desired properties then prove that all terms we are considering fall within it. In our case we parameterise these sets by types; we define a set $\text{RUN}(\overline{A})$ that holds the terms with the probabilistic machine termination property for type $\overline{A}$ and show that all terms of type $\overline{A}$ belong to this set.

Definition 28 The set $\text{RUN}(-)$ is defined by mutual induction on types $\overline{A}$ and

type vectors $\overline{A}$ as a set of *closed terms*, respectively of closed stacks, as follows:

$$\text{RUN}(\overline{A} \stackrel{p}{\Rightarrow} \overline{B}) = \{ \textcolor{red}{M} \mid \forall S \in \text{RUN}(\overline{A}). \exists \mathcal{T} \in \mathcal{D}_p(\text{RUN}(\overline{B})). S, \textcolor{red}{M} \Downarrow \mathcal{T} \}$$

$$\text{RUN}(A_1 \dots A_n) = \{ \varepsilon \textcolor{red}{M}_1 \dots \textcolor{red}{M}_n \mid \textcolor{red}{M}_i \in \text{RUN}(A_i) \}$$

Where $\mathcal{D}_p(X)$ is the set of finite multi-subdistributions $\mathcal{X}$ over a set X which have weight $|\mathcal{X}| \geq p$.

Since the definition is inductive, the properties of such sets cannot be immediately deduced. What can be seen directly from the definitions is that *if* these sets were occupied by typed terms *then* the terms contained in them would satisfy our desired property. More formally, if all S of type $\overline{A}$ are in $\text{RUN}(\overline{A})$ then any $\textcolor{red}{M} : \overline{A} \stackrel{p}{\Rightarrow} \overline{B}$ in $\text{RUN}(\overline{A} \stackrel{p}{\Rightarrow} \overline{B})$ will have a witness to Theorem 27 by definition.

In Lemma 29 which follows, we will show that this holds: that any typed, closed term $\textcolor{red}{M} : \overline{A} \stackrel{p}{\Rightarrow} \overline{C}$ and stack $S : \overline{A}$ are in $\text{RUN}(\overline{A} \stackrel{p}{\Rightarrow} \overline{C})$ and $\text{RUN}(\overline{A})$ respectively. Theorem 27 above will then follow directly from the definition of $\text{RUN}(-)$, which implies the existence of the required evaluation $S, \textcolor{red}{M} \Downarrow \mathcal{T}$ with $|\mathcal{T}| = p$.

The following notation will be useful in the proof of Lemma 29.

- We write σ for a substitution map $\{ \textcolor{red}{M}_i / x_i \}_{i \leq n}$, where $\sigma \textcolor{red}{M}$ is the application of σ to $\textcolor{red}{M}$.
- For an event valuation $E = (a_k \mapsto i_k)_{k \leq n}$ we write $\pi_E(\textcolor{red}{M})$ for the projection on each a_i in E .

$$\pi_E(\textcolor{red}{M}) = \pi_{i_1}^{a_1}(\dots \pi_{i_n}^{a_n}(\textcolor{red}{M}) \dots)$$

The set $\text{RUN}(-)$ extends to contexts Γ as follows. Note that the definition implies that σ is closed (so by definition each substituting term is closed).

$$\text{RUN}(x_1 : A_1, \dots, x_n : A_n) = \{ \sigma \mid \sigma x_i \in \text{RUN}(A_i), 1 \leq i \leq n \}$$

The following reducibility lemma thus forms the main content of our proof.

Lemma 29 *If $E \mid \Gamma \vdash M : A$ and $\sigma \in \text{RUN}(\Gamma)$ then $\pi_E(\sigma M) \in \text{RUN}(A)$.*

Proof. Proceed by induction on the typing derivation for M . Note that since σ is closed throughout, $\pi_E(\sigma M) = \sigma(\pi_E(M))$.

Unit case:

$$\frac{}{E \mid \Gamma \vdash \star : \overline{A} \stackrel{1}{\Rightarrow} \overline{A}}$$

Note that $\pi_E(\sigma \star) = \star$.

For arbitrary $S \in \text{RUN}(\overline{A})$ the unit rule for evaluation gives $S, \star \Downarrow [1 \cdot S]$, so that the outcome is a distribution of weight one, and hence $\star \in \text{RUN}(\overline{A} \Rightarrow \overline{A})$.

Sequencing case:

$$\frac{E \mid \Gamma \vdash N : \overline{A} \stackrel{p}{\Rightarrow} \overline{B} \quad E \mid \Gamma \vdash M : \overline{B} \stackrel{q}{\Rightarrow} \overline{C}}{E \mid \Gamma \vdash N ; M : \overline{A} \stackrel{pq}{\Rightarrow} \overline{C}}$$

Let $N' = \pi_E(\sigma N)$ and $M' = \pi_E(\sigma M)$. By the induction hypothesis $N' \in \text{RUN}(\overline{A} \stackrel{p}{\Rightarrow} \overline{B})$ and $M' \in \text{RUN}(\overline{B} \stackrel{q}{\Rightarrow} \overline{C})$.

Thus, given $R \in \text{RUN}(\overline{A})$ we have $R, N' \Downarrow [p_i \cdot S_i]_{i \leq n}$ with each $S_i \in \text{RUN}(\overline{B})$ and $\sum_{i \leq n} p_i \geq p$. Again by the definition of $\text{RUN}(-)$, for each $i \leq n$ we have $S_i, M' \Downarrow \mathcal{T}_i$ with $\mathcal{T}_i \in \mathcal{D}_q(\text{RUN}(\overline{C}))$.

The sequential composition rule for $\Downarrow$ gives $R, (N' ; M') \Downarrow \mathcal{T}$ for $\mathcal{T} = \sum_{i \leq n} p_i \mathcal{T}_i$. Observe that $|\mathcal{T}| = \sum_{i \leq n} p_i |\mathcal{T}_i| \geq pq$, since each $\mathcal{T}_i$ is a distribution over $\text{RUN}(\overline{C})$ it follows that $\mathcal{T} \in \mathcal{D}_{pq}(\text{RUN}(\overline{C}))$. Thus $\pi_E(\sigma(N ; M)) = N' ; M' \in \text{RUN}(\overline{A} \stackrel{pq}{\Rightarrow} \overline{C})$.

Variable case:

$$\frac{}{E \mid \Gamma, x : \overline{A} \stackrel{p}{\Rightarrow} \overline{C} \vdash x : \overline{A} \overline{B} \stackrel{p}{\Rightarrow} \overline{B} \overline{C}}$$

Let $RS \in \text{RUN}(\overline{B} \overline{A})$ where $R \in \text{RUN}(\overline{B})$ and $S \in \text{RUN}(\overline{A})$. The assumptions give us that $\sigma \in \text{RUN}(\Gamma, x : \overline{A} \stackrel{p}{\Rightarrow} \overline{C})$ from which it immediately follows that $\sigma x \in \text{RUN}(\overline{A} \stackrel{p}{\Rightarrow} \overline{C})$. By the definition of $\text{RUN}(-)$ this is a closed term, so $\pi_E(\sigma x) = \sigma x$.

We need to show that this term is in $\text{RUN}(\overline{A\overline{B}} \stackrel{p}{\Rightarrow} \overline{B\overline{C}})$. By definition of $\text{RUN}(\overline{A\overline{B}} \stackrel{p}{\Rightarrow} \overline{B\overline{C}})$ we have $\mathcal{T} \in \mathcal{D}_p(\text{RUN}(\overline{B\overline{C}}))$ such that $S, \sigma x \Downarrow \mathcal{T}$. By Lemma 26 we have $RS, \sigma x \Downarrow R\mathcal{T}$. Since $R \in \text{RUN}(\overline{B})$, we have $R\mathcal{T} \in \mathcal{D}_p(\text{RUN}(\overline{B\overline{C}}))$, and conclude that $\sigma x \in \text{RUN}(\overline{A\overline{B}} \stackrel{p}{\Rightarrow} \overline{B\overline{C}})$.

Abstraction case:

$$\frac{E \mid \Gamma, x : A \vdash M : \overline{B} \stackrel{p}{\Rightarrow} \overline{C}}{E \mid \Gamma \vdash \langle x \rangle. M : A \overline{B} \stackrel{p}{\Rightarrow} \overline{C}}$$

Let $\sigma \in \text{RUN}(\Gamma)$ and $S N \in \text{RUN}(\overline{B} A)$. We need to demonstrate a distribution $\mathcal{T} \in \mathcal{D}_p(\text{RUN}(\overline{B\overline{C}}))$ such that $S N, \pi_E(\sigma(\langle x \rangle. M)) \Downarrow \mathcal{T}$. Let $M' = \pi_E(\sigma M)$; then since σ is not defined on x and σ and N are closed, we have $\pi_E(\sigma\{N/x\}M) = \{N/x\}M'$ and $\pi_E(\sigma(\langle x \rangle. M)) = \langle x \rangle. M'$.

Observing that $\sigma\{N/x\} \in \text{RUN}(\Gamma, x : A)$ we may apply the inductive hypothesis to the premise $E \mid \Gamma, x : A \vdash M : \overline{B} \stackrel{p}{\Rightarrow} \overline{C}$ to obtain $\mathcal{T} \in \mathcal{D}_p(\text{RUN}(\overline{B\overline{C}}))$ with evaluation $S, \{N/x\}M' \Downarrow \mathcal{T}$. Then by the definition of the evaluation relation we may deduce that $S N, M' \Downarrow \mathcal{T}$, hence $\langle x \rangle. M' \in \text{RUN}(A \overline{B} \stackrel{p}{\Rightarrow} \overline{C})$.

Application case:

$$\frac{E \mid \Gamma \vdash N : A \quad E \mid \Gamma \vdash M : A \overline{B} \stackrel{p}{\Rightarrow} \overline{C}}{E \mid \Gamma \vdash [N]. M : \overline{B} \stackrel{p}{\Rightarrow} \overline{C}}$$

Fix $S \in \text{RUN}(\overline{B})$ and $\sigma \in \text{RUN}(\Gamma)$, and let $N' = \pi_E(\sigma N)$ and $M' = \pi_E(\sigma M)$, so that $\pi_E(\sigma([N]. M)) = [N']. M'$. The inductive hypothesis gives $N' \in \text{RUN}(A)$ and $M' \in \text{RUN}(A \overline{B} \stackrel{p}{\Rightarrow} \overline{C})$. Then by the definition of $\text{RUN}(-)$ we have $S N', M' \Downarrow \mathcal{T}$ for some $\mathcal{T} \in \mathcal{D}_p(\text{RUN}(\overline{B\overline{C}}))$, and by the definition of $\Downarrow$ we have $S, [N']. M' \Downarrow \mathcal{T}$. It follows that $[N']. M' \in \text{RUN}(\overline{B} \stackrel{p}{\Rightarrow} \overline{C})$.

Choice case:

$$\frac{E, a \mapsto i \mid \Gamma \vdash M_i : A}{E, a \mapsto i \mid \Gamma \vdash M_{\top} a M_{\perp} : A}$$

Observe that $\pi_{a \mapsto i}(M_{\top} a M_{\perp}) = \pi_{a \mapsto i}(M_i)$. By the induction hypothesis $\pi_{E, a \mapsto i}(\sigma M_i) \in \text{RUN}(A)$. It follows by equality that $\pi_{E, a \mapsto i}(\sigma(M_{\top} a M_{\perp})) \in \text{RUN}(A)$.

Generator case:

$$\frac{E, a \mapsto \top \mid \Gamma \vdash M : \overline{A} \xrightarrow{p} \overline{C} \quad E, a \mapsto \perp \mid \Gamma \vdash M : \overline{A} \xrightarrow{q} \overline{C}}{E \mid \Gamma \vdash \boxed{a}. M : \overline{A} \xrightarrow{\frac{1}{2}p + \frac{1}{2}q} \overline{C}}$$

Let $\sigma \in \text{RUN}(\Gamma)$ and $S \in \text{RUN}(\overline{A})$. Let $M_i = \pi_{E, a \mapsto i}(\sigma M)$ for $i \in \top, \perp$. By the inductive hypothesis, $M_\top \in \text{RUN}(\overline{A} \xrightarrow{p} \overline{C})$ and $M_\perp \in \text{RUN}(\overline{A} \xrightarrow{q} \overline{C})$, which gives $S, M_\top \Downarrow \mathcal{T}_\top$ and $S, M_\perp \Downarrow \mathcal{T}_\perp$ for some $\mathcal{T}_\top \in \mathcal{D}_p(\text{RUN}(\overline{C}))$ and $\mathcal{T}_\perp \in \mathcal{D}_q(\text{RUN}(\overline{C}))$.

From the definition of $\Downarrow$ it follows that $S, \boxed{a}. \pi_E(\sigma M) \Downarrow \frac{1}{2}(\mathcal{T}_\top + \mathcal{T}_\perp)$. Note that $|\frac{1}{2}(\mathcal{T}_\top + \mathcal{T}_\perp)| \geq \frac{1}{2}(p + q)$ therefore $\frac{1}{2}(\mathcal{T}_\top + \mathcal{T}_\perp) \in \mathcal{D}_{\frac{1}{2}(p+q)}(\text{RUN}(\overline{C}))$.

As a is not evaluated in E , we have $\boxed{a}. \pi_E(\sigma M) = \pi_E(\sigma(\boxed{a}. M))$. We conclude that $\pi_E(\sigma(\boxed{a}. M)) \in \text{RUN}(\overline{A} \xrightarrow{r} \overline{C})$.

Zero case:

$$\overline{E \mid \Gamma \vdash M : \overline{A} \xrightarrow{0} \overline{C}}$$

By the definition of $\Downarrow$ we have $S, \pi_E(\sigma M) \Downarrow \emptyset$ for any stack S , while $|\emptyset| = 0$ and $\emptyset \in \mathcal{D}_0(\text{RUN}(\overline{C}))$, so that $\pi_E(\sigma M) \in \text{RUN}(\overline{A} \xrightarrow{0} \overline{C})$.

Low case:

$$\frac{E \mid \Gamma \vdash M : \overline{A} \xrightarrow{q} \overline{B}}{E \mid \Gamma \vdash M : \overline{A} \xrightarrow{p} \overline{B}}^{p < q}$$

We assume $M \in \text{RUN}(\overline{A} \xrightarrow{q} \overline{B})$, for any witnessing evaluation $S, M \Downarrow \mathcal{T}$ since $\mathcal{T} \in \mathcal{D}_q(\text{RUN}(\overline{B}))$ by definition $|\mathcal{T}| \geq q > p$ therefore $\mathcal{T} \in \mathcal{D}_p(\text{RUN}(\overline{B}))$.

□

The main theorem of this section then follows directly:

Proof of Theorem 27. Given a closed term $M : \overline{A} \xrightarrow{p} \overline{C}$ and closed stack $S : \overline{A}$ Lemma 29 implies that $M \in \text{RUN}(\overline{A} \xrightarrow{p} \overline{C})$ and $S \in \text{RUN}(\overline{A})$. The definition of $\text{RUN}(\)$ then tells us that there is some $\mathcal{T} \in \mathcal{D}_p$ such that $S, M \Downarrow \mathcal{T}$. By definition $|\mathcal{T}| \geq p$. □

4.4 An Abstract Machine

We have referred to machine operations when discussing the intuition behind the sequential extension of the calculus. In this section we explore the small-step operational semantics of the **SPEA** as given by an abstract machine similar to ones discussed in the introduction and at the start of this chapter.

A *state* of this machine is a triple (S, M, K) , where M is a term and S and K are stacks of terms. The term M can be thought of as a sequence of instructions which interact with the two stacks. For this reason the *state/operand stack*, S has its head to the right: $S N$, while the *continuation stack* K has the head to the left: $N K$.

$$(S P, M, N K)$$

For both stacks the empty stack is written ε and stack concatenation denoted by juxtaposition, ST .

Transitions between states of the machine will be probabilistic. The transition rule written below indicates that from the state (S, M, K) the machine transitions with probability p to the state (T, N, L) .

$$^p \frac{(S, M, K)}{(T, N, L)}$$

When $p = 1$ it may be omitted.

The formal definition of the machine, with transitions follows.

Definition 30 The *sequential probabilistic event machine* (SPEM) has *states* (S, M, K) and the following probabilistic *transitions*.

$$\begin{array}{ccc} \frac{(S, [N].M, K)}{(S N, M, K)} & \frac{(S, N; M, K)}{(S, N, M K)} & \frac{1}{2} \frac{(S, \boxed{a}.M, K)}{(S, \pi_1^a(M), K)} \\[10pt] \frac{(S N, \langle x \rangle.M, K)}{(S, \{N/x\}M, K)} & \frac{(S, \star, M K)}{(S, M, K)} & \frac{1}{2} \frac{(S, \boxed{a}.M, K)}{(S, \pi_1^a(M), K)} \end{array}$$

Each of these transitions matches the intuition of the operator involved. *Pop* $\langle x \rangle$. and *push* $[N]$. pop and push states from the stack, exactly as they do in a Krivine Machine. Sequential rule *sequence* ; pushes the next term to be evaluated onto the continuation stack, while *skip* $\star$ indicates that the current term has been completed and the next term should be fetched for evaluation. These are standard for sequential calculi [54, 34]. The novel transitions are those for the *generator* construct. Whereas the deterministic steps all occurred with probability one there are two possible transitions from sampling a generator each occurring with probability a half, the distribution of outcomes matches projective head reduction as defined earlier in the chapter.

A *run* of the machine is a sequence of steps, written as below, where p is the product of the probabilities of the constituent steps.

$$^p \frac{(S, M, K)}{(T, N, L)}$$

Note that a run of length zero has a default null product value of one.

A *successful run* is a run of the machine which terminates with *skip* as the term and an empty continuation stack:

$$^p \frac{(S, M, K)}{(T, \star, \varepsilon)}$$

The remaining stack T holds the *return values* of the computation.

This machine links back to the big step semantics through the following proposition:

Proposition 31 *If $S, M \Downarrow \mathcal{T}$ then there is a finite collection of n distinct runs*

$$_{p_i} \frac{(S, M, \varepsilon)}{(T_i, \star, \varepsilon)} \quad (i \leq n) \quad \text{such that} \quad \mathcal{T} = [p_i \cdot T_i]_{i \leq n} .$$

Proof. By induction on $S, M \Downarrow \mathcal{T}$.

Skip term The zero step run $\frac{(S, \star, \varepsilon)}{(S, \star, \varepsilon)}$ satisfies the proposition

Null distribution The empty collection of runs gives the empty distribution.

Push and Pop

$$\frac{S N, M \Downarrow \mathcal{T}}{S, [N]. M \Downarrow \mathcal{T}} \quad \frac{S, \{N/x\} M \Downarrow \mathcal{T}}{S N, \langle x \rangle. M \Downarrow \mathcal{T}}$$

For each of these cases we need only add one deterministic rule to start of the runs given by the induction step, there is no change to run weight or outcome:

$$\frac{\frac{(S, [N]. M, \varepsilon)}{(S N, M, \varepsilon)}}{p_i \frac{(T_i, \star, \varepsilon)}} \quad \frac{\frac{(S N, \langle x \rangle. M, \varepsilon)}{(S, \{N/x\} M, \varepsilon)}}{p_i \frac{(T_i, \star, \varepsilon)}}$$

Sequence

$$\frac{R, M \Downarrow [p_i \cdot S_i]_{i \leq n} \quad (S_i, N \Downarrow \mathcal{T}_i)_{i \leq n}}{R, M ; N \Downarrow \sum_{i \leq n} p_i \mathcal{T}_i}$$

By the induction hypothesis we have the following runs:

$$\frac{(R, M, \varepsilon)}{(S_i, \star, \varepsilon)} \quad q_{ij} \frac{(S_i, N, \varepsilon)}{(T_{ij}, \star, \varepsilon)}$$

Since the run above left is successful M does not access the continuation stack below that level, thus an extra term at the bottom will not be accessed during evaluation. So we may combine the runs as follows:

$$\frac{\frac{(R, M ; N, \varepsilon)}{(R, M, N)}}{p_i \frac{(S_i, \star, N)}}{q_{ij} \frac{(S_i, N, \varepsilon)}{(T_{ij}, \star, \varepsilon)}}$$

Note that $[p_i q_{ij} \cdot T_{ij}]_{i \leq n, j \leq m} = \sum_{i \leq n} p_i \mathcal{T}_i$ so the collection of such runs satisfies the proposition.

Choice

$$\frac{S, \pi_{\top}^a(M) \Downarrow \mathcal{T}_{\top} \quad S, \pi_{\perp}^a(M) \Downarrow \mathcal{T}_{\perp}}{S, \boxed{a}. M \Downarrow \frac{1}{2}(\mathcal{T}_{\top} + \mathcal{T}_{\perp})}$$

Consider the collections formed by adding the generator rule on the start of runs given by the induction hypothesis:

$$\frac{\frac{1}{2} \frac{(S, \boxed{a}. M, \varepsilon)}{(S, \pi_{\top}^a(M), \varepsilon)}}{p_i \frac{(T_{\top i}, \star, \varepsilon)}} \quad \frac{\frac{1}{2} \frac{(S, \boxed{a}. M, \varepsilon)}{(S, \pi_{\perp}^a(M), \varepsilon)}}{q_j \frac{(T_{\perp j}, \star, \varepsilon)}}$$

We know that $\mathcal{T}_{\top} = [p_i \cdot T_{\top i}]_{i \leq n}$ and $\mathcal{T}_{\perp} = [q_j \cdot T_{\perp j}]_{j \leq m}$ thus the distribution $[1/2 \cdot p_i \cdot T_{\top i}]_{i \leq n} + [1/2 \cdot q_j \cdot T_{\perp j}]_{j \leq m}$ given by combining these collections is equal to $\frac{1}{2}(\mathcal{T}_{\top} + \mathcal{T}_{\perp})$. $\square$

The converse of the above proposition holds by a similar induction performed on machine runs.

4.5 Probabilistic Machine Termination

So far in this chapter we have defined a sequential extension of PEA, provided it with a type system and an abstract machine, with both big and small step semantics. In this section we will draw these elements together, showing how the type system relates to machine termination. To this end, the main theorem of this section and in fact our main theorem about the typed SPEA will be that the type system follows its intuition: a type $\overline{A} \xrightarrow{p} \overline{B}$ guarantees machine termination with probability at least p when given an input stack of type $\overline{A}$ and these successful runs each return a stack of type $\overline{B}$.

Figure 4-3 provides an overall schema for the proof strategy used in this chapter, culminating in the following machine termination result.

Theorem 32 For a typed, closed term $M : \overline{A} \xrightarrow{p} \overline{C}$ and stack $S : \overline{A}$ there is a finite set of distinct successful runs:

$$p_i \frac{(S, M, \varepsilon)}{(T_i, \star, \varepsilon)} \quad (i \leq n)$$

with sum probability $\sum_{i \leq n} p_i \geq p$.

Note that the bound given here may not be tight, since there may be other terminating runs not considered in the statement of the theorem.

Proof. Given a closed term $M : \overline{A} \xrightarrow{p} \overline{C}$ and $S : \overline{A}$ by Theorem 27 there is a distribution $\mathcal{T}$ such that $S, M \Downarrow \mathcal{T}$ and $|\mathcal{T}| \geq p$. Proposition 31 then states that there is a collection of distinct runs $p_i \frac{(S, M, \varepsilon)}{(T_i, \star, \varepsilon)}$ for $(i \leq n)$ with $\sum_{i \leq n} p_i \geq p$. $\square$

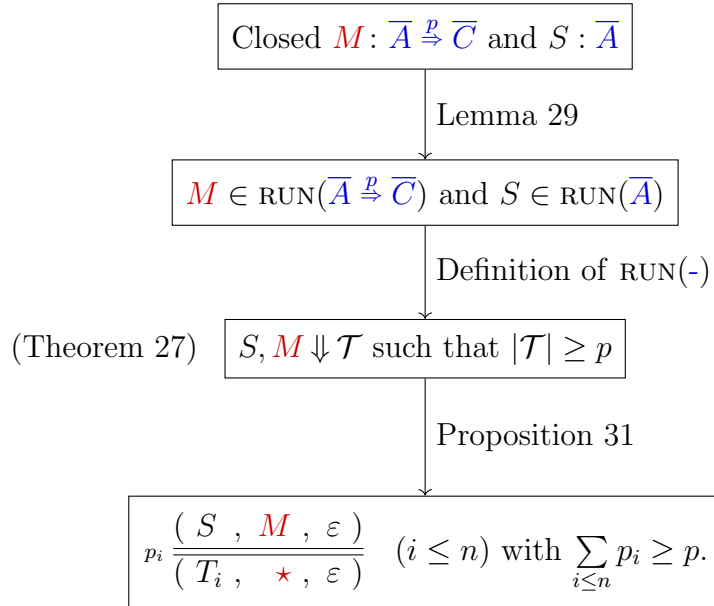


Figure 4-3: Overview of the proof of Theorem 32

Chapter 5

Termination

In the previous chapter we saw how the type system for $\text{SPE}\Lambda$ relates to both the type system for $\text{PE}\Lambda$ and termination of the abstract machine. The final step in proving head normalisation for $\text{PE}\Lambda$, the main theorem of Part I, is to relate machine termination to head reduction.

The original Krivine machine models head reduction directly via evaluation as follows. The head context $\lambda x_1 \dots \lambda x_n. \{ \} M_1 \dots M_m$ is stripped away through stack manipulation: the abstractions consume the top n elements off the stack; then the applications push the terms M_i onto it. The next two steps of evaluation correspond to reducing a beta redex. The order of evaluation of redexes is thus the same as in head reduction. This process terminates with a *variable* term because the machine has no steps to execute on a variable; this aligns with normalisation since head normal form is a variable in a head context.

The definition of successful termination in a sequential calculus, such as the $\text{SPE}\Lambda$ defined above, differs; although the link to head reduction remains. A machine has terminated successfully when it has no more terms to evaluate; in the sequential calculus this means reaching a *skip* term with an empty continuation stack. Although the machine will stop on a variable, since there are no further possible machine steps, this termination cannot be considered successful in the sense guaranteed by the type system, since evaluation is not complete. For this

reason our proof of normalisation must differ from that used on the Krivine machine.

An overview of our proof strategy is shown in Figure 5-1. The type embedding and evaluation relation were covered in the previous chapter. In this chapter we modify the evaluation relation by introducing a weight. Observing the effect of reduction on these weights allows us to partition the outcomes of reduction into head normal forms and subsequently show that the proportion that normalise is bounded below by the type system.

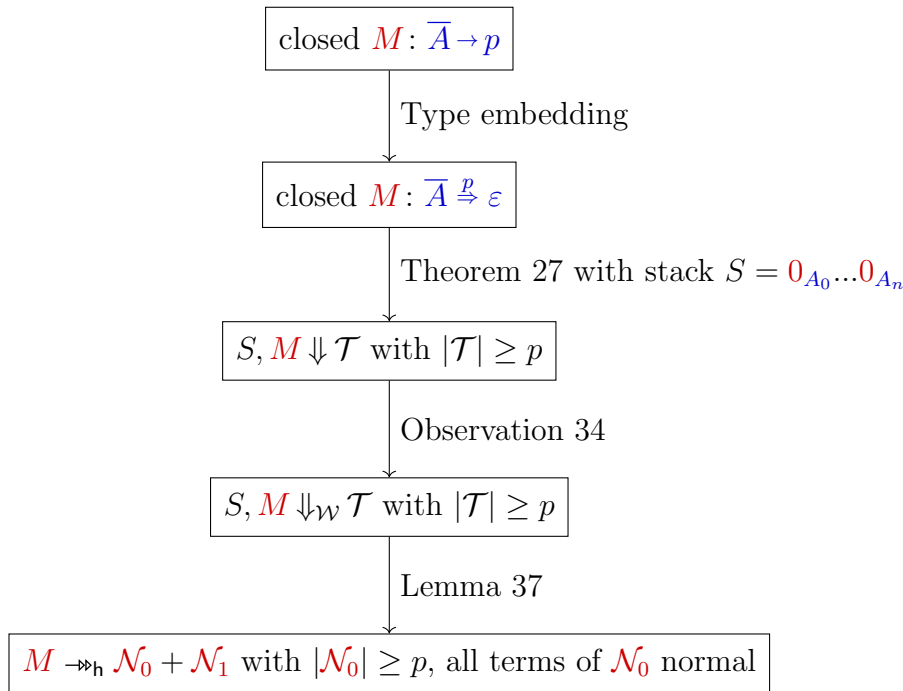


Figure 5-1: Structure for proof of Theorem 12

Head normalisation of PEA is proved by placing a measure on machine runs. We do this by defining a weighted version of the evaluation relation, which is related to the length of machine runs, and show that it is decreased by head reduction. Rather than measuring machine runs directly we work with the big-step semantics of the evaluation relation so that we may more easily reason about probability distributions over machine runs, which appear explicitly in the big-step semantics. This measure in particular counts abstractions, applications, and generators with the idea that reduction will shorten the runs of the machine by

removing exactly these operations. Projective reduction removes the generator step from each machine run; beta reduction, meanwhile, removes consecutive *push* and *pop* operations, thus removing two steps from the machine run as shown below:

$$\frac{\frac{(S, [N].\langle x \rangle.M, T)}{(S, N, \langle x \rangle.M, T)}}{(S, \{N/x\}M, T)} \rightarrow_{\beta} (S, \{N/x\}M, T)$$

For a non-probabilistic term the reasoning is straightforward since we are considering a single run which each head beta-reduction shortens by two steps. The following example demonstrates that the initial stage of the run removes the head context, followed by two steps for the redex, with the remainder of the run working on the reductand:

$$\frac{\frac{\frac{(M, \langle y \rangle.[y].[\langle z \rangle.[z]].\langle x \rangle.x, \varepsilon)}{(\varepsilon, [M].[\langle z \rangle.[z]].\langle x \rangle.x, \varepsilon)}}{(M, [\langle z \rangle.[z]].\langle x \rangle.x, \varepsilon)}}{\frac{(M, \langle z \rangle.[z], \langle x \rangle.x, \varepsilon)}{(M, \langle z \rangle.[z], \varepsilon)}}{(\varepsilon, [M], \varepsilon)}} \rightarrow_{\beta} \frac{\frac{(M, \langle y \rangle.[y].\langle z \rangle.[z], \varepsilon)}{(\varepsilon, [M].\langle z \rangle.[z], \varepsilon)}}{(\varepsilon, \langle z \rangle.[z], \varepsilon)}}{(M, \langle z \rangle.[z], \varepsilon)} \rightarrow_{\beta} \frac{(\varepsilon, [M], \varepsilon)}{(M, \star, \varepsilon)}$$

A probabilistic term, however, experiences branching: multiple alternate runs dependant on the outcome of the generator. We count the size of the evaluation tree in terms of the pushes and pops as before. The shortening of a run by beta reduction, described above, becomes a shortening of branches in the tree, which reduces the tree size, as illustrated below. Note that since head contexts do not have any probabilistic aspects reduction will occur prior to any branching, meaning that all runs in that tree are shortened.

[illegible]

Projective reduction separates the branches of the tree, producing a distribution of runs. It is due to the duplication inherent in this process that we measure the distribution of runs using a multiset.

$$\frac{1}{2} \frac{\frac{(\textcolor{red}{M}, \langle z \rangle. [z], \langle x \rangle. \textcolor{red}{a}. x a y, \varepsilon)}{(\textcolor{red}{M}, \textcolor{red}{a}. (\langle z \rangle. [z]) a y, \varepsilon)}}{(\textcolor{red}{M}, \langle z \rangle. [z], \varepsilon)} \quad (\textcolor{red}{M}, y, \varepsilon)$$

$$\frac{(\varepsilon, [M], \varepsilon)}{(\textcolor{red}{M}, \star, \varepsilon)}$$

$$\downarrow_{\pi}$$

$$\left[\frac{(\textcolor{red}{M}, \langle z \rangle. [z], \langle x \rangle. x, \varepsilon)}{(\textcolor{red}{M}, \langle z \rangle. [z], \varepsilon)} \right. \quad , \quad \left. \frac{(\textcolor{red}{M}, \langle z \rangle. [z], \langle x \rangle. y, \varepsilon)}{(\textcolor{red}{M}, y, \varepsilon)} \right]$$

$$\frac{(\varepsilon, [M], \varepsilon)}{(\textcolor{red}{M}, \star, \varepsilon)}$$

5.1 Weighted Evaluation

In this section we define a *weighted* version of the evaluation relation $\Downarrow_w$ indexed with a natural number weight w . Modifying the relation in this way is similar to previous works which use cost carrying semantics [8, 40, 64]. The purpose of w is to count the length of the runs of the abstract machine represented by $\Downarrow_w$, with the idea that head reduction shortens these runs. Given an evaluation $S, \mathcal{M} \Downarrow_w \mathcal{T}$ for a distribution of terms¹, we may reduce a term in $\mathcal{M}$ and observe that w decreases.

¹Here, as in the previous chapter, we use script letters to indicate multi-sub-distributions of terms and stacks.

$$\begin{array}{c}
\overline{S, \star \Downarrow_0 [1 \cdot S]} \qquad \overline{S, \mathcal{M} \Downarrow_0 \emptyset} \\
\\
\frac{S, \mathcal{N}, \mathcal{M} \Downarrow_w \mathcal{T}}{S, [\mathcal{N}]. \mathcal{M} \Downarrow_{w+1} \mathcal{T}} \qquad \frac{S, \{N/x\} \mathcal{M} \Downarrow_w \mathcal{T}}{S, \langle x \rangle. \mathcal{M} \Downarrow_{w+1} \mathcal{T}} \\
\\
\frac{R, \mathcal{M} \Downarrow_w [p_i \cdot S_i]_{i \leq n} \quad (S_i, \mathcal{N} \Downarrow_{v_i} \mathcal{T}_i)_{i \leq n}}{R, \mathcal{M}; \mathcal{N} \Downarrow_{(w + \sum_{i \leq n} v_i)} \sum_{i \leq n} p_i \mathcal{T}_i} \\
\\
\frac{S, \pi_{\top}^a(\mathcal{M}) \Downarrow_v \mathcal{T}_{\top} \quad S, \pi_{\perp}^a(\mathcal{M}) \Downarrow_w \mathcal{T}_{\perp}}{S, \boxed{a}. \mathcal{M} \Downarrow_{v+w+1} \frac{1}{2} \mathcal{T}_{\top} + \frac{1}{2} \mathcal{T}_{\perp}}
\end{array}$$

Figure 5-2: The weighted probabilistic evaluation relation

We start with reduction of terms, then extend the result to distributions as follows. Given an evaluation of a distribution $S, \mathcal{M} \Downarrow \mathcal{T}$ we can assign a weight $\Downarrow_v$ to the evaluation of each term in $\mathcal{M}$ we then define w as the multi set of such weights, producing $S, \mathcal{M} \Downarrow_w \mathcal{T}$. If v is non-zero, a reduction step on a redex of that term which reduces v will also reduce w . Including this extension is necessary because projective reduction will produce distributions; so in a reduction sequence there will be intermediary distributions on which the result must also hold.

Definition 33 The *weighted* evaluation relation $S, \mathcal{M} \Downarrow_w \mathcal{T}$ is given in Figure 5-2, where the *weight* w is a natural number. We extend it to a distribution of terms $\mathcal{M}$ by the following rule, carrying a multiset of weights $\mathcal{W}$.

$$\frac{(S, \mathcal{M}_i \Downarrow_{w_i} \mathcal{T}_i)_{i \leq n}}{S, [p_i \cdot \mathcal{M}_i]_{i \leq n} \Downarrow_{[w_i]_{i \leq n}} \sum_{i \leq n} p_i \mathcal{T}_i}$$

Observation 34 For any evaluation relation $S, \mathcal{M} \Downarrow \mathcal{T}$ there is a corresponding weighted evaluation relation $S, \mathcal{M} \Downarrow_w \mathcal{T}$.

The next lemma is central to this chapter: it establishes that the weight in the evaluation relation decreases for head beta-reduction and projective reduction steps, and is stable under head sequent-reduction. This will be sufficient to prove head normalisation for $\text{PE}\Lambda$ terms, which do not have sequential steps. For $\text{SPE}\Lambda$ we need a secondary measure or other termination proof in order to establish the

termination of sequential reduction.

Terms evaluated by the zero-rule, as $S, M \Downarrow_0 \emptyset$ are potentially non-terminating and, as such, do not contribute to the bound for probability of termination/normalisation. The weighted measure does not count such probability-zero terms because it cannot assign a length to a non-terminating term. Hence reduction in zero-weighted terms does not reduce the measure but notably also does not increase it. For this reason these terms are excluded from the statement of the lemma. The most important thing for our purposes is that where the evaluation does return a non-empty distribution, head-reduction progresses the term towards a head-normal form and reduces the measure.

The non-empty distribution also has an important role in the proof. The steps to derive an evaluation relation are almost unique, however, the zero rule can be applied to any term which results in many more possible derivations. Disallowing empty distributions excludes the use of this rule in most cases.

Lemma 35 *Let $S, M \Downarrow_w \mathcal{T}$ where $\mathcal{T} \neq \emptyset$.*

- *If $M \rightarrow_{\beta h} N$ then $S, N \Downarrow_v \mathcal{T}$ with $v < w$.*
- *If $M \rightarrow_{\pi} \mathcal{N}$ then $S, \mathcal{N} \Downarrow_{\mathcal{W}} \mathcal{T}$ with $v < w$ for every v in $\mathcal{W}$.*
- *If $M \rightarrow_{\sigma h} N$ then $S, N \Downarrow_w \mathcal{T}$.*

Proof. We approach all three parts of the lemma concurrently, first considering the reduction steps themselves, then showing that head contexts have a monotone effect on weight; thereby demonstrating that the statements are preserved by their closure under head contexts. The proof for each case proceeds by induction on the definition of the weighted evaluation $\Downarrow_w$. A summary of the numerical results from the first part of the proof is shown in Table 5.1.

Beta $[N]. \langle x \rangle. M \rightarrow_{\beta} \{N/x\}M$ Since $\mathcal{T}$ is non-empty none of the inferences may be replaced by a zero-rule, so the evaluation derivation for the redex must be as below left. The case is then as follows:

$$\begin{array}{ccc}
w + 2 & \rightarrow_{\beta} & w \\
v + w + 1 & \rightarrow_{\pi} & [v, w] \\
w & \rightarrow_{\sigma} & w
\end{array}$$

Table 5.1: Effect of base reductions on weight of evaluation relation

$$\frac{\frac{S, \{N/x\}M \Downarrow_w \mathcal{T}}{S N, \langle x \rangle. M \Downarrow_{w+1} \mathcal{T}}}{S, [N]. \langle x \rangle. M \Downarrow_{w+2} \mathcal{T}} \rightarrow_{\beta} S, \{N/x\}M \Downarrow_w \mathcal{T}$$

The derivation for the reductand is obtained by removing the final two rules from the derivation, which reduces the weight by two.

Projection $\boxed{a}. M \rightarrow_{\pi} [\frac{1}{2} \cdot \pi_{\top}^a M, \frac{1}{2} \cdot \pi_{\perp}^a M]$ Since $\mathcal{T}$ is non-empty, the derivation for the redex is the first below. The derivation for the reduct, second below, uses the evaluation rule for distributions.

$$\begin{array}{c}
\frac{S, \pi_{\top}^a M \Downarrow_v \mathcal{T}_{\top} \quad S, \pi_{\perp}^a M \Downarrow_w \mathcal{T}_{\perp}}{S, \boxed{a}. M \Downarrow_{v+w+1} \frac{1}{2} \mathcal{T}_{\top} + \frac{1}{2} \mathcal{T}_{\perp}} \\
\downarrow_{\pi} \\
\frac{S, \pi_{\top}^a M \Downarrow_v \mathcal{T}_{\top} \quad S, \pi_{\perp}^a M \Downarrow_w \mathcal{T}_{\perp}}{S, [\frac{1}{2} \cdot \pi_{\top}^a M, \frac{1}{2} \cdot \pi_{\perp}^a M] \Downarrow_{[v,w]} \frac{1}{2} \mathcal{T}_{\top} + \frac{1}{2} \mathcal{T}_{\perp}}
\end{array}$$

Sequential The five cases of sequential reduction follow. In each case the sequential rule is pushed further up the derivation and the weight is unchanged. If the first term of a sequential composition evaluates to an empty distribution, then so does the whole term. Since we are working with $\mathcal{T} \neq \emptyset$ we may assume that initial terms have a non-empty evaluation throughout.

Sequence-unit $\star ; P \rightarrow_{\sigma} P$ Since $\mathcal{T}$ is non-empty, the first argument of the sequential composition must also be, yielding the derivation of the redex below left.

$$\frac{\overline{S, \star \Downarrow_0 [1 \cdot S]}}{S, \star ; P \Downarrow_w \mathcal{T}} \rightarrow_{\sigma} S, P \Downarrow_w \mathcal{T}$$

The required derivation for the reduct arises as a sub-derivation, extracted

above right.

Sequence—pop $\langle x \rangle. M ; P \rightarrow_\sigma \langle x \rangle. (M ; P)$ ($x \notin \text{fv}(P)$) The derivations are as below. Again the first argument of the sequential composition must be non-empty.

$$\begin{array}{c}
 \frac{R, \{N/x\}M \Downarrow_w [p_i \cdot S_i]_{i \leq n}}{R N, \langle x \rangle. M \Downarrow_{w+1} [p_i \cdot S_i]_{i \leq n}} \quad (S_i, P \Downarrow_{v_i} \mathcal{T}_i)_{i \leq n} \\
 \hline
 R N, \langle x \rangle. M ; P \Downarrow_{(w+1 + \sum_{i \leq n} v_i)} \sum_{i \leq n} p_i \mathcal{T}_i \\
 \\
 \downarrow_\sigma \\
 \frac{R, \{N/x\}M \Downarrow_w [p_i \cdot S_i]_{i \leq n} \quad (S_i, P \Downarrow_{v_i} \mathcal{T}_i)_{i \leq n}}{R, \{N/x\}M ; P \Downarrow_{(w + \sum_{i \leq n} v_i)} \sum_{i \leq n} p_i \mathcal{T}_i} \\
 \hline
 R N, \langle x \rangle. (M ; P) \Downarrow_{(w+1 + \sum_{i \leq n} v_i)} \sum_{i \leq n} p_i \mathcal{T}_i
 \end{array}$$

The derivation for the reductand relies on the fact that x is not free in P , from which it follows that $\{N/x\}(M ; P) = (\{N/x\}M) ; P$.

Sequence—push $[N]. M ; P \rightarrow_\sigma [N]. (M ; P)$ The derivations are as follows:

$$\begin{array}{c}
 \frac{R N, M \Downarrow_w [p_i \cdot S_i]_{i \leq n}}{R, [N]. M \Downarrow_{w+1} [p_i \cdot S_i]_{i \leq n}} \quad (S_i, P \Downarrow_{v_i} \mathcal{T}_i)_{i \leq n} \\
 \hline
 R, [N]. M ; P \Downarrow_{(w+1 + \sum_{i \leq n} v_i)} \sum_{i \leq n} p_i \mathcal{T}_i \\
 \\
 \downarrow_\sigma \\
 \frac{R N, M \Downarrow_w [p_i \cdot S_i]_{i \leq n} \quad (S_i, P \Downarrow_{v_i} \mathcal{T}_i)_{i \leq n}}{R N, M ; P \Downarrow_{(w + \sum_{i \leq n} v_i)} \sum_{i \leq n} p_i \mathcal{T}_i} \\
 \hline
 R, [N]. (M ; P) \Downarrow_{(w+1 + \sum_{i \leq n} v_i)} \sum_{i \leq n} p_i \mathcal{T}_i
 \end{array}$$

Sequence—generator $\boxed{a}. M ; P \rightarrow_\sigma \boxed{a}. (M ; P)$ ($a \notin \text{fe}(P)$) The deriva-

tions are as follows:

$$\begin{array}{c}
\frac{R, \pi_{\perp}^a M \Downarrow_v [p_i \cdot S_i]_{n < i \leq m} \quad R, \pi_{\top}^a M \Downarrow_w [p_i \cdot S_i]_{i \leq n}}{R, \boxed{a}. M \Downarrow_{w+v+1} \frac{1}{2} [p_i \cdot S_i]_{i \leq m}} \quad (S_i, P \Downarrow_{u_i} \mathcal{T}_i)_{i \leq m} \\
\hline
R, \boxed{a}. M ; P \Downarrow_{w+v+1+u} \sum_{i \leq m} \frac{1}{2} p_i \mathcal{T}_i \\
\downarrow_{\sigma} \\
\frac{\frac{R, \pi_{\top}^a M \Downarrow_w [p_i \cdot S_i]_{i \leq n} \quad (S_i, P \Downarrow_{u_i} \mathcal{T}_i)_{i \leq n}}{R, \pi_{\top}^a M ; P \Downarrow_{w+u_{\perp}} \sum_{i \leq n} p_i \mathcal{T}_i} \quad \frac{R, \pi_{\perp}^a M \Downarrow_v [p_i \cdot S_i]_{n < i \leq m} \quad (S_i, P \Downarrow_{u_i} \mathcal{T}_i)_{n < i \leq m}}{R, \pi_{\perp}^a M ; P \Downarrow_{v+u_{\top}} \sum_{n < i \leq m} p_i \mathcal{T}_i}}{R, \boxed{a}. (M ; P) \Downarrow_{w+v+1+u} \sum_{i \leq m} \frac{1}{2} p_i \mathcal{T}_i}
\end{array}$$

In the derivation for the reductand the sequential premises are stacked for space. We use the following abbreviations:

$$u = \sum_{i < m} u_i \quad u_{\top} = \sum_{i \leq n} u_i \quad u_{\perp} = \sum_{n < i \leq m} u_i$$

Sequence—sequence $N ; (M ; P) \rightarrow_{\sigma} (N ; M) ; P$ The derivations are below, along with a list of the abbreviations used.

$$\begin{array}{c}
R, N \Downarrow_w [p_i \cdot S_i]_{i \leq n} \quad \left(\frac{S_i, M \Downarrow_{v_i} \mathcal{T}_i \quad (T_{ij}, P \Downarrow_{u_{ij}} \mathcal{U}_{ij})_{j \leq m_i}}{S_i, M ; P \Downarrow_{v_i+u_i} \mathcal{U}_i} \right)_{i \leq n} \\
\hline
R, N ; (M ; P) \Downarrow_{w+v+u} \mathcal{U} \\
\downarrow_{\sigma} \\
\frac{R, N \Downarrow_w [p_i \cdot S_i]_{i \leq n} \quad (S_i, M \Downarrow_{v_i} \mathcal{T}_i)_{i \leq n}}{R, N ; M \Downarrow_{w+v} \mathcal{T}} \quad (T_{ij}, P \Downarrow_{u_{ij}} \mathcal{U}_{ij})_{j \leq m_i, i \leq n} \\
\hline
R, (N ; M) ; P \Downarrow_{w+v+u} \mathcal{U}
\end{array}$$

$$\begin{aligned}
\mathcal{T}_i &= [q_{ij} \cdot T_{ij}]_{j \leq m_i} & \mathcal{U}_i &= \sum_{j \leq m_i} q_{ij} \mathcal{U}_{ij} & u_i &= \sum_{j \leq m_i} u_{ij} \\
\mathcal{T} &= \sum_{i \leq n} p_i \mathcal{T}_i & \mathcal{U} &= \sum_{i \leq n} p_i \mathcal{U}_i & v &= \sum_{i \leq n} v_i \\
&= [p_i q_{ij} \cdot T_{ij}]_{j \leq m_i, i \leq n} & &= \sum_{i \leq n} \sum_{j \leq m_i} p_i q_{ij} \mathcal{U}_{ij} & u &= \sum_{i \leq n} u_i
\end{aligned}$$

We extend what we have proven for base-reductions to results for head-reduction by induction on the definition of head contexts.

First let us look at head beta-reduction. Let $M \rightarrow_{\beta h} N$, our induction hypothesis is that the lemma holds on this reduction. The grammar of the head context tells us that the following two cases produce larger head contexts.

Application $[P].M \rightarrow_{\beta h} [P].N$ Since $\mathcal{T}$ is non-empty, the initial derivation is as below:

$$\frac{S P, M \Downarrow_w \mathcal{T}}{S, [P].M \Downarrow_{w+1} \mathcal{T}}$$

If the reduction reduces w , it also reduces $w + 1$.

Abstraction $\langle x \rangle.M \rightarrow_{\beta h} \langle x \rangle.N$ Since $\mathcal{T}$ is non-empty, the initial derivation is as below:

$$\frac{S, \{P/x\}M \Downarrow_w \mathcal{T}}{S P, \langle x \rangle.M \Downarrow_{w+1} \mathcal{T}}$$

Given that $M \rightarrow_{\beta h} N$ we also have that $\{P/x\}M \rightarrow_{\beta h} \{P/x\}N$, if this reduces the weight w it also reduces $w + 1$. The length of the head context is unchanged by substitution and thus the induction hypothesis holds.

Sequencing-reduction leaves the weight unchanged. That the weight is unchanged by sequencing-reduction within head contexts follows immediately from the above constructs. The idea is that the head context has a determined effect on the weight that is not otherwise dependant on the term in the context hole.

Finally, we will cover projective reduction in context. The change of weight observed for the base projective-reduction $[v + w + 1] \rightarrow [v, w]$ is not preserved under head contexts. To be precise, for a head context of size n , one which contains n abstractions and applications, projective reduction within the head context yields the weight relation $[n + v + w + 1] \rightarrow [v + n, w + n]$. It is due to

this duplication of the head context's weight contribution that we use multiset rather than sum for the weight of a distribution.

Let $M \rightarrow_\pi [\frac{1}{2} \cdot N_\top \frac{1}{2} \cdot N_\perp]$ and once again consider the cases which increase the head context. The proof method is similar to that seen above, but in this case the head context must be reconstructed within the projection in the reductand.

Application $[P].M \rightarrow_\pi [\frac{1}{2} \cdot [P].N_\top, \frac{1}{2} \cdot [P].N_\perp]$ The derivation for $[P].M$ is as follows:

$$\frac{S P, M \Downarrow_w \mathcal{T}}{S, [P].M \Downarrow_{w+1} \mathcal{T}}$$

Applying the induction hypothesis to the premise of this derivation we get the derivation below for the distribution $\mathcal{N} = [\frac{1}{2} \cdot N_\top, \frac{1}{2} \cdot N_\perp]$, where $\mathcal{T} = \frac{1}{2} \mathcal{T}_\top + \frac{1}{2} \mathcal{T}_\perp$ and $u, v < w$.

$$\frac{S P, N_\top \Downarrow_u \mathcal{T}_\top \quad S P, N_\perp \Downarrow_v \mathcal{T}_\perp}{S P, \mathcal{N} \Downarrow_{[u,v]} \mathcal{T}}$$

Using the premises of which we can construct the derivation for $\mathcal{P} = [\frac{1}{2} \cdot [P].N_\top \frac{1}{2} \cdot [P].N_\perp]$:

$$\frac{\frac{S P, N_\top \Downarrow_u \mathcal{T}_\top}{S, [P].N_\top \Downarrow_{u+1} \mathcal{T}_\top} \quad \frac{S P, N_\perp \Downarrow_v \mathcal{T}_\perp}{S, [P].N_\perp \Downarrow_{v+1} \mathcal{T}_\perp}}{S, \mathcal{P} \Downarrow_{[u+1, v+1]} \mathcal{T}}$$

Since $u+1, v+1 < w+1$ this derivation satisfies the lemma.

Abstraction $\langle x \rangle.M \rightarrow_\pi [\frac{1}{2} \cdot \langle x \rangle.N_\top \frac{1}{2} \cdot \langle x \rangle.N_\perp]$ The derivation for $\langle x \rangle.M$ is as follows:

$$\frac{S, \{P/x\}M \Downarrow_w \mathcal{T}}{S P, \langle x \rangle.M \Downarrow_{w+1} \mathcal{T}}$$

Since the capture-avoiding nature of substitution extends to events, it follows that given the reduction for M , we also have the following, where we abbreviate the reduct as $\mathcal{P}$.

$$\{P/x\}M \rightarrow_\pi [\frac{1}{2} \cdot \{P/x\}N_\top \frac{1}{2} \cdot \{P/x\}N_\perp] = \mathcal{N}$$

Applying the induction hypothesis we get the following derivation for $\mathcal{N}$, where again $\mathcal{T} = \frac{1}{2}\mathcal{T}_\top + \frac{1}{2}\mathcal{T}_\perp$ and $u, v < w$.

$$\frac{S, \{P/x\}N_\top \Downarrow_u \mathcal{T}_\top \quad S, \{P/x\}N_\perp \Downarrow_v \mathcal{T}_\perp}{S, \mathcal{N} \Downarrow_{[u,v]} \mathcal{T}}$$

Then for $[\frac{1}{2} \cdot \langle x \rangle. N_\top, \frac{1}{2} \cdot \langle x \rangle. N_\perp]$ we get the required derivation.

$$\frac{\frac{S, \{P/x\}N_\top \Downarrow_u \mathcal{T}_\top}{S P, \langle x \rangle. N_\top \Downarrow_{u+1} \mathcal{T}_\top} \quad \frac{S, \{P/x\}N_\perp \Downarrow_v \mathcal{T}_\perp}{S P, \langle x \rangle. N_\perp \Downarrow_{v+1} \mathcal{T}_\perp}}{S P, [\frac{1}{2} \cdot \langle x \rangle. N_\top, \frac{1}{2} \cdot \langle x \rangle. N_\perp] \Downarrow_{[u+1, v+1]} \mathcal{T}}$$

□

Corollary 36 *Let $S, M \Downarrow_w \mathcal{T}$ where $\mathcal{T} \neq \emptyset$. Head reduction of M is terminating.*

Proof. This follows directly from Lemma 35 as follows. Each β -step or projective-step reduces w . Meanwhile sequencing-reduction does not increase the weight of the relation. Since w is a multiset of naturals, the number of β and projective reduction cannot be infinite. Sequencing-reduction alone is terminating by Lemma 18. The number of times a block of sequencing reductions can arise in a reduction is bounded by the number of other reduction steps: thus sequencing-reduction overall contributes a finite number of steps to the reduction. It follows that the process of head reduction cannot occur infinitely. □

5.2 Head normalisation

In the previous section we showed two things; that head reduction of a term does not affect the return stack of a machine run and that reduction of terms with non-null return is terminating. The latter gives us an approach to proving *partial normalisation* in the sense that a term undergoing reduction will reach a distribution which has a given proportion of terms in head-normal form. By partitioning out the terms of the distribution with non-null return we can show that head reduction terminates on such terms, these terms reach a head-normal

form.

This is formalised in the following lemma, which determines that if evaluation returns a distribution of weight p , then head-reduction terminates with a probability of at least p .

Lemma 37 *If $S, \mathcal{M} \Downarrow_{\mathcal{W}} \mathcal{T}$ then $\mathcal{M} \rightarrow_{\text{h}} \mathcal{N}_0 + \mathcal{N}_1$ where all terms in $\mathcal{N}_0$ are head normal and $|\mathcal{N}_0| \geq |\mathcal{T}|$.*

Proof. Let $\mathcal{M} = [p_i \cdot M_i]_{i \leq n}$. As a distribution its evaluation derivation must be of the following form:

$$\frac{(S, M_i \Downarrow_{w_i} \mathcal{T}_i)_{i \leq n}}{S, [p_i \cdot M_i]_{i \leq n} \Downarrow_{[w_i]_{i \leq n}} \sum_{i \leq n} p_i \mathcal{T}_i}$$

Head reduction on any M_i where $\mathcal{T}_i \neq \emptyset$ terminates by Corollary 36.

Thus $\mathcal{M} \rightarrow_{\text{h}} \mathcal{N} = [p_i \cdot N_i]$ such that $S, N_i \Downarrow \mathcal{T}_i$ and if $\mathcal{T}_i \neq \emptyset$ then N_i is head-normal (note the slight abuse of notation here: N_i may be a distribution due to projective reduction). The derivation for $\mathcal{N}$ decomposes as follows:

$$\frac{S, \mathcal{N}_0 \Downarrow \mathcal{T} \quad S, \mathcal{N}_1 \Downarrow \emptyset}{S, \mathcal{N} \Downarrow \mathcal{T}}$$

Then the distribution $\mathcal{N}_0 = [p_i \cdot N_i \mid i \leq n, \mathcal{T}_i \neq \emptyset]$ is head-normal. The weights of $\mathcal{N}_0$ and $\mathcal{T}$ are as follows.

$$|\mathcal{N}_0| = \sum [p_i \mid i \leq n, \mathcal{T}_i \neq \emptyset] \quad |\mathcal{T}| = \sum_{i \leq n} p_i |\mathcal{T}_i|$$

Since null returns contribute nothing to the weight of $\mathcal{T}$ and the sub-distributions each have weight at most one, it follows that $|\mathcal{N}_0| \geq |\mathcal{T}|$. $\square$

Thus far we have related head-normalisation to the evaluation relation and the evaluation relation to typability. All that is left is to sew it together, giving the desired relationship between the type system and normalisation. I have already demonstrated the required components: typing gives successful evaluation on the machine when provided a stack of the correct type; type inhabitation means that

there is always such a stack available; and evaluation guarantees probabilistic termination of head reduction.

Theorem 38 *For a closed M , if $M : \overline{A} \stackrel{p}{\Rightarrow} \overline{C}$ then $M \rightarrow_{\text{h}} \mathcal{N}_0 + \mathcal{N}_1$ where all terms in $\mathcal{N}_0$ are head-normal and $|\mathcal{N}_0| \geq p$.*

Proof. We provide M with an input stack consisting only of zero terms 0_A . Let S be the stack of zero terms for $\overline{A}$. By Proposition 24 zero-terms inhabit their types, so $S : \overline{A}$.

Noting that zero terms are also closed, Theorem 27 immediately gives an evaluation $S, M \Downarrow \mathcal{T}$ where $|\mathcal{T}| \geq p$.

Finally, applying Lemma 37 to the corresponding weighted derivation $S, M \Downarrow_w \mathcal{T}$ (which exists by Observation 34) gives a head-reduction $M \rightarrow_{\text{h}} \mathcal{N}_0 + \mathcal{N}_1$ where all terms in $\mathcal{N}_0$ are head-normal and $|\mathcal{N}_0| \geq |\mathcal{T}| \geq p$.

□

We recall that the main theorem for this part of the thesis was Theorem 12: probabilistic termination for the PEA. This result follows immediately from Theorem 38, above, by conservativity of the calculus and type system of SPEA over the PEA (for details see Figure 5-1). Notably, the proof of Theorem 38 makes essential use of properties of the sequential extension, in particular the notion of successful termination which is absent from the PEA. Proving probabilistic head-normalisation of the PEA is done by running the Krivine machine with a stack of terms that are outside of the calculus itself.

Part II

Towards Almost-Sure Termination

Chapter 6

The Join Calculus

In this, the second part of the thesis, we shift our approach slightly. From now on we work with a formulation of the *Join Calculus* from Downen et al’s papers on continuations for compiler optimisation [21, 52]. This calculus gives an alternative way to model complex control flows, integrating the different modes of computation with the slogan “Think in sequent calculus; work in λ -calculus.” [52]. The main point of these works on the join calculus was to distinguish some let bindings as join points and exploit the recursive nature of these joins to optimise programs with *fusion*: the recombination of distinct computation branches. We have opted to work with this calculus because it contains a join construct which is tail-recursive by construction, and thus gives a term calculus with while-loops. We speculate that this is an appropriate setting for the study of almost sure termination in a higher-order calculus. In our formulation the *tail-recursive join* is decomposed into a *non-recursive join* and, in Chapter 8, an explicit *co-recursor*.

The first task of this chapter will be to show that a version of the join-calculus can be typed by a multi-conclusion sequent calculus. Our intention is that this sequent calculus will represent the inputs and outputs of a branching computation. The multi-conclusion will be taken to represent the outcomes of alternative branches. This motivation shapes the rules we have chosen. The use of a multi-conclusion system is further enforced when we work with co-recursion in later chapters. The type system we introduce here naturally provides the annota-

tion necessary for a multi-conclusion implication-right rule, in the form of a free-variable condition. We will further show the connection between the join-calculus as a sequent calculus, and the join-calculus as a continuation-interpretation of this sequent calculus in standard intuitionistic logic: where the multiple conclusions are continuations. This interpretation connects to the lambda-mu-calculus which is defined as a multi-conclusion natural-deduction calculus. The extra construct, mu-abstraction, makes the lambda-mu calculus classical while retaining computational properties.

The Join Calculus is a calculus with *continuation passing* [68, 54, 70]. A continuation interrupts the execution of a program and directs evaluation to a different part of the program. Continuations are the basis of control flow in programming and occur in many programming languages in the form of error catching, breaks and go-to statements. Our aim when working with this calculus is to model non-deterministic branching of computation, where the outcome of a computation may be passed to a program appropriate to its type or output partway through the computation. Jump continuations provide a means to indicate the jump required from each outcome to its use case. This should allow for more expressive probabilistic systems than is covered by type matched choice.

For this purpose we introduce *co-variables*. These gather outcomes which are destined to be used in a particular function and pass them where they are required. An outcome may even trigger reuse of the original function; resulting in co-recursive loops, which we focus on in Chapter 8. Although we do not deal with loops at this stage they have informed several of our design decisions, in particular the expression of joins as sums and the one sided nature of weakening. For now we consider the non-recursive fragment of the Join Calculus. The sequent system provides a type semantics for these terms, while a related natural deduction system is isomorphic to the term calculus. These type systems will provide admissibility checks on the notions of beta and permutative reduction defined on the term calculus in the next chapter.

6.1 Multi-conclusion Sequent Calculi

One of the main problems associated with a multi conclusion system is failure of cut elimination. A naive multi conclusion system, such as we introduce in this section, will have valid sequents with no cut free proof. In previous works the root of this issue has been identified as the unrestricted implication right rule:

$$\frac{\Gamma, A \vdash B, \Delta}{\Gamma \vdash A \rightarrow B, \Delta} \rightarrow R$$

This rule gives rise to statements such as $(A + B \rightarrow A) + B$ which are unsound in an intuitionistic setting [36]. Thus the logic reverts to a classical system and computational properties such as confluence of reduction are lost under the natural semantics.

To solve this issue de Pavia [18] restricted the rule to a single conclusion variant:

$$\frac{\Gamma, A \vdash B}{\Gamma \vdash A \rightarrow B} \rightarrow R$$

This approach has the disadvantage of blocking full cut-elimination [7, 5]. In later work with Hyland [36], de Paiva critiques this as lacking “computational significance” and introduces an independence condition in its place. An annotated version of the calculus allows a multi-conclusion rule on the proviso that A does not occur in the rest of the conclusion Δ . In other words that that the consequent B of the introduced implication $A \rightarrow B$ is the only proof in the conclusion which depends on the antecedent A . They show this condition to be sufficient by way of a categorical semantics, showing that in the categorical model of such systems a function from the antecedent to the conclusion of a sequent has a corresponding function for the abstracted sequent when the free variable condition holds. Note that the single conclusion variant arises as a special case of this more general condition.

Additionally to the semantic disadvantages of the single conclusion variant recall that we wish to treat the multi-conclusion as representing multiple outcomes of a probabilistic computation, thus restriction to a single conclusion is not possible.

Definition 39 *Formulae* of the sequent calculus are given by the following grammar.

$$A, B ::= \perp \mid \top \mid A \rightarrow B \mid A \wedge B \mid A \vee B$$

From left to right these are: the *logical constants* $\perp$ and $\top$, an *implication* $A \rightarrow B$, a *conjunction* $A \wedge B$, and a *disjunction* $A \vee B$.

Definition 40 Proofs in the sequent calculus are given by the system of rules in Figure 6-1.

$$\begin{array}{c}
\frac{}{\Gamma, A \vdash A} \text{ax} \qquad \frac{\Gamma \vdash \Delta, A \quad A, \Gamma' \vdash \Delta'}{\Gamma, \Gamma' \vdash \Delta, \Delta'} \text{cut} \\
\\
\frac{\Gamma, A \vdash B, \Delta}{\Gamma \vdash A \rightarrow B, \Delta} \rightarrow^{\dagger} \qquad \frac{\Gamma \vdash \Delta, A \quad B, \Gamma' \vdash \Delta'}{\Gamma, A \rightarrow B, \Gamma' \vdash \Delta, \Delta'} \rightarrow^{\text{L}} \\
\\
\frac{A, B, \Gamma \vdash \Delta}{A \wedge B, \Gamma \vdash \Delta} \wedge^{\text{L}} \qquad \frac{\Gamma \vdash \Delta, A \quad \Gamma' \vdash \Delta', B}{\Gamma, \Gamma' \vdash \Delta, \Delta', A \wedge B} \wedge^{\text{R}} \\
\\
\frac{\Gamma \vdash \Delta, A, B}{\Gamma \vdash \Delta, A \vee B} \vee^{\text{R}} \qquad \frac{A, \Gamma \vdash \Delta \quad B, \Gamma' \vdash \Delta'}{A \vee B, \Gamma, \Gamma' \vdash \Delta, \Delta'} \vee^{\text{L}} \\
\\
\frac{A, A, \Gamma \vdash \Delta}{A, \Gamma \vdash \Delta} \text{cl} \qquad \frac{\Gamma \vdash \Delta, A, A}{\Gamma \vdash \Delta, A} \text{cr}
\end{array}$$

Figure 6-1: Towards a multi-conclusion intuitionistic sequent system. The implication right rule ($\dagger$) requires modification to retain intuitionistic properties.

As currently stated this system falls foul of the implication-right rule and is not intuitionistic. We want both this rule, to embed the lambda calculus, and multi-conclusions, to reflect distributions of outcomes. Since we are not willing to restrict the conclusion of our sequent system to a single formula, we need to annotate the sequent calculus with a term calculus. This will allow us to enforce the side condition from [36] of “binding all x ” to force the rule to be intuitionistic.

Note that in this calculus the axiom rule provides an implicit left sided weakening with no corresponding right sided weakening. This imbalance is due to our desire to have the sequent conclusion represent outcomes of a branching computation.

In this context right sided weakening would introduce invalid outcomes and thus skew any analysis of the distribution of outcomes.

6.2 The Term Calculus

The Join calculus is defined in two stages. *Value terms* extend the lambda calculus with pairing, projection and injection operators. *Computation terms* are then defined using value terms, co-variables and terms as one of: the application of a co-variable to a value term, the joining of two terms along a variable/covariable link or a choice between two terms as decided by the evaluation of a preterm.

Definition 41 (Terms) The *terms* of the *Join Calculus* are constructed according to the following grammar.

Value terms: $P, Q ::= x \mid \lambda x. P \mid P Q \mid \pi_i P \mid \iota_i P \mid (P, Q)$

Computation terms: $M, N ::= j P \mid \text{join } j x = N \text{ in } M \mid$
 $\text{case } P \text{ of } (\iota_1 x \rightarrow M, \iota_2 y \rightarrow N)$

From left to right, the value terms are a *variable* x , an *abstraction* $\lambda x. P$ that binds x in P , an *application* $P Q$, a *projection* $\pi_i P$ and an *injection* $\iota_i P$ where $i \in \{1, 2\}$, and a *pair* (P, Q) .

The *computation terms* are a *jump* $j P$, a *join* $\text{join } j x = N \text{ in } M$ that binds x in N and j in M , a *case* $\text{case } P \text{ of } (\iota_1 x \rightarrow M, \iota_2 y \rightarrow N)$ that binds x in M and y in N .

In this definition the co-variables j always occur in function position, acting on arguments.

This two phase definition is reminiscent of call-by-push value [48] for a very good reason. Value terms are exactly those terms which can be passed as arguments: the terms allowed to be pushed onto a stack (Appendix A) whereas computation terms cannot; these can only be reduced.

We say a co-variable j is *bound* in a term N if all occurrences of j are in the scope of a **join** of the form $jx = \dots$. The sets of free and bound variables and co-variables are denoted as follows:

Free variables:	$\text{fv}(-)$
Bound variables:	$\text{bv}(-)$
Free co-variables:	$\text{fc}(-)$
Bound co-variables:	$\text{bc}(-)$

There are a number of ways to think about the role of co-variables. Relating back to the previous part of this thesis co-variables can be considered a generalisation of the termination condition denoted by **skip** in the sequential calculus: indicating a qualified failure where there is still a specified computation to complete. Along this vein since **join** is indicating a further computation, it may be thought of as a placeholder for a term although **join** does not behave as substitution might. A better intuition may be to consider them as labels, which direct the output into a different module of the computation, thus influencing the control flow. Continuations can also be thought of as the Curry Howard interpretation of negation from classical logic [32]. Mathematically and linguistically we establish co-variables as the dual of variables [22, 12, 37], most of the rules for this calculus have explicit duals.

6.3 Term Assignment System

In this section we present the sequent calculus as a type system for the Join calculus defined above.

Definition 42 A typing *sequent* $M :: x : A, \Gamma \vdash j : B, \Delta$ assigns types to the free co-variables of a term in the context of a typing Γ of the free variables of the term.

Definition 43 The *sequent type system* for the join calculus is given by the typing rules in Figure 6-2

The system needs the following definition which amounts to substitution of pattern matched sub-terms [7].

Definition 44 Compound substitution $\{T\{P\}/j P\}$ is defined inductively as:

$$\begin{aligned} \{T\{P\}/j P\}iQ &= \begin{cases} T\{Q\} & \text{if } i = j \\ iQ & \text{if } i \neq j \end{cases} \\ \{T\{P\}/j P\} \text{join } ix = N \text{ in } M &= \begin{cases} \text{join } ix = \{T\{P\}/j P\}N \text{ in } M & \text{if } i = j \\ \text{join } ix = \{T\{P\}/j P\}N \text{ in } \{T\{P\}/j P\}M & \text{if } i \neq j \end{cases} \\ \{T\{P\}/j P\} \text{case } Q \text{ of } (\iota_1 x \rightarrow M, \iota_2 y \rightarrow N) &= \text{case } Q \text{ of } (\iota_1 x \rightarrow \{T\{P\}/j P\}M, \iota_2 y \rightarrow \{T\{P\}/j P\}N) \end{aligned}$$

The side condition on the implication right rule ($\rightarrow R$) is the necessary one to make the calculus intuitionistic. The independence of the isolated conclusion and thus admissibility of the rule can be confirmed through a permutation of rules which essentially unravels the derivation to separate the derivation with the observed outcome from all other outcomes, creating one section which may be abstracted and another that may be retyped as the variable does not occur free in it.

$$\begin{array}{c}
\frac{}{j\ x :: \Gamma, x : A \vdash j : A}^{\text{ax}} \\
\\
\frac{M :: \Gamma \vdash \Delta, j : A \quad N :: x : A, \Gamma' \vdash \Delta'}{\text{join } j\ x = N \text{ in } M :: \Gamma, \Gamma' \vdash \Delta, \Delta'}^{\text{cut}} \\
\\
\frac{M :: \Gamma, x : A \vdash i : B, \Delta}{\{j\ (\lambda x. P)/i\ P\} M :: \Gamma \vdash j : A \rightarrow B, \Delta}^{\rightarrow R} \quad (\text{no } x \text{ remains free}) \\
\\
\frac{M :: \Gamma \vdash \Delta, j : A \quad N :: x : B, \Gamma' \vdash \Delta'}{\text{join } j\ z = \{y\ z/x\} N \text{ in } M :: \Gamma, y : A \rightarrow B, \Gamma' \vdash \Delta, \Delta'}^{\rightarrow L} \\
\\
\frac{M :: x : A, y : B, \Gamma \vdash \Delta}{\{\pi_1\ z/x\} \{\pi_2\ z/y\} M :: z : A \wedge B, \Gamma \vdash \Delta}^{\wedge L} \\
\\
\frac{M :: \Gamma \vdash \Delta, i : A \quad N :: \Gamma' \vdash \Delta', j : B}{\text{join } i\ x = (\text{join } j\ y = k\ (x, y) \text{ in } N) \text{ in } M :: \Gamma, \Gamma' \vdash \Delta, \Delta', k : A \wedge B}^{\wedge R} \\
\\
\frac{M :: \Gamma \vdash \Delta, i : A, j : B}{\{k\ (\iota_1\ P)/i\ P\} \{k\ (\iota_2\ P)/j\ P\} M :: \Gamma \vdash \Delta, k : A \vee B}^{\vee R} \\
\\
\frac{M :: x : A, \Gamma \vdash \Delta \quad N :: y : B, \Gamma' \vdash \Delta'}{\text{case } z \text{ of } (\iota_1\ x \rightarrow M, \iota_2\ y \rightarrow N) :: z : A \vee B, \Gamma, \Gamma' \vdash \Delta, \Delta'}^{\vee L} \\
\\
\frac{M :: x : A, y : A, \Gamma \vdash \Delta}{\{z/x\} \{z/y\} M :: z : A, \Gamma \vdash \Delta}^{\text{cl}} \\
\\
\frac{M :: \Gamma \vdash \Delta, i : A, j : A}{\{k/i\} \{k/j\} M :: \Gamma \vdash \Delta, k : A}^{\text{cr}}
\end{array}$$

Figure 6-2: Sequent typing rules for the Join Calculus

6.3.1 Diagram Intuition

The following diagrams are useful for understanding the intuition of the calculus. They trace the connections between variables and co-variables enforced by the typing rules; hence showing the expected behaviour of both computational machine and logic.

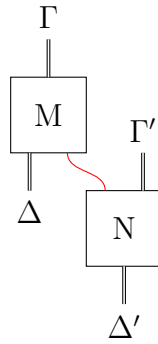
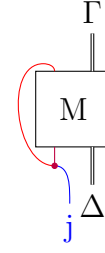
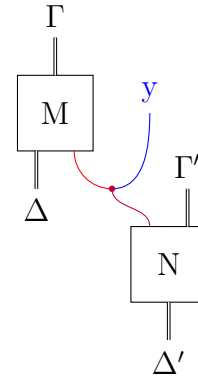


Figure 6-3: CUT

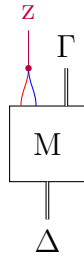
The cut rule joins a single co-variable from the conclusion of one term to a single variable in the arguments of another. This rule is fundamental to the calculus and behaves like a logical switch, altering the scope of the highlighted edge. In these diagrams the relationship between antecedents is taken to be conjunction and that between conclusions disjunction.

Figure 6-4: $\rightarrow R$

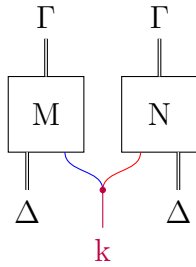
The right implication rule binds an argument of the function in a single conclusion. The side condition that no x remains free is equivalent to saying that this variable is not used in the evaluation of the remaining conclusions.

Figure 6-5: $\rightarrow L$

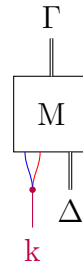
The left implication rule introduces a new variable which transforms the outcome of one term to be a compatible argument of the other.

Figure 6-6: $\wedge L$

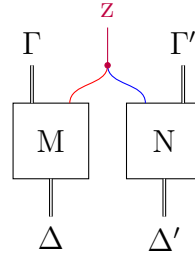
The conjunction left rule relabels a pair of arguments of a term as a single formal pair/product variable.

Figure 6-7: $\wedge R$

The conjunction right rule combines two terms as a product combining one outcome from each in a single product co-variable.

Figure 6-8: $\vee R$

The disjunction right rule relabels two co-variable outcomes of a term as a single sum co-variable.

Figure 6-9: $\vee L$

The disjunction left rule combines two terms in a sum, one variable from each term combine into a single sum variable.

6.4 Natural Deduction

The sequent calculus shows the intended meaning of the term calculus. However, it is not suitable for reduction: the presence of both contraction and co-contraction means it is subject to the Lafont counterexample to normalisation, given below. Therefore an equivalent natural-deduction type system will be given in addition in which reduction will take place, instead.

$$\frac{\frac{\Gamma \vdash \Delta, A, A}{\Gamma \vdash \Delta, A} \text{CR} \quad \frac{A, A, \Gamma' \vdash \Delta'}{A, \Gamma' \vdash \Delta'} \text{CL}}{\Gamma, \Gamma' \vdash \Delta, \Delta'} \text{CUT}$$

In this section we will interpret continuations as functions which apply to a term and produce a return type. The standard higher-order (System F) encoding of disjunction is as follows:

$$A \vee B = \forall R. (A \rightarrow R) \rightarrow (B \rightarrow R) \rightarrow R$$

However this type also contains terms that do not represent disjunctions. It is for this reason that we require the syntactic restrictions on the join calculus, which enforce this, and the typing in the sequent system, which demonstrates that they enforce this.

The join calculus has two meanings: as literal terms annotating sequent proofs; and as the continuation-encoding of these terms into standard natural deduction. Here we see the latter. To distinguish the two calculi, we will refer to *sequents* $\Gamma \vdash \Delta$ and *judgements* $\Gamma \vdash M : A$.

The interpretation of a sequent $\llbracket \Gamma \vdash \Delta \rrbracket$ will be a natural-deduction judgement written $\Gamma \mid \Delta \vdash M : \perp$, where the vertical bar $\mid$ is simply a different notation for the comma to separate the two parts of the original sequent Γ and Δ , and where the latter is silently converted by taking each $j : A$ in the sequent to $j : A \rightarrow \perp$ in the judgement.

Definition 45 Figure 6.4 shows the standard proof rules for natural deduction

$$\begin{array}{c}
\frac{}{\Gamma, x: A \vdash x: A} \quad \frac{\Gamma, x: A \vdash P: B}{\Gamma \vdash \lambda x. P: A \rightarrow B} \quad \frac{\Gamma \vdash Q: A \quad \Gamma \vdash P: A \rightarrow B}{\Gamma \vdash PQ: B} \\
\\
\frac{\Gamma \vdash P: A_i}{\Gamma \vdash \iota_i P: A_1 \vee A_2} \quad \frac{\Gamma \vdash P: A_1 \wedge A_2}{\Gamma \vdash \pi_i P: A_i} \quad \frac{\Gamma \vdash P: A \quad \Gamma \vdash Q: B}{\Gamma \vdash (P, Q): A \wedge B} \\
\\
\frac{\Gamma \vdash P: A \vee B \quad x: A, \Gamma \vdash M: C \quad y: B, \Gamma \vdash N: C}{\Gamma \vdash \text{case } P \text{ of } (\iota_1 x \rightarrow M, \iota_2 y \rightarrow N): C}
\end{array}$$

Figure 6-10: Natural Deduction with Sums

with sums [29, 61].

Definition 46 The *continuation interpretation* $\llbracket - \rrbracket$ of the sequent system in natural deduction is given as follows:

- The interpretation $\llbracket M \rrbracket$ of a join-term M is a term of the lambda calculus (extended with sums and products).
- The interpretation $\llbracket M :: \Gamma \vdash \Delta \rrbracket$ of a sequent $M :: \Gamma \vdash \Delta$ is a judgement $\Gamma \mid \Delta \vdash \llbracket M \rrbracket: \perp$ where a typing $j: A$ of Δ is translated into its continuation interpretation $j: A \rightarrow \perp$.
- The interpretation of a sequent derivation is a natural deduction derivation.

Note that the value terms of the join calculus embed directly into the λ -calculus. The interpretation extends this embedding to computation terms. This extension simply interprets co-variables as variables and commutes with the case construct.

$$\llbracket \text{case } P \text{ of } (\iota_1 x \rightarrow M, \iota_2 y \rightarrow N) \rrbracket = \text{case } P \text{ of } (\iota_1 x \rightarrow \llbracket M \rrbracket, \iota_2 y \rightarrow \llbracket N \rrbracket)$$

The interesting case is a *join* which we interpret by two abstractions and an application:

$$\llbracket \text{join } j \text{ } x = M \text{ in } N \rrbracket = (\lambda j. \llbracket N \rrbracket)(\lambda x. \llbracket M \rrbracket)$$

Beta reduction of this term gives the same evaluation as through substitutions, in the sense of reduction defined in the next chapter, since j in N must occur as

$j P$:

$$(\lambda j. N)(\lambda x. M) \rightarrow \{\lambda x. M/j\}N \rightarrow \cdots \rightarrow \{\{P/x\}M/j P\}N$$

The semantics of the sequent system are preserved by this formulation, but there is less need for side conditions.

Proposition 47 *For M the interpretation of a term of the join calculus the following natural deduction typing rule is admissible, where x does not remain free in the conclusion.*

$$\frac{\Gamma, x: A, j: B \rightarrow \perp \vdash M: \perp}{\Gamma, j: (A \rightarrow B) \rightarrow \perp \vdash \{j(\lambda x. P)/j P\}M: \perp} \lambda'$$

Proof. Supposing that we have a matching typing for M in the Natural deduction system we show by induction on the grammar of computation terms that we have a matching typing judgement for the substitution.

Jump case: $M = jP$

$$\frac{\Gamma, x: A \mid j: B \rightarrow \perp \vdash P: B \quad \overline{\Gamma, x: A \mid j: B \rightarrow \perp \vdash j: B \rightarrow \perp}}{\Gamma, x: A \mid j: B \rightarrow \perp \vdash jP: \perp}$$

Note that the type of P is necessarily derivable without j in the context, so the change of type does not affect this judgement.

$$\frac{\frac{\Gamma, x: A \mid j: (A \rightarrow B) \rightarrow \perp \vdash P: B}{\Gamma \mid j: (A \rightarrow B) \rightarrow \perp \vdash \lambda x. P: A \rightarrow B} \quad \overline{\Gamma, x: A \mid j: (A \rightarrow B) \rightarrow \perp \vdash j: (A \rightarrow B) \rightarrow \perp}}{\Gamma \mid j: (A \rightarrow B) \rightarrow \perp \vdash j(\lambda x. P): \perp}$$

Switch case: $M = \text{case } Q \text{ of } (\iota_1 y_1 \rightarrow N_1, \iota_2 y_2 \rightarrow N_2)$

$$\frac{\Gamma, x: A \mid j: B \rightarrow \perp \vdash Q: C_1 \vee C_2 \quad (y_i: C_i, \Gamma, x: A \mid j: B \rightarrow \perp \vdash N_i: \perp)_{i \in \{1,2\}}}{\Gamma, x: A \mid j: B \rightarrow \perp \vdash \text{case } Q \text{ of } (\iota_1 y_1 \rightarrow N_1, \iota_2 y_2 \rightarrow N_2): \perp}$$

Since the sequent typing has the side condition that “no x remains free” under this substitution, we may assume that x is not free in Q which as a value term has no co-variables, in particular no j for the substitution to act on: thus it may be retyped without x in the context and with free choice

of continuation type for j . The other sub-judgements allow the abstraction by induction.

$$\frac{\Gamma \mid j: (A \rightarrow B) \rightarrow \perp \vdash Q: C_1 \vee C_2 \quad \left(\frac{y_i: C_i, \Gamma, x: A \mid j: B \rightarrow \perp \vdash N_i: \perp}{y_i: C_i, \Gamma \mid j: (A \rightarrow B) \rightarrow \perp \vdash \{j(\lambda x. P)/j P\} N_i: \perp} \lambda' \right)_{i \in \{1,2\}}}{\Gamma, x: A \mid j: B \rightarrow \perp \vdash \{j(\lambda x. P)/j P\} \text{ case } Q \text{ of } (\iota_1 y_1 \rightarrow N_1, \iota_2 y_2 \rightarrow N_2): \perp}$$

Join case: $M = \llbracket \text{join } k y = N \text{ in } L \rrbracket$ The interpretation of join derives the judgement for M from those for N and L . We again rely on the side condition to ensure that x and j occur in the same subterms:

$$\frac{\frac{x: A, \Gamma \mid \Delta, j: B \rightarrow \perp, k: C \rightarrow \perp \vdash L: \perp}{x: A, \Gamma \mid \Delta, j: B \rightarrow \perp \vdash \lambda k. L: (C \rightarrow \perp) \rightarrow \perp} \lambda \quad \frac{y: C, x: A, \Gamma \mid \Delta, j: B \rightarrow \perp \vdash N: \perp}{x: A, \Gamma \mid \Delta, j: B \rightarrow \perp \vdash \lambda y. N: C \rightarrow \perp} \lambda}{x: A, \Gamma \mid \Delta, j: B \rightarrow \perp \vdash (\lambda k. L)(\lambda y. N): \perp} @$$

Applying the induction hypothesis to the subjudgements and rejoining the resultant terms gives the required derivation:

$$\frac{\frac{\Gamma \mid \Delta, j: (A \rightarrow B) \rightarrow \perp, k: C \rightarrow \perp \vdash L': \perp}{\Gamma \mid \Delta, j: (A \rightarrow B) \rightarrow \perp \vdash \lambda k. L': (C \rightarrow \perp) \rightarrow \perp} \lambda \quad \frac{y: C, \Gamma \mid \Delta, j: (A \rightarrow B) \rightarrow \perp \vdash N': \perp}{\Gamma \mid \Delta, j: (A \rightarrow B) \rightarrow \perp \vdash \lambda y. N': C \rightarrow \perp} \lambda}{\Gamma \mid \Delta, j: (A \rightarrow B) \rightarrow \perp \vdash \{j(\lambda x. P)/j P\}(\lambda k. L)(\lambda y. N): \perp} @$$

Where $L' = \{j(\lambda x. P)/j P\}L$ and $N' = \{j(\lambda x. P)/j P\}N$. $\square$

An equivalent statement is required for the sum of covariables.

Lemma 48 *For M an interpretation of a term of the join calculus the following natural deduction typing rule is admissible:*

$$\frac{\Gamma \mid \Delta, i: A \rightarrow \perp, j: B \rightarrow \perp \vdash M: \perp}{\Gamma \mid \Delta, k: A \vee B \rightarrow \perp \vdash \{k(\iota_1 P)/i P\} \{k(\iota_2 P)/j P\} M: \perp}$$

Proof. Similar to the proof of Proposition 47. $\square$

To confirm that the sequent interpretation is valid we show that that any join term which is typeable in the sequent system has an interpretation which is typeable, with an equivalent resultant type, in the natural deduction system.

Theorem 49 *If a term M of the join calculus has a sequent typing $M :: \Gamma \vdash \Delta$ then there is a corresponding judgement $\Gamma \mid \Delta \vdash \llbracket M \rrbracket: \perp$*

Proof. See Figures 6-11 and 6-12. The proof consists of extending $\llbracket - \rrbracket$ to the sequent rules. The equations use the substitution lemma: if $\Gamma \vdash M : A$ and $x : A, \Gamma \vdash N : B$ then $\Gamma \vdash \{M/x\}N : B$ $\square$

6.5 Lambda-mu

Parigot's lambda-mu-calculus [56] has similarities with our formulation of the join calculus. Both are extensions of the lambda calculus which model continuations and is typed by a multi conclusion sequent calculus. In the lambda mu calculus the lambda calculus is extended with a mu operator which acts as an abstraction for names or mu variables. The mu abstraction passes any argument to the sub-terms labeled with that name. This may be considered as an infinite abstraction.

Definition 50 Unnamed terms U, V and named terms M, N of the lambda-mu calculus are given by the following grammars:

$$U, V ::= x \mid \lambda x. U \mid U V \mid \mu \alpha. M \qquad M, N ::= [\alpha] U$$

This calculus has the usual beta reduction rule for the lambda calculus, a renaming rule for mu variables, and a structural reduction rule given as:

$$(\mu \beta. U) V \rightarrow \mu \beta. \{[\beta] W V / [\beta] W\} U$$

The substitution in this rule uses pattern matching which treats W as a variable, similarly to in our compound substitution in Definition 44.

The close connection between our join calculus and the lambda-mu calculus may be formalised by considering our co-variables as its *mu-variables*, our computation terms as its *named* terms, and our value terms as its *unnamed* terms.

Using the notation from our continuation interpretation, the μ -abstraction and -application are as follows, where the context is again split between regular and μ -variables.

$$\begin{aligned}
& \left[\left[j\ x :: \Gamma, x : A \vdash j : A, \Delta \right]^{\text{ax}} \right] = \frac{\frac{\Gamma, x : A \mid j : A \rightarrow \perp, \Delta \vdash x : A}{\Gamma, x : A \mid j : A \rightarrow \perp, \Delta \vdash j : \perp}^{\text{ax}}}{\Gamma, x : A \mid j : A \rightarrow \perp, \Delta \vdash j : \perp}^{\text{ax}} \\
& \left[\left[M :: \Gamma \vdash \Delta, j : A \quad N :: x : A, \Gamma' \vdash \Delta' \right]^{\text{cut}} \right] = \frac{\frac{\Gamma \mid \Delta, j : A \rightarrow \perp \vdash M : \perp}{\Gamma \mid \Delta \vdash \lambda j. M : (A \rightarrow \perp) \rightarrow \perp}^{\lambda} \quad \frac{x : A, \Gamma' \mid \Delta' \vdash N : \perp}{\Gamma' \mid \Delta' \vdash \lambda x. N : A \rightarrow \perp}^{\lambda}}{\Gamma, \Gamma' \mid \Delta, \Delta' \vdash (\lambda j. M)(\lambda x. N) : \perp}^{\text{ax}} \\
& \left[\left[\frac{M :: \Gamma, x : A \vdash i : B, \Delta}{\{j(\lambda x. P)/i\} M :: \Gamma \vdash j : A \rightarrow B, \Delta} \rightarrow^{\text{R}} \right] \right] = \frac{\Gamma, x : A \mid i : B \rightarrow \perp, \Delta \vdash M : \perp}{\Gamma \mid j : (A \rightarrow B) \rightarrow \perp, \Delta \vdash \{j(\lambda x. P)/i\} M : \perp}^{\lambda'} \\
& \left[\left[\frac{M :: \Gamma \vdash \Delta, j : A \quad N :: x : B, \Gamma' \vdash \Delta'}{\text{join } j\ z = \{y\ z/x\} N \text{ in } M :: \Gamma, y : A \rightarrow B, \Gamma' \vdash \Delta, \Delta'} \rightarrow^{\text{L}} \right] \right] = \\
& \quad \frac{\frac{y : A \rightarrow B, z : A, \Gamma' \mid \Delta' \vdash \{y\ z/x\} N : \perp}{y : A \rightarrow B, \Gamma' \mid \Delta' \vdash \lambda z. \{y\ z/x\} N : A \rightarrow \perp}^{\lambda} \quad \frac{\Gamma \mid \Delta, j : A \rightarrow \perp \vdash M : \perp}{\Gamma \mid \Delta \vdash \lambda j. M : (A \rightarrow \perp) \rightarrow \perp}^{\lambda}}{\Gamma, y : A \rightarrow B, \Gamma' \mid \Delta, \Delta' \vdash (\lambda j. M)(\lambda z. \{y\ z/x\} N) : \perp}^{\text{ax}} \\
& \left[\left[\frac{M :: x : A, y : B, \Gamma \vdash \Delta}{\{\pi_1\ z/x\} \{\pi_2\ z/y\} M :: z : A \wedge B, \Gamma \vdash \Delta} \wedge^{\text{L}} \right] \right] = \quad z : A \wedge B, \Gamma \mid \Delta \vdash \{\pi_1\ z/x\} \{\pi_2\ z/y\} M : \perp
\end{aligned}$$

Figure 6-11: The interpretation of sequent rules (Theorem 49), part I

$$\begin{aligned}
& \left[\frac{M :: \Gamma \vdash \Delta, i : A \quad N :: \Gamma' \vdash \Delta', j : B}{\text{join } i \ x = (\text{join } j \ y = k(x, y) \text{ in } N) \text{ in } M :: \Gamma, \Gamma' \vdash \Delta, \Delta', k : A \wedge B} \wedge R \right] = \\
& \frac{x : A, y : B \mid k : (A \wedge B) \rightarrow \perp \vdash k(x, y) : \perp \quad \frac{\Gamma' \mid \Delta', j : B \rightarrow \perp \vdash N : \perp}{\Gamma' \mid \Delta' \vdash \lambda j. N : (B \rightarrow \perp) \rightarrow \perp} \lambda}{x : A \mid k : A \wedge B \rightarrow \perp \vdash \lambda y. k(x, y) : B \rightarrow \perp} \lambda \quad \frac{x : A, \Gamma' \mid \Delta', k : A \wedge B \rightarrow \perp \vdash (\lambda j. N)(\lambda y. k(x, y)) : \perp}{\Gamma' \mid \Delta', k : A \wedge B \rightarrow \perp \vdash \lambda x. (\lambda j. N)(\lambda y. k(x, y)) : A \rightarrow \perp} \lambda \quad \frac{\Gamma \mid \Delta, i : A \rightarrow \perp \vdash M : \perp}{\Gamma \mid \Delta \vdash \lambda i. M : (A \rightarrow \perp) \rightarrow \perp} \lambda \quad \frac{}{\Gamma, \Gamma' \mid \Delta, \Delta', k : A \wedge B \rightarrow \perp \vdash (\lambda i. M)(\lambda j. N)(\lambda y. k(x, y)) : \perp} \odot \\
& \left[\frac{M :: \Gamma \vdash \Delta, i : A, j : B}{\{k(\iota_1 P)/i \ P\} \{k(\iota_2 P)/j \ P\} M :: \Gamma \vdash \Delta, k : A \vee B} \vee R \right] = \frac{\Gamma \mid \Delta, i : A \rightarrow \perp, j : B \rightarrow \perp \vdash M : \perp}{\Gamma \mid \Delta, k : A \vee B \rightarrow \perp \vdash \{k(\iota_1 P)/i \ P\} \{k(\iota_2 P)/j \ P\} M : \perp} \\
& \left[\frac{M :: x : A, \Gamma \vdash \Delta \quad N :: y : B, \Gamma' \vdash \Delta'}{\text{case } z \text{ of } (\iota_1 x \rightarrow M, \iota_2 y \rightarrow N) :: z : A \vee B, \Gamma, \Gamma' \vdash \Delta, \Delta'} \vee L \right] = \\
& \frac{z : A \vee B, \Gamma, \Gamma' \mid \Delta, \Delta' \vdash z : A \vee B \quad x : A, \Gamma \mid \Delta \vdash M : \perp \quad j : B, \Gamma' \mid \Delta' \vdash N : \perp}{z : A \vee B, \Gamma, \Gamma' \mid \Delta, \Delta' \vdash \text{case } z \text{ of } (\iota_1 x \rightarrow M, \iota_2 y \rightarrow N) : \perp} \\
& \left[\frac{M :: x : A, y : A, \Gamma \vdash \Delta}{\{z/x\} \{z/y\} M :: z : A, \Gamma \vdash \Delta} \text{cl} \right] = \quad z : A, \Gamma \mid \Delta \vdash \{z/x\} \{z/y\} M : \perp \\
& \left[\frac{M :: \Gamma \vdash \Delta, i : A, j : A}{\{k/i\} \{k/j\} M :: \Gamma \vdash \Delta, k : A} \text{cr} \right] = \quad \Gamma \mid \Delta, k : A \vdash \{k/i\} \{k/j\} M : \perp
\end{aligned}$$

Figure 6-12: The interpretation of sequent rules (Theorem 49), part II

$$\frac{\Gamma \mid \Delta, j: B \rightarrow \perp \vdash M: \perp}{\Gamma \mid \Delta \vdash \mu j. M: B} \quad \frac{\Gamma \mid \Delta, j: A \rightarrow \perp \vdash P: A}{\Gamma \mid \Delta, j: A \rightarrow \perp \vdash [j]P: \perp}$$

Clearly, μ -application $[j] P$ is just our co-variable application $j P$, and we may use both notations interchangeably; μ -abstraction is absent from the join calculus, and extends the logic from intuitionistic to classical.

The sequent system derives *named* terms (computation terms), where $\lambda\mu$ -calculus derives types of *unnamed* terms. The distinction is bridged by making the “name”, the main co-variable of a named term, explicit, by writing $[j] P$; the sequent rules may then *only* be applied to the element j in the sequent. Then μ -abstraction may be introduced into the sequent system by the following rule, combining it with a μ -application:

$$\frac{[i] P :: \Gamma \vdash \Delta, i: A, j: B}{[j] \mu j. [i] P :: \Gamma \vdash \Delta, i: A, j: B}$$

We observe that our calculus forces two restrictions on co-variables: 1) they must occur in function position with an unnamed term as argument, and 2) they cannot be abstracted over. Interestingly, from this perspective $\lambda\mu$ -calculus relaxes 2) by introducing the μ -abstraction, but continues to enforce 1).

Completing the interpretation of $\lambda\mu$ -calculus into the sequent system, abstraction and application are as follows. First, abstraction:

$$\frac{[j] P :: \Gamma, x: A \vdash \Delta, j: B}{[j] (\lambda x. P) :: \Gamma \vdash \Delta, j: A \rightarrow B}$$

The substitution $\{j (\lambda x. Q) / j Q\}$ on variable x and co-variable j , for the term $[j] P$, gives the required $[j] (\lambda x. P)$. Next, application, where the derivation uses the interpretation of a join $\text{join } j x = N \text{ in } M$ as a double substitution $\{\{P/x\}N / j P\}M$:

$$\frac{[j] P :: \Gamma \vdash \Delta, j: A \rightarrow B \quad \frac{[i] Q :: \Gamma \vdash \Delta, i: A \quad \overline{[k] x :: x: B, \Gamma \vdash \Delta, k: B}^{\text{AX}}}{[k] (y Q) :: y: A \rightarrow B, \Gamma \vdash \Delta, k: B}^{\wedge L}}{[k] (P Q) :: \Gamma \vdash \Delta, k: B}^{\text{CUT}}$$

The $\wedge\text{L}$ rule instance interprets $\text{join } i \ z = [k] (y \ z) \text{ in } [i] \ Q$ as $[k] (y \ Q)$, and the CUT rule instance then interprets $\text{join } j \ y = [k] (y \ Q) \text{ in } [j] \ P$ as $[k] (P \ Q)$.

Note that *contraction* in $\lambda\mu$ -calculus is implicit with the μ -rule; hence, we need not worry about contraction on the top-level name $[j]$ in $[j] \ P :: \Gamma \vdash \Delta$.

Chapter 7

Reductions

In this chapter we focus on the reduction of terms of the join calculus. These reductions take two forms: a semantically driven extension of beta reduction which uses substitution to evaluate the term; and a proof reduction, which simplifies derivations when related terms are cut against each other.

The derivation reduction encounters some issues. The calculus contains both a contraction and co-contraction rule; a cut along the contracted variable/co-variable pair does not have a valid reduction. There is also a choice to be made when defining an outcome of a reduction. Terms which are equivalent according to our intuition have non equivalent expressions. We ascribe this to the intrinsic conflict between the algebraic expression and the parallel nature of the semantics, which leads non-determinism in the order of evaluation. To deal with this we introduce an equivalence relation which collapses the non-deterministically equivalent terms.

The final section of this chapter looks at permutation equivalence of terms. When independent rules can be applied in either order the type system admits these rules to be permuted. This equivalence is shown to combine with the other relations defined in the chapter to form a reduction system which is equivalent to standard β - η reduction on the interpretation defined in Section 6.4.

To further understand the semantics of reduction in this join calculus refer to the

abstract machine defined in Appendix A, where it is also shown that a run of such a machine is expressed by the reduction relation.

7.1 Beta Reduction Rules

In this section we define an extension of standard beta reduction of the lambda calculus to a reduction relation on join terms. This extension aligns with the semantics of the calculus given by the continuation interpretation in the previous chapter.

Definition 51 *Beta reduction* consists of the following rules, extended to all contexts:

$$\begin{aligned}
 (\lambda x. P)Q &\rightarrow_{\beta} \{Q/x\}P \\
 \pi_i(P_1, P_2) &\rightarrow_{\beta} P_i \\
 \text{case } (\iota_i L) \text{ of } (\iota_1 x_1 \rightarrow M_1, \iota_2 x_2 \rightarrow M_2) &\rightarrow_{\beta} \{L/x_i\}M_i \\
 \text{join } jx = N \text{ in } M &\rightarrow_{\beta} \{\{P/x\}N/jP\}M
 \end{aligned}$$

The first rule is the standard one for beta reduction, the next two deal with pairing and sums respectively while the final rule is specific to the join semantics, and shows the evaluation step for such terms. You will recall the final one as the rule mentioned when justifying the continuation encoding of such terms. Note that the first two rules apply to value terms, while the final two apply to computation terms, thus the contexts in which the redexes appear differ slightly.

The intuition of the join reduction can be seen via the following special cases, in which the join amounts to a renaming of co-variables/variables respectively:

$$\begin{aligned}
 \text{join } jx = kx \text{ in } M &\rightarrow_{\beta} \{kP/jP\}M \\
 \text{join } jx = N \text{ in } jy &\rightarrow_{\beta} \{y/x\}N
 \end{aligned}$$

7.2 Derivation Reduction

The standard method for reducing a sequent calculus is to identify grouping of related rules which can be replaced with a smaller group, here we consider three such before touching on why this alone is not a feasible strategy for term reduction. When two related terms are cut against each other it is possible to reduce the number of rules with the following *derivation reductions*:

$\rightarrow R / \rightarrow L$:

$$\text{join } kz = (\text{join } jw = \{zw/x\}N \text{ in } L) \text{ in } \{k(\lambda y. P)/iP\}M \\ \rightarrow_p \text{join } ix = N \text{ in } \text{join } jy = M \text{ in } L$$

$\wedge R / \wedge L$:

$$\text{join } kz = \{\pi_1 z/x\}\{\pi_2 z/y\}N \text{ in } \text{join } jx = (\text{join } iy = k(x, y) \text{ in } L) \text{ in } M \\ \rightarrow_p \text{join } jx = (\text{join } iy = N \text{ in } L) \text{ in } M$$

$\vee R / \vee L$:

$$\text{join } kz = (\text{case } z \text{ of } \iota_1 x \rightarrow L \ \iota_2 y \rightarrow N) \text{ in } \{k(\iota_1 P)/iP\}\{k(\iota_2 P)/jP\}M \\ \rightarrow_p \text{join } ix = L \text{ in } \text{join } jy = N \text{ in } M$$

The sequent derivations corresponding to these rules are shown in Figure 7-5.

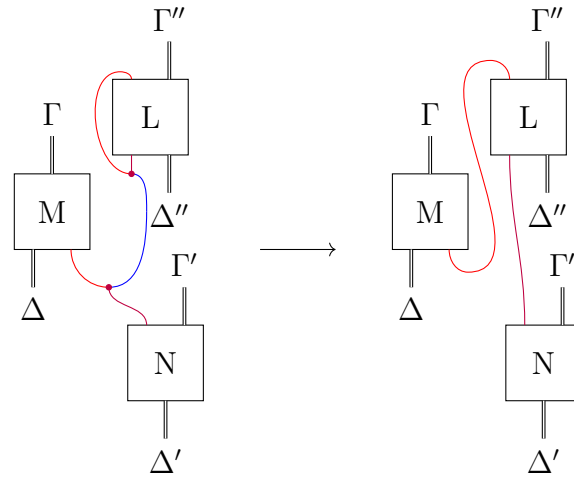
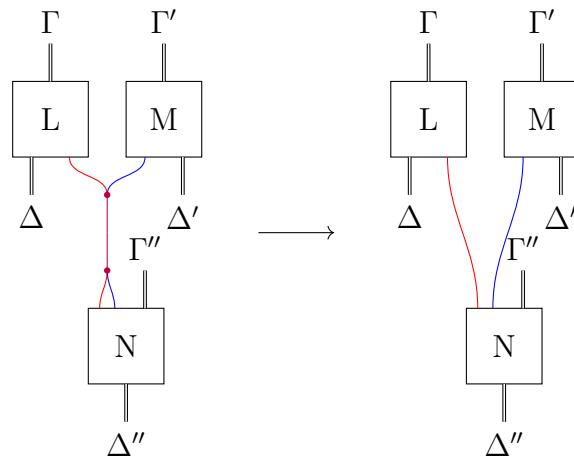
Note, these do not follow immediately from the beta rules, for instance these reductands are not obviously the same:

$$\text{join } kz = (\text{join } jw = \{zw/x\}N \text{ in } L) \text{ in } \{k(\lambda y. P)/iP\}M \\ \rightarrow_\beta \{\{\{\{Q/y\}P/x\}N/jQ\}L/iP\}M$$

$$\text{join } ix = N \text{ in } \text{join } jy = M \text{ in } L \rightarrow_\beta \{\{Q/y\}\{\{P/x\}N/iP\}M/jQ\}L$$

However, it does follow the intuition of the calculus.

The issue with this formulation is that we have contraction rules, which rename variables/co-variables without changing the intrinsic meaning of the term. When such terms are cut against each other removing the rules results in a non equivalent term.

Figure 7-1: $\rightarrow_L/\rightarrow_R$ ReductionFigure 7-2: $\wedge_L/\wedge_R$

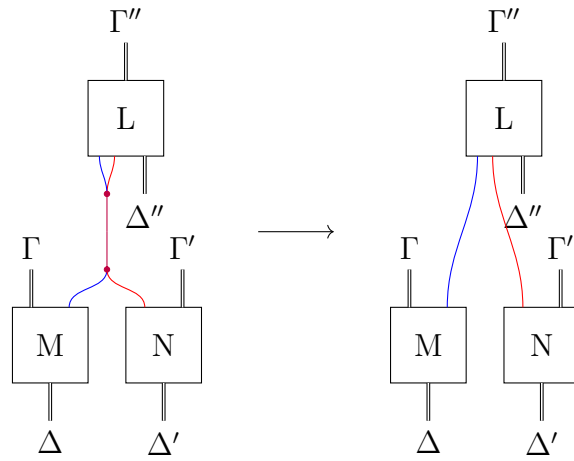
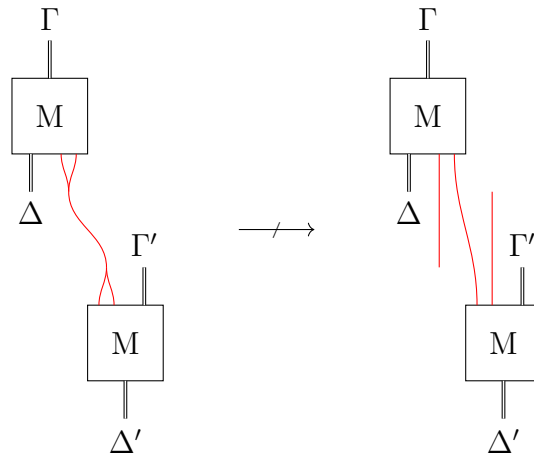
Figure 7-3: $\vee R/\vee L$ Reduction

Figure 7-4: CR/CL does not have a valid reduction

$$\begin{array}{c}
\rightarrow R / \rightarrow L \text{ reduction:} \\
\frac{M :: x : A, \Gamma \vdash \Delta, j : B \quad L :: \Gamma' \vdash \Delta', i : A \quad N :: y : B, \Gamma'' \vdash \Delta''}{\frac{\{j(\lambda x. P)/j P\} M :: \Gamma \vdash \Delta, j : A \rightarrow B \quad (\text{join } i z = \{y z/y\} N \text{ in } L) :: \Gamma', y : A \rightarrow B, \Gamma'' \vdash \Delta', \Delta''}{\text{join } j y = (\text{join } i z = \{y z/y\} N \text{ in } L) \text{ in } \{j(\lambda x. P)/j P\} M :: \Gamma, \Gamma', \Gamma'' \vdash \Delta, \Delta', \Delta''}} \rightarrow L \\
\frac{L :: \Gamma' \vdash \Delta', i : A \quad M :: x : A, \Gamma \vdash \Delta, j : B}{\text{join } i x = M \text{ in } L :: \Gamma, \Gamma' \vdash \Delta, \Delta', j : B} \text{CUT} \quad \frac{N :: y : B, \Gamma'' \vdash \Delta''}{\text{join } j y = N \text{ in join } i x = M \text{ in } L :: \Gamma, \Gamma', \Gamma'' \vdash \Delta, \Delta', \Delta''} \text{CUT} \\
\rightarrow_p \\
\wedge R / \wedge L \text{ reduction:} \\
\frac{L :: \Gamma \vdash \Delta, i : A \quad M :: \Gamma' \vdash \Delta', j : B \quad N :: x : A, y : B, \Gamma'' \vdash \Delta''}{\frac{\text{join } j y = k(x, y) \text{ in } M \text{ in } L :: \Gamma, \Gamma' \vdash \Delta, \Delta', k : A \wedge B \quad \{\pi_1 z/x\} \{\pi_2 z/y\} N :: z : A \wedge B, \Gamma'' \vdash \Delta''}{\text{join } k z = \{\pi_1 z/x\} \{\pi_2 z/y\} N \text{ in join } i x = (\text{join } j y = k(x, y) \text{ in } M) \text{ in } L :: \Gamma, \Gamma', \Gamma'' \vdash \Delta, \Delta', \Delta''}} \wedge L \\
\frac{M :: \Gamma' \vdash \Delta', j : B \quad N :: x : A, y : B, \Gamma'' \vdash \Delta''}{\text{join } j y = N \text{ in } M} \text{CUT} \\
\rightarrow_p \quad \frac{L :: \Gamma \vdash \Delta, i : A \quad \text{join } j y = N \text{ in } M :: x : A, \Gamma', \Gamma'' \vdash \Delta', \Delta''}{\text{join } i x = (\text{join } j y = N \text{ in } M) \text{ in } L :: \Gamma, \Gamma', \Gamma'' \vdash \Delta, \Delta', \Delta''} \text{CUT} \\
\vee R / \vee L \text{ reduction:} \\
\frac{L :: \Gamma \vdash \Delta, i : A, j : B \quad M :: x : A, \Gamma' \vdash \Delta' \quad N :: y : B, \Gamma'' \vdash \Delta''}{\frac{\{k(\iota_1 P)/i P\} \{k(\iota_2 P)/j P\} L :: \Gamma \vdash \Delta, k : A \vee B \quad \text{case } z \text{ of } (\iota_1 x \rightarrow M, \iota_2 y \rightarrow N) :: z : A \vee B, \Gamma', \Gamma'' \vdash \Delta', \Delta''}{\text{join } k z = \text{case } z \text{ of } (\iota_1 x \rightarrow M, \iota_2 y \rightarrow N) \text{ in } \{k(\iota_1 P)/i P\} \{k(\iota_2 P)/j P\} L :: \Gamma, \Gamma', \Gamma'' \vdash \Delta, \Delta', \Delta''}} \vee R \\
\frac{L :: \Gamma \vdash \Delta, i : A, j : B \quad M :: x : A, \Gamma' \vdash \Delta'}{\text{join } i x = M \text{ in } L :: \Gamma, \Gamma' \vdash \Delta, \Delta', j : B} \text{CUT} \quad \frac{N :: y : B, \Gamma'' \vdash \Delta''}{\text{join } j y = N \text{ in join } i x = M \text{ in } L :: \Gamma, \Gamma', \Gamma'' \vdash \Delta, \Delta', \Delta''} \text{CUT} \\
\rightarrow_p
\end{array}$$

Figure 7-5: Admissibility of the permutation reductions

7.3 Non-Determinism

When two separate values are required for computation any encoding will require that one is computed before the other. The order of these computations introduces an aspect of nondeterminism.

Consider the following derivations:

$$\frac{M :: \Gamma \vdash \Delta, j : A \quad \frac{N :: \Gamma' \vdash \Delta', i : B \quad L :: x : A, y : B, \Gamma'' \vdash \Delta''}{\text{join } i y = L \text{ in } N :: x : A, \Gamma', \Gamma'' \vdash \Delta', \Delta''} \text{CUT}}{\text{join } j x = (\text{join } i y = L \text{ in } N) \text{ in } M :: \Gamma, \Gamma', \Gamma'' \vdash \Delta, \Delta', \Delta''} \text{CUT}$$

$$\frac{N :: \Gamma' \vdash \Delta', i : B \quad \frac{M :: \Gamma \vdash \Delta, j : A \quad L :: x : A, y : B, \Gamma'' \vdash \Delta''}{\text{join } j x = L \text{ in } M :: y : B, \Gamma, \Gamma'' \vdash \Delta, \Delta''} \text{CUT}}{\text{join } i y = (\text{join } j x = L \text{ in } M) \text{ in } N :: \Gamma, \Gamma', \Gamma'' \vdash \Delta, \Delta', \Delta''} \text{CUT}$$

All of our intuition tells us that these are the same terms (alternative outcomes of an $\wedge L/\wedge R$ reduction). The computation involves an evaluation of M and N and, provided they result in the correct outcomes both are fed as variables to L . However the encoding does not reflect this:

$$\begin{aligned} \llbracket \text{join } j x = (\text{join } i y = L \text{ in } N) \text{ in } M \rrbracket &= (\lambda j. M)(\lambda x. (\lambda i. N)(\lambda y. L)) \\ &\rightarrow_{\beta} \{\lambda x. \{\lambda y. L/i\}N/j\}M \\ \llbracket \text{join } i y = (\text{join } j x = L \text{ in } M) \text{ in } N \rrbracket &= (\lambda i. N)(\lambda y. (\lambda j. M)(\lambda x. L)) \\ &\rightarrow_{\beta} \{\lambda y. \{\lambda x. L/j\}M/i\}N \end{aligned}$$

We thus introduce this equivalence directly to give a non-determinism interpretation.

Definition 52 The following *non-determinism equivalence* relation holds if $i, j \notin \text{fc}(L)$ and $x, y \notin \text{fv}(M) \cup \text{fv}(N)$:

$$C\{\text{join } j x = (\text{join } i y = L \text{ in } N) \text{ in } M\} =_{nd} C\{\text{join } i y = (\text{join } j x = L \text{ in } M) \text{ in } N\}$$

Note that this definition covers the cases where **CUT** is replaced by $\rightarrow_L$ as the free variable condition is preserved by the fresh variable substitutions. As in the following cases:

$$\begin{array}{c}
\frac{N :: \Gamma' \vdash \Delta', i : B \quad L :: x : A, y : C, \Gamma'' \vdash \Delta''}{M :: \Gamma \vdash \Delta, j : A \quad \text{join } i w = \{yw/y\} L \text{ in } N :: x : A, y : B \rightarrow C, \Gamma', \Gamma'' \vdash \Delta', \Delta''} \rightarrow_L \\
\frac{}{\text{join } j x = (\text{join } i w = \{yw/y\} L \text{ in } N) \text{ in } M :: \Gamma, y : B \rightarrow C, \Gamma', \Gamma'' \vdash \Delta, \Delta', \Delta''} \text{CUT} \\
=_{nd} \\
\frac{M :: \Gamma \vdash \Delta, j : A \quad L :: x : A, y : C, \Gamma'' \vdash \Delta''}{N :: \Gamma' \vdash \Delta', i : B \quad \text{join } j x = L \text{ in } M :: y : C, \Gamma, \Gamma'' \vdash \Delta, \Delta''} \text{CUT} \\
\frac{}{\text{join } i w = \{yw/y\} (\text{join } j x = L \text{ in } M) \text{ in } N :: y : B \rightarrow C, \Gamma, \Gamma', \Gamma'' \vdash \Delta, \Delta', \Delta''} \rightarrow_L \\
\hline
\frac{N :: \Gamma' \vdash \Delta', i : B \quad L :: x : C, y : D, \Gamma'' \vdash \Delta''}{M :: \Gamma \vdash \Delta, j : A \quad \text{join } i w = \{yw/y\} L \text{ in } N :: x : C, y : B \rightarrow D, \Gamma', \Gamma'' \vdash \Delta', \Delta''} \rightarrow_L \\
\frac{}{\text{join } j z = \{xz/x\} (\text{join } i w = \{yw/y\} L \text{ in } N) \text{ in } M :: x : A \rightarrow C, y : B \rightarrow D, \Gamma, \Gamma', \Gamma'' \vdash \Delta, \Delta', \Delta''} \rightarrow_L \\
=_{nd} \\
\frac{M :: \Gamma \vdash \Delta, j : A \quad L :: x : C, y : D, \Gamma'' \vdash \Delta''}{N :: \Gamma' \vdash \Delta', i : B \quad \text{join } j z = \{xz/x\} L \text{ in } M :: x : A \rightarrow C, y : B, \Gamma, \Gamma'' \vdash \Delta, \Delta''} \rightarrow_L \\
\frac{}{\text{join } i w = \{yw/y\} (\text{join } j z = \{xz/x\} L \text{ in } M) \text{ in } N :: x : A \rightarrow C, y : B \rightarrow D, \Gamma, \Gamma', \Gamma'' \vdash \Delta, \Delta', \Delta''} \rightarrow_L
\end{array}$$

This definition also covers the two ways of defining the pairing operation, the free (co)variable condition coming for free from the fresh variables used to define pairing:

$$\text{join } j x = (\text{join } i y = k(x, y) \text{ in } L) \text{ in } M =_{nd} \text{join } i y = (\text{join } j x = k(x, y) \text{ in } M) \text{ in } L$$

Since we will be working modulo this rule for the remainder of the chapter we adopt the following shorthand for the pairing relation:

$$M - j \times_k i - L = \text{join } i y = (\text{join } j x = k(x, y) \text{ in } M) \text{ in } L$$

7.4 Permutations

As noted previously the sequent calculus is not suitable for cut elimination, due to the contraction rules. Instead we consider the changes which can be made to

Definition 53 We say two terms are *permutation equivalent* when two consecutive sequent rules, acting on different formulae in the sequent, can be applied in either order. This may require the duplication of one rule to act on multiple branches.

Theorem 54 *For any sequent derivations which are equivalent under permutations:*

the join terms are beta-eta-nd-equivalent: $M =_{nd} N$ or $\llbracket M \rrbracket =_{\beta} \llbracket N \rrbracket$

[illegible]

1. These permutations have one case, where both terms are equal by independent substitutions. For example, the permutation $\rightarrow R/\rightarrow R$ gives the following.

$$\{j(\lambda x. P)/j P\}\{i(\lambda x. Q)/i Q\}M = \{i(\lambda x. Q)/i Q\}\{j(\lambda x. P)/j P\}M$$

2. These permutations have two cases, and in each case both terms are equal by substitution. For example, the permutation $\rightarrow R/\rightarrow L$ gives the following two cases.

$$\begin{aligned} \{j(\lambda x. P)/j P\} \text{join } i z = \{y z/y\}N \text{ in } M &= \\ \begin{cases} \text{join } i z = \{y z/y\}\{j(\lambda x. P)/j P\}N \text{ in } M & \text{if } j \notin \text{fc}(M), x \notin \text{fv}(M) \\ \text{join } i z = \{y z/y\}N \text{ in } \{j(\lambda x. P)/j P\}M & \text{if } j \notin \text{fc}(N), x \notin \text{fv}(N) \end{cases} \end{aligned}$$

For each permutation case, there are substitutions for two variables, two co-variables, or as above, a variable and a co-variable. The permutation is only possible if both (co)variables occur in the same subterm; otherwise the two inferences in the sequent proof may not permute.

3. There are three CUT/CUT permutations. They are given in Figure 7-6 and correspond directly to equivalences on terms. $\rightarrow L/\rightarrow L$ (Figure 7-7) is analogous, since $\rightarrow L$ has the same prerequisites as cut.

The first case of Figure 7-6 is covered by the non-determinism relation. Considering the second case: the occurrence of free variables simplifies the substitutions so that the two terms beta reduce as follows:

$$\begin{aligned} \llbracket \text{join } j x = (\text{join } i y = L \text{ in } N) \text{ in } M \rrbracket &= (\lambda j. M)(\lambda x. (\lambda i. N)(\lambda y. L)) \\ &\rightarrow_{\beta} \{\lambda x. \{\lambda y. L/i\}N/j\}M \\ &= \{\lambda y. L/i\}\{\lambda x. N/j\}M \\ &[\text{As } x \notin \text{fv}(L), i \notin \text{fc}(M)] \end{aligned}$$

$$\begin{aligned} \llbracket \text{join } i y = L \text{ in join } j x = N \text{ in } M \rrbracket &= (\lambda i. (\lambda j. M)(\lambda x. N))(\lambda y. L) \\ &\rightarrow_{\beta} \{\lambda y. L/i\}\{\lambda x. N/j\}M \end{aligned}$$

4. These permutations have four cases, as sum and product each occur on a single side of the sequent there is no interaction, so all permutations are possible when there is no variable linking.
5. These permutations have two cases. Sum and product do not interfere with themselves. This technically creates four cases, however, they are in equivalent pairs, so the number is halved.
6. There are four $\wedge R/\vee L$ cases: Figure 7-9.
7. Four $CUT/\rightarrow L$ cases (Figure 7-8), similar to 3, but without the symmetric cases collapsed.

7.5 Omissions

There are still several important things to be checked about this calculus. Firstly we need to check the consistency of the calculus under non-deterministic equivalence: namely that the relation does not go so far as to make all proofs of a given formula equivalent. Secondly we have not given much space to exploring the semantics of this system, or its connections with similar systems in the literature.

However these omissions are not likely to represent insurmountable problems with the calculus as presented. At worst the non-deterministic equivalence results in a trivial system. This would show that we have an intrinsically asymmetric product. A state of affairs which would put us in line with *closed Freyd-categories* [60] rather than cartesian closed categories. In this scenario terms still reduce to normal form, since the nd-equation affects only the permutations, not the cut-elimination steps. The probabilistic formulation, which we focus on in the next chapter, is intrinsically asymmetric, evading these concerns.

$$\begin{array}{c}
\frac{M :: \Gamma \vdash \Delta, j : A \quad \frac{N :: \Gamma' \vdash \Delta', i : B \quad L :: x : A, y : B, \Gamma'' \vdash \Delta''}{\text{join } i y = L \text{ in } N :: x : A, \Gamma', \Gamma'' \vdash \Delta', \Delta''} \text{CUT}}{\text{join } j x = (\text{join } i y = L \text{ in } N) \text{ in } M :: \Gamma, \Gamma', \Gamma'' \vdash \Delta, \Delta', \Delta''} \text{CUT} \\
\sim \\
\frac{N :: \Gamma' \vdash \Delta', i : B \quad \frac{M :: \Gamma \vdash \Delta, j : A \quad L :: x : A, y : B, \Gamma'' \vdash \Delta''}{\text{join } j x = L \text{ in } M :: y : B, \Gamma, \Gamma'' \vdash \Delta, \Delta''} \text{CUT}}{\text{join } i y = (\text{join } j x = L \text{ in } M) \text{ in } N :: \Gamma, \Gamma', \Gamma'' \vdash \Delta, \Delta', \Delta''} \text{CUT} \\
\hline
\frac{M :: \Gamma \vdash \Delta, j : A \quad \frac{N :: x : A, \Gamma' \vdash \Delta', i : B \quad L :: y : B, \Gamma'' \vdash \Delta''}{\text{join } i y = L \text{ in } N :: x : A, \Gamma', \Gamma'' \vdash \Delta', \Delta''} \text{CUT}}{\text{join } j x = (\text{join } i y = L \text{ in } N) \text{ in } M :: \Gamma, \Gamma', \Gamma'' \vdash \Delta, \Delta', \Delta''} \text{CUT} \\
\sim \\
\frac{M :: \Gamma \vdash \Delta, j : A \quad N :: x : A, \Gamma' \vdash \Delta', i : B}{\text{join } j x = N \text{ in } M :: \Gamma, \Gamma' \vdash \Delta, \Delta', i : B} \text{CUT} \quad L :: y : B, \Gamma'' \vdash \Delta''}{\text{join } i y = L \text{ in join } j x = N \text{ in } M :: \Gamma, \Gamma', \Gamma'' \vdash \Delta, \Delta', \Delta''} \text{CUT} \\
\hline
\frac{M :: \Gamma \vdash \Delta, j : A, i : B \quad L :: y : B, \Gamma'' \vdash \Delta''}{\text{join } i y = L \text{ in } M :: \Gamma, \Gamma'' \vdash \Delta, \Delta'', j : A} \text{CUT} \quad N :: x : A, \Gamma' \vdash \Delta'}{\text{join } j x = N \text{ in join } i y = L \text{ in } M :: \Gamma, \Gamma', \Gamma'' \vdash \Delta, \Delta', \Delta''} \text{CUT} \\
\sim \\
\frac{M :: \Gamma \vdash \Delta, j : A, i : B \quad N :: x : A, \Gamma' \vdash \Delta'}{\text{join } j x = N \text{ in } M :: \Gamma, \Gamma' \vdash \Delta, \Delta', i : B} \text{CUT} \quad L :: y : B, \Gamma'' \vdash \Delta''}{\text{join } i y = L \text{ in join } j x = N \text{ in } M :: \Gamma, \Gamma', \Gamma'' \vdash \Delta, \Delta', \Delta''} \text{CUT}
\end{array}$$

Figure 7-6: The three CUT/CUT permutations

$$\begin{array}{c}
\frac{N :: \Gamma' \vdash \Delta', i : B \quad L :: x : C, y : D, \Gamma'' \vdash \Delta''}{\frac{M :: \Gamma \vdash \Delta, j : A \quad \text{join } i w = \{yw/y\} L \text{ in } N :: x : C, y : B \rightarrow D, \Gamma', \Gamma'' \vdash \Delta', \Delta''}{\text{join } j z = \{xz/x\} (\text{join } i w = \{yw/y\} L \text{ in } N) \text{ in } M :: x : A \rightarrow C, y : B \rightarrow D, \Gamma, \Gamma', \Gamma'' \vdash \Delta, \Delta', \Delta''} \rightarrow L} \sim \\
\frac{M :: \Gamma \vdash \Delta, j : A \quad L :: x : C, y : D, \Gamma'' \vdash \Delta''}{\frac{N :: \Gamma' \vdash \Delta', i : B \quad \text{join } j z = \{xz/x\} L \text{ in } M :: x : A \rightarrow C, y : B, \Gamma, \Gamma'' \vdash \Delta, \Delta''}{\text{join } i w = \{yw/y\} (\text{join } j z = \{xz/x\} L \text{ in } M) \text{ in } N :: x : A \rightarrow C, y : B \rightarrow D, \Gamma, \Gamma', \Gamma'' \vdash \Delta, \Delta', \Delta''} \rightarrow L} \sim \\
\frac{N :: x : B, \Gamma' \vdash \Delta', i : C \quad L :: y : D, \Gamma'' \vdash \Delta''}{\frac{M :: \Gamma \vdash \Delta, j : A \quad \text{join } i w = \{yw/y\} L \text{ in } N :: x : B, y : C \rightarrow D, \Gamma', \Gamma'' \vdash \Delta', \Delta''}{\text{join } j z = \{xz/x\} (\text{join } i w = \{yw/y\} L \text{ in } N) \text{ in } M :: x : A \rightarrow B, y : C \rightarrow D, \Gamma, \Gamma', \Gamma'' \vdash \Delta, \Delta', \Delta''} \rightarrow L} \sim \\
\frac{M :: \Gamma \vdash \Delta, j : A \quad N :: x : B, \Gamma' \vdash \Delta', i : C}{\frac{\text{join } j z = \{xz/x\} N \text{ in } M :: x : A \rightarrow B, \Gamma, \Gamma' \vdash \Delta, \Delta', i : C \quad L :: y : D, \Gamma'' \vdash \Delta''}{\text{join } i w = \{yw/y\} L \text{ in } \text{join } j z = \{xz/x\} N \text{ in } M :: x : A \rightarrow B, y : C \rightarrow D, \Gamma, \Gamma', \Gamma'' \vdash \Delta, \Delta', \Delta''} \rightarrow L} \sim \\
\frac{M :: \Gamma \vdash \Delta, j : A, i : B \quad L :: y : C, \Gamma'' \vdash \Delta''}{\frac{\text{join } i w = \{yw/y\} L \text{ in } M :: y : B \rightarrow C, \Gamma, \Gamma'' \vdash \Delta, \Delta'', j : A \quad N :: x : D, \Gamma' \vdash \Delta'}{\text{join } j z = \{xz/x\} N \text{ in } \text{join } i w = \{yw/y\} L \text{ in } M :: x : A \rightarrow D, y : B \rightarrow C, \Gamma, \Gamma', \Gamma'' \vdash \Delta, \Delta', \Delta''} \rightarrow L} \sim \\
\frac{M :: \Gamma \vdash \Delta, j : A, i : B \quad N :: x : D, \Gamma' \vdash \Delta'}{\frac{\text{join } j z = \{xz/x\} N \text{ in } M :: x : A \rightarrow D, \Gamma, \Gamma' \vdash \Delta, \Delta', i : B \quad L :: y : C, \Gamma'' \vdash \Delta''}{\text{join } i w = \{yw/y\} L \text{ in } \text{join } j z = \{xz/x\} N \text{ in } M :: x : A \rightarrow D, y : B \rightarrow C, \Gamma, \Gamma', \Gamma'' \vdash \Delta, \Delta', \Delta''} \rightarrow L} \sim
\end{array}$$

Figure 7-7: The three $\rightarrow L / \rightarrow L$ permutations

$$\begin{array}{c}
\frac{M :: \Gamma \vdash \Delta, j: A \quad \frac{N :: \Gamma' \vdash \Delta', i: B \quad L :: x: A, y: C, \Gamma'' \vdash \Delta''}{\text{join } i w = \{yw/y\}L \text{ in } N :: x: A, y: B \rightarrow C, \Gamma', \Gamma'' \vdash \Delta', \Delta''} \rightarrow^L}{\text{join } j x = (\text{join } i w = \{yw/y\}L \text{ in } N) \text{ in } M :: \Gamma, y: B \rightarrow C, \Gamma', \Gamma'' \vdash \Delta, \Delta', \Delta''} \text{CUT} \\
\sim \\
\frac{N :: \Gamma' \vdash \Delta', i: B \quad \frac{M :: \Gamma \vdash \Delta, j: A \quad L :: x: A, y: C, \Gamma'' \vdash \Delta''}{\text{join } j x = L \text{ in } M :: y: C, \Gamma, \Gamma'' \vdash \Delta, \Delta''} \text{CUT}}{\text{join } i w = \{yw/y\}(\text{join } j x = L \text{ in } M) \text{ in } N :: y: B \rightarrow C, \Gamma, \Gamma', \Gamma'' \vdash \Delta, \Delta', \Delta''} \rightarrow^L \\
\hline
\frac{M :: \Gamma \vdash \Delta, j: A \quad \frac{N :: x: A, \Gamma' \vdash \Delta', i: B \quad L :: y: C, \Gamma'' \vdash \Delta''}{\text{join } i w = \{yw/y\}L \text{ in } N :: x: A, y: B \rightarrow C, \Gamma', \Gamma'' \vdash \Delta', \Delta''} \rightarrow^L}{\text{join } j x = (\text{join } i w = \{yw/y\}L \text{ in } N) \text{ in } M :: y: B \rightarrow C, \Gamma, \Gamma', \Gamma'' \vdash \Delta, \Delta', \Delta''} \text{CUT} \\
\sim \\
\frac{M :: \Gamma \vdash \Delta, j: A \quad N :: x: A, \Gamma' \vdash \Delta', i: B}{\text{join } j x = N \text{ in } M :: \Gamma, \Gamma' \vdash \Delta, \Delta', i: B} \text{CUT} \quad L :: y: C, \Gamma'' \vdash \Delta''}{\text{join } i w = \{yw/y\}L \text{ in } \text{join } j x = N \text{ in } M :: y: B \rightarrow C, \Gamma, \Gamma', \Gamma'' \vdash \Delta, \Delta', \Delta''} \rightarrow^L \\
\hline
\frac{M :: \Gamma \vdash \Delta, i: B \quad L :: y: C, \Gamma' \vdash \Delta', j: A}{\text{join } i w = \{yw/y\}L \text{ in } M :: y: B \rightarrow C, \Gamma, \Gamma' \vdash \Delta, \Delta', j: A} \rightarrow^L \quad N :: x: A, \Gamma'' \vdash \Delta''}{\text{join } j x = N \text{ in } \text{join } i w = \{yw/y\}L \text{ in } M :: y: B \rightarrow C, \Gamma, \Gamma', \Gamma'' \vdash \Delta, \Delta', \Delta''} \text{CUT} \\
\sim \\
\frac{L :: y: C, \Gamma' \vdash \Delta', j: A \quad N :: x: A, \Gamma'' \vdash \Delta''}{\text{join } j x = N \text{ in } L :: y: C, \Gamma', \Gamma'' \vdash \Delta', \Delta''} \text{CUT} \quad M :: \Gamma \vdash \Delta, i: B}{\text{join } i w = \{yw/y\}(\text{join } j x = N \text{ in } L) \text{ in } M :: y: B \rightarrow C, \Gamma, \Gamma', \Gamma'' \vdash \Delta, \Delta', \Delta''} \rightarrow^L \\
\hline
\frac{M :: \Gamma \vdash \Delta, j: A, i: B \quad L :: y: C, \Gamma' \vdash \Delta'}{\text{join } i w = \{yw/y\}L \text{ in } M :: y: B \rightarrow C, \Gamma, \Gamma' \vdash \Delta, \Delta', j: A} \rightarrow^L \quad N :: x: A, \Gamma'' \vdash \Delta''}{\text{join } j x = N \text{ in } \text{join } i w = \{yw/y\}L \text{ in } M :: y: B \rightarrow C, \Gamma, \Gamma', \Gamma'' \vdash \Delta, \Delta', \Delta''} \text{CUT} \\
\sim \\
\frac{M :: \Gamma \vdash \Delta, j: A, i: B \quad N :: x: A, \Gamma'' \vdash \Delta''}{\text{join } j x = N \text{ in } M :: \Gamma, \Gamma'' \vdash \Delta, \Delta'', i: B} \text{CUT} \quad L :: y: C, \Gamma' \vdash \Delta'}{\text{join } i w = \{yw/y\}L \text{ in } \text{join } j x = N \text{ in } M :: y: B \rightarrow C, \Gamma, \Gamma', \Gamma'' \vdash \Delta, \Delta', \Delta''} \rightarrow^L
\end{array}$$

Figure 7-8: The four CUT/ $\rightarrow^L$ permutations

$$\begin{array}{c}
\frac{M :: x : A, \Gamma \vdash \Delta \quad N :: y : B, \Gamma' \vdash \Delta', i : C}{\text{case } z \text{ of } (\iota_1 x \rightarrow M, \iota_2 y \rightarrow N) :: z : A \vee B, \Gamma, \Gamma' \vdash \Delta, \Delta', i : C}^{\vee L} \quad \frac{L :: \Gamma'' \vdash \Delta'', j : D}{\text{case } z \text{ of } (\iota_1 x \rightarrow M, \iota_2 y \rightarrow N) - i \times_k j - L :: z : A \vee B, \Gamma, \Gamma', \Gamma'' \vdash \Delta, \Delta', \Delta'', k : C \wedge D}^{\wedge R} \\
\sim \\
\frac{M :: x : A, \Gamma \vdash \Delta \quad \frac{N :: y : B, \Gamma' \vdash \Delta', i : C \quad L :: \Gamma'' \vdash \Delta'', j : D}{N - i \times_k j - L :: y : B, \Gamma', \Gamma'' \vdash \Delta', \Delta'', k : C \wedge D}^{\wedge R}}{\text{case } z \text{ of } (\iota_1 x \rightarrow M, \iota_2 y \rightarrow N - i \times_k j - L) :: z : A \vee B, \Gamma, \Gamma', \Gamma'' \vdash \Delta, \Delta', \Delta'', k : C \wedge D}^{\vee L} \\
\hline
\frac{M :: x : A, \Gamma \vdash \Delta, i : C \quad N :: y : B, \Gamma' \vdash \Delta'}{\text{case } z \text{ of } (\iota_1 x \rightarrow M, \iota_2 y \rightarrow N) :: z : A \vee B, \Gamma, \Gamma' \vdash \Delta, \Delta', i : C}^{\vee L} \quad \frac{L :: \Gamma'' \vdash \Delta'', j : D}{\text{case } z \text{ of } (\iota_1 x \rightarrow M, \iota_2 y \rightarrow N) - i \times_k j - L :: z : A \vee B, \Gamma, \Gamma', \Gamma'' \vdash \Delta, \Delta', \Delta'', k : C \wedge D}^{\wedge R} \\
\sim \\
\frac{M :: x : A, \Gamma' \vdash \Delta', i : C \quad L :: \Gamma'' \vdash \Delta'', j : D}{M - i \times_k j - L :: x : A, \Gamma', \Gamma'' \vdash \Delta', \Delta'', k : C \wedge D}^{\wedge R} \quad \frac{N :: y : B, \Gamma \vdash \Delta}{\text{case } z \text{ of } (\iota_1 x \rightarrow M - i \times_k j - L, \iota_2 y \rightarrow N) :: z : A \vee B, \Gamma, \Gamma', \Gamma'' \vdash \Delta, \Delta', \Delta'', k : C \wedge D}^{\vee L} \\
\hline
\frac{N :: x : B, \Gamma' \vdash \Delta' \quad L :: y : C, \Gamma'' \vdash \Delta'', j : D}{\text{case } z \text{ of } (\iota_1 x \rightarrow N, \iota_2 y \rightarrow L) :: z : B \vee C, \Gamma', \Gamma'' \vdash \Delta', \Delta'', j : D}^{\vee L} \quad \frac{M :: \Gamma \vdash \Delta, i : A}{M - i \times_k j - (\text{case } z \text{ of } (\iota_1 x \rightarrow N, \iota_2 y \rightarrow L) :: z : B \vee C, \Gamma, \Gamma', \Gamma'' \vdash \Delta, \Delta', \Delta'', k : A \wedge D)}^{\wedge R} \\
\sim \\
\frac{M :: \Gamma \vdash \Delta, i : A \quad L :: y : C, \Gamma'' \vdash \Delta'', j : D}{M - i \times_k j - L :: y : C, \Gamma, \Gamma'' \vdash \Delta, \Delta'', k : A \wedge D}^{\wedge R} \quad \frac{N :: x : B, \Gamma' \vdash \Delta' \quad M - i \times_k j - L :: y : C, \Gamma, \Gamma'' \vdash \Delta, \Delta'', k : A \wedge D}{\text{case } z \text{ of } (\iota_1 x \rightarrow N, \iota_2 y \rightarrow M - i \times_k j - L) :: z : B \vee C, \Gamma, \Gamma', \Gamma'' \vdash \Delta, \Delta', \Delta'', k : A \wedge D}^{\vee L} \\
\hline
\frac{N :: x : B, \Gamma' \vdash \Delta', j : C \quad L :: y : D, \Gamma'' \vdash \Delta''}{\text{case } z \text{ of } (\iota_1 x \rightarrow N, \iota_2 y \rightarrow L) :: z : B \vee D, \Gamma', \Gamma'' \vdash \Delta', \Delta'', j : C}^{\vee L} \quad \frac{M :: \Gamma \vdash \Delta, i : A}{M - i \times_k j - \text{case } z \text{ of } (\iota_1 x \rightarrow N, \iota_2 y \rightarrow L :: z : B \vee D, \Gamma, \Gamma', \Gamma'' \vdash \Delta, \Delta', \Delta'', k : A \wedge C)}^{\wedge R} \\
\sim \\
\frac{M :: \Gamma \vdash \Delta, i : A \quad N :: x : B, \Gamma' \vdash \Delta', j : C}{M - i \times_k j - N :: x : A, \Gamma, \Gamma' \vdash \Delta, \Delta', k : A \wedge C}^{\wedge R} \quad \frac{L :: y : D, \Gamma'' \vdash \Delta''}{\text{case } z \text{ of } (\iota_1 x \rightarrow M - i \times_k j - N, \iota_2 y \rightarrow L) :: z : B \vee D, \Gamma, \Gamma', \Gamma'' \vdash \Delta, \Delta', \Delta'', k : A \wedge C}^{\vee L}
\end{array}$$

Figure 7-9: The four $\vee L/\wedge R$ permutations

Chapter 8

Towards Almost-Sure Termination

The final contribution of this thesis will be to introduce a *probabilistic* sequent system in which the multiple conclusions are a probability distribution rather than a disjunction/sum. This chapter aims to draw together the threads of reasoning outlined thus far, however, the main result remains a conjecture: that terms in this system, which is a lambda-calculus with loops, are almost surely terminating.

For the past two chapters we have worked with a non-recursive fragment of the join calculus. Normalisation in this fragment follows from its embedding in a natural deduction system. We now reintroduce the co-recursive version of join as a distinct construct in order to study looping in probabilistic programs. We then define a probabilistic type system for the recursive join calculus, similar to the ones studied in Part I. Finally we hypothesise that this type system gives rise to an almost-sure termination result for a notion of probabilistic reduction on the potentially recursive terms of the full join-calculus.

8.1 Co-recursion

The motivation which lead us to work with the join calculus was tackling the termination of loops in a probabilistic setting. The ability to model loops is an intrinsic feature of continuations. So far we have seen continuations, in the form of co-variables, used to indicate forward jumps (or goto statements), this is the non-recursive variant of the join construct. In the original formulation join is potentially recursive: the continuation may redirect to an earlier part of the program. This is used in [52] to model while loops and optimise let statements.

In our formulation we stick with a non-recursive join and obtain the co-recursive version of the calculus by extending the computation terms to include a *corecursor* $\text{corec } j \ x \ M$ that binds j in M . Whereas in the join construct j is bound forwards to the next stage of the computation, corec binds j in a controlled backward jump to an earlier part of the computation, forming a loop.

The loop construct is a syntactic interpretation of the standard semantics for iteration, which is dual to recursion. Thus when considering a type system for such a loop it is useful to look first at the typing of recursion. A recursor is given by a fixed point operator, the standard typing rule [67] for which is as follows:

$$\frac{A \times B \rightarrow A}{B \rightarrow A}$$

A loop, in contrast, is given by a tail-recursor, which has the dual type [9]:

$$\frac{A \rightarrow B + A}{A \rightarrow B}$$

Note that the inclusion of this rule requires us to work in a type system with summation, in other words, an intrinsically multi-conclusion system. Converting to a sequent system we want a co-recursor of type:

$$\frac{A, \Gamma \vdash \Delta, A}{A, \Gamma \vdash \Delta}$$

The main rule we consider to study co-recursion is the annotated version of this, shown below:

$$\frac{M :: x : A, \Gamma \vdash \Delta, j : A}{\text{corec } j \ x \ M :: x : A, \Gamma \vdash \Delta} \text{COREC}$$

Since co-variables always occur in function position, they syntactically enforce that the co-recursor is tail-recursive.

The corec behaves as an infinite join in which a particular branch of outcomes always feeds back into the original computation.

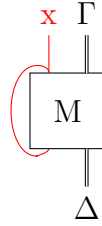


Figure 8-1: COREC

In the form stated in the last chapter the join reduction cannot be applied to co-recursive terms (since it eliminates the free x required in the loop). To deal with this the first iteration of the the loop must be unrolled as follows:

$$\text{corec } j \ x \ M \rightarrow \text{join } j \ x = (\text{corec } j \ x \ M) \text{ in } M$$

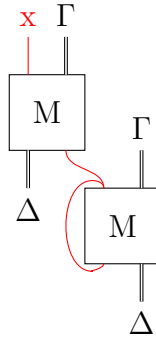


Figure 8-2: Unrolling COREC

If we view a step of the co-recursion as having two outcomes, one corresponding to success and termination and the other corresponding to failure and repeated

computation it is clear that on each iteration the failure component contributes a vanishingly small proportion of the outcomes. In a purely non-deterministic setting having a singular nonterminating run classes a term as non terminating, however, in a probabilistic setting we can work modulo events which happen on a null set of the probability space. If the conclusion of the sequent is considered as a distribution, the corec rule should provide almost-sure termination.

8.2 Type System

In this section we adapt the join-calculus to probabilistic computation by making sums $A \vee B$ and consequents Δ into probability distributions: $[p \cdot A, q \cdot B]$ and $\Delta = p_1 \cdot A_1, \dots, p_k \cdot B_k$ respectively, where p and q are probabilities that sum to one, and likewise the p_i .

Definition 55 *Distribution types* are given by the following grammar:

$$A, B ::= \perp \mid \top \mid A \rightarrow B \mid A \wedge B \mid [p \cdot A, (1 - p) \cdot B]$$

Definition 56 The *probabilistic type system* for the join-calculus is given by the rules in Figure 8-3.

A probabilistic sequent of the form $M :: \Gamma \vdash i : p \cdot B, \Delta$ is taken to mean that when M is given inputs of the types and distribution of the context Γ the outcome i of type B occurs with probability p . Each of the typing rules obeys this intuition, following the rules of probability under the assumption that sub-terms of a program are independent and outcomes mutually exclusive. Examining the rules it is possible to see that each preserves the conclusion as a distribution of types.

Probabilistic choices such as we studied in the PEA arise in this calculus as a result of the case rule. In this variant the variable argument of case exists according to a distribution of outcomes. The subsequent computation is dependant on the evaluation of this variable or, equivalently, sampling from the distribution.

$$\begin{array}{c}
\frac{}{jx :: \Gamma, x: A \vdash j: 1 \cdot A} \text{AX} \\
\\
\frac{M :: \Gamma \vdash \Delta, j: p \cdot A \quad N :: x: A, \Gamma' \vdash \Delta'}{\text{join } jx = N \text{ in } M :: \Gamma, \Gamma' \vdash \Delta, p \cdot \Delta'} \text{CUT} \\
\\
\frac{M :: \Gamma, x: A \vdash i: p \cdot B, \Delta}{\{j(\lambda x. P)/iP\}M :: \Gamma \vdash j: p \cdot A \rightarrow B, \Delta} \rightarrow R \quad (\text{no } x \text{ remains free}) \\
\\
\frac{M :: \Gamma \vdash \Delta, j: p \cdot A \quad N :: x: B, \Gamma' \vdash \Delta'}{\text{join } jz = \{yz/x\}N \text{ in } M :: \Gamma, y: A \rightarrow B, \Gamma' \vdash \Delta, p \cdot \Delta'} \rightarrow L \\
\\
\frac{M :: x: A, y: B, \Gamma \vdash \Delta}{\{\pi_1 z/x\}\{\pi_2 z/y\}M :: z: A \wedge B, \Gamma \vdash \Delta} \wedge L \\
\\
\frac{M :: \Gamma \vdash \Delta, i: p \cdot A \quad N :: \Gamma' \vdash \Delta', j: q \cdot B}{\text{join } ix = (\text{join } jy = k(x, y) \text{ in } N) \text{ in } M :: \Gamma, \Gamma' \vdash \Delta, p \cdot \Delta', k: pq \cdot A \wedge B} \wedge R \\
\\
\frac{M :: \Gamma \vdash \Delta, i: p \cdot A, j: q \cdot B}{\{k(\iota_1 P)/iP\}\{k(\iota_2 P)/jP\}M :: \Gamma \vdash \Delta, k: p + q \cdot [\frac{p}{p+q} \cdot A, \frac{q}{p+q} \cdot B]} \vee R \\
\\
\frac{M :: x: A, \Gamma \vdash \Delta \quad N :: y: B, \Gamma' \vdash \Delta'}{\text{case } z \text{ of } (\iota_1 x \rightarrow M, \iota_2 y \rightarrow N) :: z: [p \cdot A, q \cdot B], \Gamma, \Gamma' \vdash p \cdot \Delta, q \cdot \Delta'} \vee L \\
\\
\frac{M :: x: A, y: A, \Gamma \vdash \Delta}{\{z/x\}\{z/y\}M :: z: A, \Gamma \vdash \Delta} \text{CL} \\
\\
\frac{M :: \Gamma \vdash \Delta, i: p \cdot A, j: q \cdot A}{\{k/i\}\{k/j\}M :: \Gamma \vdash \Delta, k: p + q \cdot A} \text{CR} \\
\\
\frac{M :: x: A, \Gamma \vdash \Delta, j: p \cdot A}{\text{corec } jxM :: x: A, \Gamma \vdash \frac{1}{1-p} \Delta} \text{COREC} \quad (p \neq 1)
\end{array}$$

Figure 8-3: Sequent typing rules for the Probabilistic Join Calculus

The idea behind the typing of a co-recursive term is that the non terminating outcome occupies a smaller and smaller proportion of outcomes until it vanishes in the limit. All other outcomes occur repeatedly, in the same proportions, thus the co-recursor can be typed by rescaling the distribution of other outcomes. The one sided nature of weakening is important in this context, as it precludes both the bloating of a term with dummy outcomes which would distort the probabilities, and the inclusion of probability zero outcomes which would destroy the COREC typing rule.

The rule which causes the most problems in this formulation is actually $\wedge R$. As noted in the previous chapter this construction does not adhere to the symmetry usually associated with pairing. If you recall we introduced a non-determinism equation to deal with the asymmetry when it caused issues with the derivation reductions. In the probabilistic formulation this asymmetry is apparent immediately in the type system. Since the order of evaluation has a direct impact on the probability of outcomes a choice had to be made as to which distribution to rescale.

The fragment of the non-recursive calculus typed under these rules is exactly the same as defined in our original sequent system.

Proposition 57 *A term M of the non recursive join calculus has a sequent-typing $M :: \Gamma \vdash \Delta$ if and only if it has a probabilistic typing $M :: \Gamma' \vdash \Delta'$ where the variables and co-variables typed by the derivations agree.*

Proof. By direct comparison of typing rules.

It is unnecessary to examine each rule formally, here we use right disjunction as an example, since it represents the change in formulation of the type system from disjunctions to distributions.

The sequent typing rule in this case is:

$$\frac{M :: \Gamma \vdash \Delta, i: A, j: B}{\{k(\iota_1 P)/i P\}\{k(\iota_2 P)/j P\}M :: \Gamma \vdash \Delta, k: A \vee B}^{\vee R}$$

We assume that the conversion holds for the antecedent, thus the sequent-typing $M :: \Gamma \vdash \Delta, i: A, j: B$ is valid if and only if there is a valid probabilistic typing of the form $M :: \Gamma \vdash \Delta, i: p \cdot A, j: q \cdot B$ from which it is possible to derive:

$$\frac{M :: \Gamma \vdash \Delta, i: p \cdot A, j: q \cdot B}{\{k(\iota_1 P)/i P\}\{k(\iota_2 P)/j P\}M :: \Gamma \vdash \Delta, k: p + q \cdot [\frac{p}{p+q} \cdot A, \frac{q}{p+q} \cdot B]}^{\text{VR}}$$

The reverse argument also holds.

□

8.3 Conclusion

In the first part of this thesis we defined a probabilistic type system for the PEA and proved the main result: that it enforces lower bounds on the probability of termination for probabilistic terms. Separately, in Part II, we studied a calculus in which it is possible to encode co-recursive loops. The study of co-recursive loops is important for probabilistic computation, since behaviour in the limit of a probabilistic machine is very different to in a deterministic one. In this final chapter we have given a probabilistic type system for the calculus with co-recursion. The analogous main result for this system would be almost-sure termination of terms with loops.

Conjecture 58 *A probabilistically-typed term of the join-calculus is almost-surely terminating.*

As with the lower bounds achieved in part I, this termination result is not very interesting from a probabilistic perspective: the looping construct is established in such a way that the time to success follows a geometric distribution, thus positive-almost-sure termination is guaranteed. The point of interest is in proving the result through a well designed type system.

We propose that the almost sure termination result would arise as a consequence of establishing a connection between the type system and a reduction/normali-

sation which corresponds to a semantic notion of termination. The type system itself forces the outcome to be a full distribution of types (rather than a sub-distribution) so confirming that the individual outcomes are indeed instances of termination and that the distributions are an accurate assessment of the probability of that outcome should be sufficient to guarantee termination of the typed terms with probability one.

Proving this result would not necessarily be the end of the story. The join-calculus provides fertile ground for further growth towards simplified models of probabilistic computation. As well as the technical results mentioned in Section 7.5 it would be interesting to compare the calculus with symmetric models of computation [12] and two-sided sequential-calculi.

Bibliography

- [1] Melissa Antonelli. Some remarks on counting propositional logic, 2022.
- [2] Melissa Antonelli, Ugo Dal Lago, and Paolo Pistone. Curry and howard meet borel. In *Proceedings of the 37th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS'22)*, pages 45:1–45:13. ACM, 2022.
- [3] Melissa Antonelli, Ugo Dal Lago, and Paolo Pistone. On counting propositional logic. *CoRR*, abs/2103.12862, 2021.
- [4] Martin Avanzini, Ugo Dal Lago, and Akihisa Yamada. On probabilistic term rewriting. In John P. Gallagher and Martin Sulzmann, editors, *Proc. Functional and Logic Programming - 14th International Symposium, FLOPS 2018*, volume 10818 of *Lecture Notes in Computer Science*, pages 132–148. Springer, 2018.
- [5] Gianluigi Bellin and Willem B. Heijltjes. Proof nets for bi-intuitionistic linear logic. In *International Conference on Formal Structures for Computation and Deduction*, 2018.
- [6] Raven Beutner and Luke Ong. On probabilistic termination of functional programs with continuous distributions. In Stephen N. Freund and Eran Yahav, editors, *Proceedings 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI)*, pages 1312–1326. ACM, 2021.
- [7] G.M. Bierman. A note on full intuitionistic linear logic. *Annals of Pure and Applied Logic*, 79(3):281–287, 1996.

- [8] Guy Blelloch and John Greiner. Parallelism in sequential functional languages. In *Proceedings of the Seventh International Conference on Functional Programming Languages and Computer Architecture*, FPCA '95, pages 226–237, New York, NY, USA, 1995. Association for Computing Machinery.
- [9] Stephen L. Bloom and Zoltan Esik. *Iteration Theories: The Equational Logic of Iterative Processes*. Springer Publishing Company, Incorporated, 1st edition, 1993.
- [10] Flavien Breuvar and Ugo Dal Lago. On intersection types and probabilistic lambda calculi. In *Proceedings of the 20th International Symposium on Principles and Practice of Declarative Programming, PPDP 2018*, pages 8:1–8:13. ACM, 2018.
- [11] Alonzo Church. *The Calculi of Lambda Conversion. (AM-6)*. Princeton University Press, 1941.
- [12] Pierre-Louis Curien and Hugo Herbelin. The duality of computation. *SIGPLAN Not.*, 35(9):233–243, September 2000.
- [13] Fredrik Dahlqvist and Dexter Kozen. Semantics of higher-order probabilistic programs with conditioning. *Proceedings of the ACM on Programming Languages*, 4(POPL):57:1–57:29, 2020.
- [14] Ugo Dal Lago and Charles Grellois. Probabilistic termination by monadic affine sized typing. *ACM Transactions on Programming Languages and Systems*, 41(2):10:1–10:65, 2019.
- [15] Ugo Dal Lago, Giulio Guerrieri, and Willem Heijltjes. Decomposing probabilistic lambda-calculi. In *Proceedings of Foundations of Software Science and Computation Structures (FoSSaCS)*, volume 12077 of *LNCS*, pages 136–156, 2020.
- [16] Ugo Dal Lago and Margherita Zorzi. Probabilistic operational semantics for the lambda calculus. *RAIRO - Theoretical Informatics and Applications*, 46(3):413–450, 2012.

- [17] Vincent Danos and Thomas Ehrhard. Probabilistic coherence spaces as a model of higher-order probabilistic computation. *Information and Computation*, 209(6):966–991, 2011.
- [18] Valeria Correa Vaz de Paiva. The dialectica categories. Technical report, University of Cambridge, Computer Laboratory, 1991.
- [19] Ugo de’Liguoro and Adolfo Piperno. Non deterministic extensions of untyped lambda-calculus. *Information and Computation*, 122(2):149–177, 1995.
- [20] Rémi Douence and Pascal Fradet. A systematic study of functional language implementations. *ACM Transactions on Programming Languages and Systems*, 20(2):344–387, 1998.
- [21] Paul Downen, Luke Maurer, Zena M. Ariola, and Simon Peyton Jones. Sequent calculus as a compiler intermediate language. *SIGPLAN Not.*, 51(9):74–88, September 2016.
- [22] Paul Downen, Zachary Sullivan, Zena M. Ariola, and Simon Peyton Jones. Codata in action. In Luís Caires, editor, *Programming Languages and Systems*, pages 119–146, Cham, 2019. Springer International Publishing.
- [23] Thomas Ehrhard and Giulio Guerrieri. The bang calculus: An untyped lambda-calculus generalizing call-by-name and call-by-value. In *Proceedings of the 18th International Symposium on Principles and Practice of Declarative Programming (PPDP’16)*, pages 174–187, 2016.
- [24] Thomas Ehrhard and Christine Tasson. Probabilistic call by push value. *Logical Methods in Computer Science*, 15(1), 2019.
- [25] Thomas Ehrhard, Christine Tasson, and Michele Pagani. Probabilistic coherence spaces are fully abstract for probabilistic pcf. In *Proceedings of the 41st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL ’14, pages 309–320, New York, NY, USA, 2014. Association for Computing Machinery.
- [26] Claudia Faggian and Simona Ronchi Della Rocca. Lambda calculus and

- probabilistic computation. In *34th Annual ACM/IEEE Symposium on Logic in Computer Science, LICS 2019*, pages 1–13, 2019.
- [27] Tobias Fritz. A synthetic approach to markov kernels, conditional independence and theorems on sufficient statistics. *Advances in Mathematics*, 370:107239, August 2020.
 - [28] Robin O Gandy. Proofs of strong normalisation. In J Seldin and J R Hindley, editors, *To H.B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*, pages 457–477. Academic Press, 1980.
 - [29] G Gentzen. 3. investigations into logical deduction. In M.E. Szabo, editor, *The Collected Papers of Gerhard Gentzen*, volume 55 of *Studies in Logic and the Foundations of Mathematics*, pages 68–131. Elsevier, 1969.
 - [30] John T. Gill. Computational complexity of probabilistic turing machines. In *Proceedings of the Sixth Annual ACM Symposium on Theory of Computing*, STOC '74, pages 91–95, New York, NY, USA, 1974. Association for Computing Machinery.
 - [31] Jean Goubault-Larrecq. A probabilistic and non-deterministic call-by-push-value language. In *34th Annual ACM/IEEE Symposium on Logic in Computer Science, LICS 2019*, pages 1–13. IEEE Computer Society, 2019.
 - [32] Timothy Griffin. A formulae-as-types notion of control. *Principles of Programming Languages (POPL '90)*, 03 2000.
 - [33] Masahito Hasegawa. Decomposing typed lambda calculus into a couple of categorical programming languages. In David Pitt, David E. Rydeheard, and Peter Johnstone, editors, *Category Theory and Computer Science*, pages 200–219, Berlin, Heidelberg, 1995. Springer Berlin Heidelberg.
 - [34] Willem Heijltjes. The functional machine calculus. In *Proceedings of the 38th Conference on the Mathematical Foundations of Programming Semantics, MFPS XXXVIII*, volume 1 of *ENTICS*, 2022.
 - [35] M.C.B. Hennessy and E.A. Ashcroft. A mathematical semantics for a nonde-

- terministic typed λ -calculus. *Theoretical Computer Science*, 11(3):227–245, 1980.
- [36] Martin Hyland and Valeria de Paiva. Full intuitionistic linear logic (extended abstract) presented at the 9th international congress of logic, methodology and philosophy of science held in uppsala, sweden, august 7-14, 1991. *Annals of Pure and Applied Logic*, 64(3):273–291, 1993.
- [37] Bart Jacobs and Jan J. M. M. Rutten. A tutorial on (co)algebras and (co)induction. *Bulletin of The European Association for Theoretical Computer Science*, 1997.
- [38] C. Jones and Gordon D. Plotkin. A probabilistic powerdomain of evaluations. In *Proceedings of the Fourth Annual Symposium on Logic in Computer Science (LICS '89)*, pages 186–195. IEEE Computer Society, 1989.
- [39] Achim Jung and Regina Tix. The troublesome probabilistic powerdomain. *Electronic Notes in Theoretical Computer Science*, 13:70–91, 1998.
- [40] G. A. Kavvos, Edward Morehouse, Daniel R. Licata, and Norman Danner. Recurrence extraction for functional programs through call-by-push-value. *Proc. ACM Program. Lang.*, 4(POPL), December 2019.
- [41] Naoki Kobayashi, Ugo Dal Lago, and Charles Grellois. On the termination problem for probabilistic higher-order recursive programs. In *Proceedings of the 34th Annual ACM/IEEE Symposium on Logic in Computer Science, LICS 2019*, pages 1–14. IEEE, 2019.
- [42] Jean-Louis Krivine. A call-by-name lambda-calculus machine. *Higher-Order and Symbolic Computation*, 20(3):199–207, 2007.
- [43] Ugo Dal Lago. On probabilistic λ -calculi. *Foundations of Probabilistic Programming*, 2020.
- [44] Ugo Dal Lago, Claudia Faggian, and Simona Ronchi Della Rocca. Intersection types and (positive) almost-sure termination. *Proceedings of the ACM on Programming Languages*, 5(POPL):1–32, 2021.

- [45] P. J. Landin. The mechanical evaluation of expressions. *The Computer Journal*, 6(4):308–320, 01 1964.
- [46] Peter J. Landin. The next 700 programming languages. *Commun. ACM*, 9:157–166, 1966.
- [47] Thomas Leventis. A deterministic rewrite system for the probabilistic λ -calculus. *Mathematical Structures in Computer Science*, 29(10):1479–1512, 2019.
- [48] Paul Blain Levy. *Call-by-push-value: A functional/imperative synthesis*, volume 2 of *Semantic Structures in Computation*. Springer Netherlands, 2003.
- [49] Sheng Liang, Paul Hudak, and Mark Jones. Monad transformers and modular interpreters. In *Proceedings of the 22nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL ’95, pages 333–343, New York, NY, USA, 1995. Association for Computing Machinery.
- [50] Christoph Lüth and Neil Ghani. Composing monads using coproducts. *SIGPLAN Not.*, 37(9):133–144, September 2002.
- [51] Udi Manber and Martin Tompa. Probabilistic, nondeterministic, and alternating decision trees. In *14th Annual ACM Symposium on Theory of Computing*, pages 234–244, 1982.
- [52] Luke Maurer, Paul Downen, Zena M. Ariola, and Simon Peyton Jones. Compiling without continuations. *SIGPLAN Not.*, 52(6):482–494, June 2017.
- [53] Conor McBride and Ross Paterson. Applicative programming with effects. *Journal of Functional Programming*, 18(1):1–13, 2008.
- [54] Étienne Miquey. A classical sequent calculus with dependent types. *ACM Trans. Program. Lang. Syst.*, 41(2), March 2019.
- [55] Eugenio Moggi. Notions of computation and monads. *Information and Computation*, 93(1):55–92, 1991. Selections from 1989 IEEE Symposium on Logic in Computer Science.

- [56] Michel Parigot. $\lambda\mu$ -Calculus: An algorithmic interpretation of classical natural deduction. In *Logic Programming and Automated Reasoning*, pages 190–201. Springer-Verlag, Berlin/Heidelberg, 2005.
- [57] Daniel Pless and George Luger. Toward general analysis of recursive probability models, 2013.
- [58] G.D. Plotkin. Call-by-name, call-by-value and the λ -calculus. *Theoretical Computer Science*, 1(2):125–159, 1975.
- [59] Gordon D Plotkin. A structural approach to operational semantics. 1981.
- [60] A.J. Power and Hayo Thielecke. Closed Freyd- and κ -categories. In *International Colloquium on Automata, Languages, and Programming (ICALP)*, volume 1644 of *Lecture Notes in Computer Science*, pages 625–634. Springer, 1999.
- [61] Dag Prawitz. Ideas and results in proof theory. In J.E. Fenstad, editor, *Proceedings of the Second Scandinavian Logic Symposium*, volume 63 of *Studies in Logic and the Foundations of Mathematics*, pages 235–307. Elsevier, 1971.
- [62] Norman Ramsey and Avi Pfeffer. Stochastic lambda calculus and monads of probability distributions. In *Conference Record of POPL 2002: The 29th SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '02, pages 154–165, 2002.
- [63] Nasser Saheb-Djahromi. Probabilistic LCF. In *Mathematical Foundations of Computer Science 1978, Proceedings, 7th Symposium*, volume 64 of *Lecture Notes in Computer Science*, pages 442–451. Springer, 1978.
- [64] David Sands. *Calculi for time analysis of functional programs*. PhD thesis, University of London, 1990.
- [65] Davide Sangiorgi and Valeria Vignudelli. Environmental bisimulations for probabilistic higher-order languages. In *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL 2016, pages 595–607, 2016.

- [66] Walter J. Savitch. Deterministic simulation of non-deterministic turing machines (detailed abstract). In *Proceedings of the First Annual ACM Symposium on Theory of Computing*, STOC '69, pages 247–248, New York, NY, USA, 1969. Association for Computing Machinery.
- [67] A. Simpson and G. Plotkin. Complete axioms for categorical fixed-point operators. In *Proceedings Fifteenth Annual IEEE Symposium on Logic in Computer Science (Cat. No.99CB36332)*, pages 30–41, 2000.
- [68] Christopher Strachey and Christopher P. Wadsworth. Continuations a mathematical semantics for handling full jumps. Technical Report PRG11, OUCL, January 1974.
- [69] W. W. Tait. Intensional interpretations of functionals of finite type i. *The Journal of Symbolic Logic*, 32(2):198–212, 1967.
- [70] Hayo Thielecke. Continuations, functions and jumps. *SIGACT News*, 30(2):33–42, June 1999.
- [71] A. M. Turing. On computable numbers, with an application to the entscheidungsproblem. *Proceedings of the London Mathematical Society*, s2-42(1):230–265, 1937.
- [72] Jaco van de Pol. Two different strong normalization proofs?: Computability versus functional of finite type. In *Higher-Order Algebra, Logic, and Term Rewriting*, pages 201–220. Springer Berlin Heidelberg, Berlin, Heidelberg, 1996.
- [73] A.N. Whitehead and B. Russell. *Principia Mathematica*. Cambridge University Press, 1927.

Appendix A

Machine Semantics for the Join Calculus

To further understand the semantics of the calculus we consider an abstract machine and show that a run of such a machine can be captured by the reduction relation. The machine has two stacks; one takes a mixture of value terms, operators and choices, while the other consists of co-(variable)-valuations. For term evaluation the machine starts with a computation term and an empty first stack, but at intermediary stages -when the first stack is non empty- it operates on value terms.

$$\begin{array}{c}
 \frac{(\ S \ , \ E \ , \ PQ \)}{(\ SQ \ , \ E \ , \ P \)} \qquad \frac{(\ \varepsilon \ , \ E \ , \ \text{join } jx = N \text{ in } M \)}{(\ \varepsilon \ , \ E[jx = N] \ , \ M \)} \\
 \\
 \frac{(\ SQ \ , \ E \ , \ \lambda x. P \)}{(\ S \ , \ E \ , \ \{Q/x\}P \)} \qquad \frac{(\ \varepsilon \ , \ E[jx = N] \ , \ j P \)}{(\ \varepsilon \ , \ E[jx = N] \ , \ \{P/x\}N \)} \\
 \\
 \frac{(\ S \ , \ E \ , \ \pi_i P \)}{(\ S \pi_i \ , \ E \ , \ P \)} \qquad \frac{(\ \varepsilon \ , \ E \ , \ \text{case } P \text{ of } (\iota_1 x \rightarrow M, \iota_2 y \rightarrow N) \)}{(\ (\iota_1 x \rightarrow M, \iota_2 y \rightarrow N) \ , \ E \ , \ P \)} \\
 \\
 \frac{(\ S \pi_i \ , \ E \ , \ (P_1, P_2) \)}{(\ S \ , \ E \ , \ P_i \)} \qquad \frac{(\ (\iota_1 x \rightarrow M_1, \iota_2 x \rightarrow M_2) \ , \ E \ , \ \iota_i P \)}{(\ \varepsilon \ , \ E \ , \ \{P/x\}M_i \)}
 \end{array}$$

We write $(S, E, \mathbf{M}) \Downarrow (S', E', \mathbf{M}')$ if there is a *run*, a sequence of steps of the machine, between the two given states.

The *readback* $\lfloor (S, E, \mathbf{M}) \rfloor$ of a state (S, E, M) is the term given as follows. In effect, it reverses the four machine steps that push to the stack or add to the environment.

$$\begin{aligned} \lfloor (S \mathbf{Q}, E, \mathbf{P}) \rfloor &= \lfloor (S, E, \mathbf{P} \mathbf{Q}) \rfloor \\ \lfloor (S \pi_i, E, \mathbf{P}) \rfloor &= \lfloor (S, E, \pi_i \mathbf{P}) \rfloor \\ \lfloor ((\iota_1 x \rightarrow M_1, \iota_2 x \rightarrow M_2), E, \mathbf{P}) \rfloor &= \lfloor (\varepsilon, E, \mathbf{case} \mathbf{P} \mathbf{of} (\iota_1 x \rightarrow M_1, \iota_2 x \rightarrow M_2)) \rfloor \\ \lfloor (\varepsilon, E[j x = \mathbf{N}], \mathbf{M}) \rfloor &= \lfloor (\varepsilon, E, \mathbf{join} j x = \mathbf{N} \mathbf{in} \mathbf{M}) \rfloor \\ \lfloor (\varepsilon, \varepsilon, \mathbf{M}) \rfloor &= \mathbf{M} \end{aligned}$$

Observe that readback is nondeterministic in the order of **join** constructions.

Proposition 59 *If $(S, E, \mathbf{M}) \Downarrow (S', E', \mathbf{M}')$ then $\lfloor (S, E, \mathbf{M}) \rfloor \rightarrow \lfloor (S', E', \mathbf{M}') \rfloor$.*

Proof. Proceed by induction on the length of run, considering the first step of the run.

Four cases are dealt with directly by the definition of readback.

$$\begin{array}{c} \frac{(S, E, \mathbf{P} \mathbf{Q})}{(S \mathbf{Q}, E, \mathbf{P})} \quad \frac{(\varepsilon, E, \mathbf{join} j x = \mathbf{N} \mathbf{in} \mathbf{M})}{(\varepsilon, E[j x = \mathbf{N}], \mathbf{M})} \\[10pt] \frac{(S, E, \pi_i \mathbf{P})}{(S \pi_i, E, \mathbf{P})} \quad \frac{(\varepsilon, E, \mathbf{case} \mathbf{P} \mathbf{of} (\iota_1 x \rightarrow M, \iota_2 y \rightarrow N))}{((\iota_1 x \rightarrow M, \iota_2 y \rightarrow N), E, \mathbf{P})} \end{array}$$

In these cases $(S, E, \mathbf{M}) \Downarrow (T, F, \mathbf{N}) \Downarrow (S', E', \mathbf{M}')$ and $\lfloor (S, E, \mathbf{M}) \rfloor = \lfloor (T, F, \mathbf{N}) \rfloor \rightarrow \lfloor (S', E', \mathbf{M}') \rfloor$

Considering those that remain

$$\frac{(S \mathbf{Q}, E, \lambda x. \mathbf{P})}{(S, E, \{Q/x\} \mathbf{P})} \quad \lfloor (S, E, \{Q/x\} \mathbf{P}) \rfloor \rightarrow \lfloor (S', E', \mathbf{M}') \rfloor$$

$$\begin{aligned}
\llbracket (S \ Q, E, \lambda x. P) \rrbracket &= \llbracket (S, E, (\lambda x. P)Q) \rrbracket \\
&= \llbracket (\varepsilon, E, ((\lambda x. P)Q)\Sigma) \rrbracket \\
&= \llbracket (\varepsilon, \varepsilon, \text{join } E \text{ in } ((\lambda x. P)Q)\Sigma) \rrbracket \\
&= \text{join } E \text{ in } ((\lambda x. P)Q)\Sigma \\
&\rightarrow \text{join } E \text{ in } (\{Q/x\}P)\Sigma \\
&= \llbracket (\varepsilon, \varepsilon, \text{join } E \text{ in } (\{Q/x\}P)\Sigma) \rrbracket \\
&= \llbracket (\varepsilon, E, (\{Q/x\}P)\Sigma) \rrbracket \\
&= \llbracket (S, E, \{Q/x\}P) \rrbracket
\end{aligned}$$

$$\frac{(\varepsilon, E[j \ x = N], \quad j \ P)}{(\varepsilon, E[j \ x = N], \ \{P/x\}N)} \quad \llbracket (\varepsilon, E[j \ x = N], \{P/x\}N) \rrbracket \twoheadrightarrow \llbracket (S', E', M') \rrbracket$$

$$\begin{aligned}
\llbracket (\varepsilon, E[j \ x = N], j \ P) \rrbracket &= \llbracket (\varepsilon, E, \text{join } j \ x = N \text{ in } j \ P) \rrbracket \\
&= \llbracket (\varepsilon, \varepsilon, \text{join } E \text{ in } \text{join } j \ x = N \text{ in } j \ P) \rrbracket \\
&= \text{join } E \text{ in } \text{join } j \ x = N \text{ in } j \ P \\
&\rightarrow \text{join } E \text{ in } \{ \text{join } j \ x = N \text{ in } \{Q/x\}N / j \ Q \} j \ P \\
&= \text{join } E \text{ in } \text{join } j \ x = N \text{ in } \{P/x\}N \\
&= \llbracket (\varepsilon, \varepsilon, \text{join } E \text{ in } \text{join } j \ x = N \text{ in } \{P/x\}N) \rrbracket \\
&= \llbracket (\varepsilon, E, \text{join } j \ x = N \text{ in } \{P/x\}N) \rrbracket \\
&= \llbracket (\varepsilon, E[j \ x = N], \{P/x\}N) \rrbracket
\end{aligned}$$

$$\frac{(S \ \pi_i, \ E, \ (P_1, P_2))}{(S, \ E, \ P_i)} \quad \llbracket (S, E, P_i) \rrbracket \twoheadrightarrow \llbracket (S', E', M') \rrbracket$$

$$\begin{aligned}
\llbracket (S \pi_i, E, (P_1, P_2)) \rrbracket &= \llbracket (S, E, \pi_i (P_1, P_2)) \rrbracket \\
&= \llbracket (\varepsilon, E, \pi_i (P_1, P_2) \Sigma) \rrbracket \\
&= \llbracket (\varepsilon, \varepsilon, \text{join } E \text{ in } \pi_i (P_1, P_2) \Sigma) \rrbracket \\
&= \text{join } E \text{ in } \pi_i (P_1, P_2) \Sigma \\
&\rightarrow \text{join } E \text{ in } P_i \Sigma \\
&= \llbracket (\varepsilon, \varepsilon, \text{join } E \text{ in } P_i \Sigma) \rrbracket \\
&= \llbracket (\varepsilon, E, P_i \Sigma) \rrbracket \\
&= \llbracket (S, E, P_i) \rrbracket
\end{aligned}$$

$$\frac{(\ (\iota_1 x \rightarrow M_1, \iota_2 x \rightarrow M_2) \ , \ E \ , \ \iota_i P \)}{(\ \varepsilon \ , \ E \ , \ \{P/x\} M_i \)} \quad \llbracket (\varepsilon, E, \{P/x\} M_i) \rrbracket \twoheadrightarrow \llbracket (S', E', M') \rrbracket$$

$$\begin{aligned}
\llbracket ((\iota_1 x \rightarrow M_1, \iota_2 x \rightarrow M_2), E, \iota_i P) \rrbracket &= \llbracket (\varepsilon, E, \text{case } \iota_i P \text{ of } (\iota_1 x \rightarrow M_1, \iota_2 x \rightarrow M_2)) \rrbracket \\
&= \llbracket (\varepsilon, \varepsilon, \text{join } E \text{ in case } \iota_i P \text{ of } (\iota_1 x \rightarrow M_1, \iota_2 x \rightarrow M_2)) \rrbracket \\
&= \text{join } E \text{ in case } \iota_i P \text{ of } (\iota_1 x \rightarrow M_1, \iota_2 x \rightarrow M_2) \\
&\rightarrow \text{join } E \text{ in } \{P/x\} M_i \\
&= \llbracket (\varepsilon, \varepsilon, \text{join } E \text{ in } \{P/x\} M_i) \rrbracket \\
&= \llbracket (\varepsilon, E, \{P/x\} M_i) \rrbracket
\end{aligned}$$

□